

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Thiết kế và xây dựng nền tảng cộng đồng game tích
hợp giao tiếp thời gian thực và quản lý giải đấu**

Nguyễn Hữu Thắng

thang.nh225763@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Đỗ Bá Lâm

Khoa: Khoa học máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Thiết kế và xây dựng nền tảng cộng đồng game tích
hợp giao tiếp thời gian thực và quản lý giải đấu

NGUYỄN HỮU THẮNG

thang.nh225763@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Đỗ Bá Lâm _____

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trong suốt thời gian học tập và rèn luyện tại Đại học Bách Khoa, tôi xin chân thành cảm ơn các thầy/cô giáo trong nhà trường đã luôn tạo điều kiện thuận lợi nhất cho sinh viên trong việc học tập và nghiên cứu.

Tôi cũng xin gửi lời cảm ơn chân thành tới tất cả các thầy cô giáo trong Khoa Công nghệ Thông tin cùng các thầy cô bộ môn liên quan đã giảng dạy và truyền đạt những kiến thức bổ ích trong suốt thời gian qua, giúp tôi có thêm hành trang vững chắc để sẵn sàng bước tiếp trên con đường sau này.

Đặc biệt, tôi xin gửi lời cảm ơn sâu sắc tới thầy Đỗ Bá Lâm. Trong suốt thời gian làm đồ án tốt nghiệp vừa qua, thầy đã dành rất nhiều thời gian để theo sát, tận tình hướng dẫn và định hướng giúp tôi hoàn thành đề tài.

Bên cạnh đó, tôi muốn dành những lời tri ân sâu sắc nhất tới gia đình và bạn bè. Cảm ơn gia đình đã luôn là điểm tựa tinh thần vững chắc, hy sinh và tạo mọi điều kiện tốt nhất để tôi có thể yên tâm tu nghiệp. Cảm ơn những người bạn đồng môn đã luôn sát cánh, động viên và hỗ trợ tôi vượt qua những khoảng thời gian áp lực nhất.

Xin chân thành gửi những lời tri ân sâu sắc này tới mọi người.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Sự phát triển của ngành công nghiệp game kéo theo nhu cầu kết nối cộng đồng và tổ chức giải đấu ngày càng lớn. Việc tích hợp liền mạch ba yếu tố: chơi game trực tuyến, giao tiếp thời gian thực và quản lý cộng đồng vào một nền tảng web thống nhất vẫn là bài toán khó. Các giải pháp hiện tại thường phân mảnh, buộc người chơi sử dụng kết hợp nhiều ứng dụng độc lập (như Discord để nhắn tin, web game để chơi, nền tảng khác để xếp lịch đấu), gây bất tiện và gián đoạn trải nghiệm. Nhằm giải quyết triệt để vấn đề này, hướng tiếp cận được lựa chọn là xây dựng một cổng game web (Web Gaming Portal) tích hợp toàn diện. Hướng đi này giúp tối ưu hóa trải nghiệm người dùng ngay trên trình duyệt mà không cần cài đặt thêm phần mềm, đồng thời ứng dụng các công nghệ thời gian thực tiên tiến để giảm thiểu tối đa độ trễ trong quá trình tương tác.

Dựa trên hướng tiếp cận đó, đồ án đã tiến hành phát triển nền tảng cộng đồng game tích hợp giao tiếp thời gian thực và quản lý giải đấu, bao gồm các phân hệ cốt lõi: hệ thống quản lý định danh; mạng xã hội hỗ trợ kết bạn, kênh trò chuyện văn bản và giọng nói; hệ thống máy chủ cộng đồng cho phép phân quyền và quản trị vai trò; cùng hệ thống quản lý giải đấu chuyên nghiệp. Ứng dụng được thiết kế theo kiến trúc đa tầng kết hợp mô hình client-server, sử dụng Go cho backend kết hợp WebSocket và WebRTC để xử lý luồng dữ liệu thời gian thực. Đóng góp chính của đồ án là việc phân tích, thiết kế và triển khai thành công một nền tảng tương tác game thống nhất, có tính khả mở và hiệu năng cao. Kết quả đạt được là một hệ thống phần mềm hoàn chỉnh, chứng minh được khả năng tích hợp sâu các tính năng giao tiếp xã hội và quản lý eSports vào một nền tảng cộng đồng game duy nhất, mang lại giá trị thực tiễn cho cộng đồng người chơi.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

The rapid growth of online games has increased the demand for platforms where players can communicate, form communities, play web-based games, and organize tournaments in a unified environment. In practice, these activities are often separated across different tools: communication platforms for chat and voice, game distribution platforms for publishing and playing games, and tournament platforms for brackets and results. This fragmentation interrupts the user experience, makes community data difficult to manage, and increases the operational effort for small and medium-sized game communities.

This graduation project proposes the design and implementation of a game community platform that integrates real-time communication and tournament management. The system is built as a web-oriented client-server application with core modules for user accounts, friend relationships, community servers, channels, role-based permissions, text messaging, voice communication, game marketplace, embedded HTML game execution, Game SDK integration, and tournament lifecycle management. The backend is developed with Go, while real-time features are supported through WebSocket and WebRTC-based media infrastructure.

The main contribution of the project is an integrated prototype that combines community management, real-time interaction, web game publishing, embedded game communication, and multi-role tournament operation in one ecosystem. The final result demonstrates the feasibility of building a unified platform for independent game developers and game communities, where users can communicate, play, share achievements, and organize competitions without relying on multiple disconnected services.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	3
1.4 Bố cục đồ án	4
CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU.....	5
2.1 Khảo sát hiện trạng	5
2.2 Tổng quan chức năng	8
2.2.1 Biểu đồ use case tổng quát	8
2.2.2 Biểu đồ use case phân rã Quản lý tài khoản	9
2.2.3 Biểu đồ use case phân rã Quản lý bạn bè	10
2.2.4 Biểu đồ use case phân rã Quản lý máy chủ	11
2.2.5 Biểu đồ use case phân rã Quản lý tin nhắn.....	12
2.2.6 Biểu đồ use case phân rã Chơi game và Đăng tải game	13
2.2.7 Biểu đồ use case phân rã Quản lý và Tham gia giải đấu	14
2.2.8 Quy trình nghiệp vụ	15
2.3 Đặc tả chức năng	19
2.3.1 Đặc tả use case Tham gia máy chủ.....	19
2.3.2 Đặc tả use case Quản lý kênh và phân quyền.....	20
2.3.3 Đặc tả use case Đăng tải và kiểm duyệt game	21
2.3.4 Đặc tả use case Tổ chức giải đấu.....	22
2.3.5 Đặc tả use case Vận hành trận đấu và xử lý kết quả.....	23
2.4 Yêu cầu phi chức năng	24

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG.....	26
3.1 Tổng quan công nghệ sử dụng	26
3.2 Frontend	26
3.3 Backend.....	27
3.4 Cơ sở dữ liệu và lưu trữ	27
3.5 Giao tiếp thời gian thực và truyền thông media	28
3.6 Hạ tầng phát triển và triển khai	29
3.7 Kết luận chương	29
CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	30
4.1 Thiết kế kiến trúc.....	30
4.1.1 Lựa chọn kiến trúc phần mềm	30
4.1.2 Thiết kế tổng quan.....	32
4.1.3 Thiết kế chi tiết gói	34
4.2 Thiết kế chi tiết.....	37
4.2.1 Thiết kế giao diện	37
4.2.2 Thiết kế lớp	40
4.2.3 Thiết kế cơ sở dữ liệu	46
4.3 Xây dựng ứng dụng.....	50
4.3.1 Thư viện và công cụ sử dụng	51
4.3.2 Kết quả đạt được	51
4.4 Kiểm thử.....	53
4.5 Triển khai	55
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	60
5.1 Giải pháp phân quyền linh hoạt trong máy chủ và kênh	60
5.1.1 Bài toán đặt ra	60

5.1.2 Giải pháp thực hiện	60
5.1.3 Kết quả đạt được	61
5.2 Giải pháp Game SDK và cầu nối iframe	62
5.2.1 Bài toán đặt ra	62
5.2.2 Giải pháp thực hiện	62
5.2.3 Kết quả đạt được	64
5.3 Giải pháp đồng bộ thời gian thực cho nhắn tin và gameplay	64
5.3.1 Bài toán đặt ra	64
5.3.2 Giải pháp thực hiện	64
5.3.3 Kết quả đạt được	65
5.4 Kết luận chương	65
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	67
6.1 Kết luận	67
6.2 Hướng phát triển.....	68
TÀI LIỆU THAM KHẢO.....	70
PHỤ LỤC.....	72
A. ĐẶC TẢ USE CASE TIÊU BIỂU	72
A.1 Danh sách use case đặc tả	72
A.2 Tóm tắt đặc tả use case	72
A.2.1 UC001 – Tham gia máy chủ	72
A.2.2 UC002 – Quản lý kênh và phân quyền.....	72
A.2.3 UC003 – Đăng tải và kiểm duyệt game.....	73
A.2.4 UC004 – Tổ chức giải đấu	73
A.2.5 UC005 – Vận hành trận đấu và xử lý kết quả.....	73
A.3 Bảng trường dữ liệu tham khảo	74

DANH MỤC HÌNH VẼ

Hình 2.1	Biểu đồ use case tổng quát	8
Hình 2.2	Biểu đồ use case phân rã Quản lý tài khoản	9
Hình 2.3	Biểu đồ use case phân rã Quản lý bạn bè	10
Hình 2.4	Biểu đồ use case phân rã Quản lý máy chủ	11
Hình 2.5	Biểu đồ use case phân rã Quản lý tin nhắn	12
Hình 2.6	Biểu đồ use case phân rã Chơi game và Đăng tải game	13
Hình 2.7	Biểu đồ use case phân rã Thiết lập và Tham gia giải đấu	14
Hình 2.8	Biểu đồ use case phân rã Vận hành trận đấu trong giải đấu	15
Hình 2.9	Quy trình nghiệp vụ Quản lý và Tham gia máy chủ	16
Hình 2.10	Quy trình nghiệp vụ Tổ chức và Vận hành giải đấu	17
Hình 2.11	Quy trình nghiệp vụ Đăng tải game	18
Hình 2.12	Đặc tả use case Tham gia máy chủ	20
Hình 2.13	Đặc tả use case Quản lý kênh và phân quyền	21
Hình 2.14	Đặc tả use case Đăng tải và kiểm duyệt game	22
Hình 2.15	Đặc tả use case Tổ chức giải đấu	23
Hình 2.16	Đặc tả use case Vận hành trận đấu và xử lý kết quả	24
Hình 4.1	Kiến trúc đa tầng được áp dụng vào hệ thống	31
Hình 4.2	Biểu đồ phụ thuộc gói ứng dụng Frontend	32
Hình 4.3	Biểu đồ phụ thuộc gói ứng dụng Backend	33
Hình 4.4	Biểu đồ chi tiết gói chức năng Tạo mới máy chủ	35
Hình 4.5	Biểu đồ chi tiết gói chức năng Tổ chức và vận hành giải đấu	36
Hình 4.6	Phác thảo giao diện màn hình máy chủ và kênh	39
Hình 4.7	Phác thảo giao diện danh sách game và đăng tải game	39
Hình 4.8	Phác thảo giao diện màn hình giải đấu	40
Hình 4.9	Thiết kế lớp chức năng Tạo giải đấu	41
Hình 4.10	Biểu đồ trình tự use case Tạo giải đấu	46
Hình 4.11	Sơ đồ E-R cụm dữ liệu cộng đồng và máy chủ	47
Hình 4.12	Sơ đồ E-R cụm dữ liệu game	48
Hình 4.13	Sơ đồ E-R cụm dữ liệu giải đấu	49
Hình 4.14	Mô hình triển khai thử nghiệm tự host với ngrok và LiveKit	56
Hình 5.1	Mô hình Game SDK và cầu nối iframe	63

DANH MỤC BẢNG BIỂU

Bảng 4.1	Thiết kế chi tiết lớp Tournament	42
Bảng 4.2	Thiết kế chi tiết lớp TournamentCreateEditDialog	43
Bảng 4.3	Thiết kế chi tiết lớp TournamentService	44
Bảng 4.4	Thiết kế chi tiết interface TournamentRepository	45
Bảng 4.5	Tổng hợp các cụm dữ liệu chính của hệ thống	47
Bảng 4.6	Danh sách thư viện và công cụ sử dụng	51
Bảng 4.7	Các sản phẩm xây dựng và đóng gói	52
Bảng 4.8	Thông kê tổng quan mã nguồn	52
Bảng 4.9	Thông kê theo nhóm thư mục chính	53
Bảng 4.10	Kết quả kiểm thử chức năng xác thực và hồ sơ người dùng . .	54
Bảng 4.11	Kết quả kiểm thử chức năng máy chủ và kênh	54
Bảng 4.12	Kết quả kiểm thử chức năng nhắn tin và đồng bộ thời gian thực	55
Bảng 4.13	Tổng hợp kết quả kiểm thử	55
Bảng 4.14	Cấu hình môi trường triển khai thử nghiệm	57
Bảng 4.15	Danh sách container trong môi trường triển khai	57
Bảng 4.16	Kịch bản JMeter mô phỏng 10 người dùng đồng thời	58
Bảng 4.17	Kết quả mô phỏng triển khai với JMeter và LiveKit	58
Bảng A.1	Danh sách use case đặc tả tiêu biểu	72
Bảng A.2	Các trường dữ liệu tham khảo trong đặc tả use case	74

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng
CRUD	Nhóm thao tác tạo, đọc, cập nhật và xóa dữ liệu
CSS	Ngôn ngữ định kiểu giao diện web
ERD	Biểu đồ thực thể – liên kết
HTML	Ngôn ngữ đánh dấu siêu văn bản
JWT	Chuẩn token dùng để truyền thông tin xác thực dạng JSON
ORM	Kỹ thuật ánh xạ đối tượng trong chương trình với bảng trong cơ sở dữ liệu quan hệ
RBAC	Kiểm soát truy cập dựa trên vai trò
REST	Kiểu kiến trúc truyền trạng thái đại diện
SDK	Bộ công cụ phát triển phần mềm
SQL	Ngôn ngữ truy vấn có cấu trúc
UI	Giao diện người dùng
UML	Ngôn ngữ mô hình hóa thống nhất
UX	Trải nghiệm người dùng
WebRTC	Công nghệ giao tiếp âm thanh, hình ảnh và dữ liệu thời gian thực trên trình duyệt
WebSocket	Giao thức giao tiếp hai chiều giữa máy khách và máy chủ

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong những năm gần đây, trò chơi điện tử trực tuyến không chỉ còn là một hình thức giải trí cá nhân mà đã trở thành môi trường giao lưu, thi đấu và xây dựng cộng đồng. Người chơi thường tham gia các nhóm theo sở thích, trao đổi chiến thuật, tổ chức trận đấu, chia sẻ thành tích và theo dõi các hoạt động của cộng đồng thông qua nhiều nền tảng khác nhau. Các ứng dụng như Discord cung cấp mô hình máy chủ, kênh trò chuyện, kênh thoại và chia sẻ màn hình, giúp cộng đồng duy trì tương tác thời gian thực. Các nền tảng như Challonge hỗ trợ tạo giải đấu, quản lý người tham gia, bảng đấu và kết quả thi đấu. Trong khi đó, các nền tảng phân phối game như itch.io cho phép nhà phát triển đăng tải trò chơi HTML5 để người dùng có thể chơi trực tiếp trên trình duyệt, còn Steam cung cấp các dịch vụ hỗ trợ như bảng xếp hạng và thành tích cho trò chơi [1], [2], [3], [4].

Tuy nhiên, các giải pháp trên thường tập trung vào từng nhóm chức năng riêng biệt. Một cộng đồng game nhỏ muốn vận hành đầy đủ các hoạt động thường phải sử dụng Discord để giao tiếp, Challonge để tổ chức giải đấu, một nền tảng khác để đăng tải trò chơi, và các dịch vụ riêng để lưu điểm số hoặc đồng bộ trạng thái chơi. Cách tiếp cận phân tán này làm tăng chi phí tích hợp, gây gián đoạn trải nghiệm người dùng và khiến dữ liệu cộng đồng bị chia cắt giữa nhiều hệ thống. Đối với các nhóm phát triển game độc lập hoặc cộng đồng sinh viên, hạn chế này càng rõ rệt vì họ cần một môi trường đơn giản, có thể tùy biến, vừa hỗ trợ giao tiếp, vừa hỗ trợ phát hành và chơi thử game, đồng thời có khả năng tổ chức các hoạt động thi đấu nội bộ.

Từ thực tế đó, bài toán đặt ra là xây dựng một nền tảng tích hợp cho cộng đồng game, trong đó người dùng có thể tạo không gian sinh hoạt chung, trao đổi qua tin nhắn và thoại, tham gia hoặc tổ chức giải đấu, chơi các trò chơi HTML được đăng tải trên hệ thống, đồng thời chia sẻ điểm số, thành tích và trạng thái chơi theo thời gian thực. Việc giải quyết bài toán này có ý nghĩa thực tiễn đối với các cộng đồng game quy mô nhỏ và vừa, các nhóm phát triển trò chơi độc lập, cũng như các môi trường học tập cần một nền tảng thử nghiệm cho các sản phẩm game web. Ngoài lĩnh vực game, hướng tiếp cận này cũng có thể mở rộng cho các hệ thống cộng đồng trực tuyến có nhu cầu kết hợp giao tiếp, nội dung tương tác và quản lý sự kiện.

1.2 Mục tiêu và phạm vi đề tài

Mục tiêu của đề tài là phân tích, thiết kế và xây dựng một nền tảng cộng đồng game tích hợp giao tiếp thời gian thực và quản lý giải đấu. Thay vì chỉ xây dựng một ứng dụng trò chuyện hoặc một hệ thống quản lý giải đấu độc lập, đề tài tập trung vào việc kết hợp nhiều thành phần có liên quan trực tiếp đến trải nghiệm của cộng đồng game, bao gồm quản lý người dùng, quan hệ xã hội, máy chủ, kênh trò chuyện, phân quyền, nhắn tin, thoại, thông báo thời gian thực, kho game, phiên chơi, phòng chơi nhiều người và giải đấu.

Về phía người dùng cuối, hệ thống cần hỗ trợ các chức năng cơ bản như đăng ký, đăng nhập, quản lý hồ sơ cá nhân, kết bạn, chặn người dùng, tham gia máy chủ và tương tác trong các kênh. Trong mỗi máy chủ, người dùng có thể trao đổi qua kênh văn bản, gửi phản hồi, quản lý tin nhắn và theo dõi các nội dung được chia sẻ. Chủ sở hữu hoặc người quản trị máy chủ có thể quản lý thành viên, vai trò và quyền truy cập theo từng nhóm người dùng, từ đó tạo ra không gian cộng đồng có kiểm soát. Đối với giao tiếp thời gian thực, hệ thống hướng tới hỗ trợ kênh thoại, trạng thái người tham gia, ghi hình phòng thoại và phát trực tiếp thông qua hạ tầng media server.

Về phía nội dung game, hệ thống cho phép người dùng tạo thông tin trò chơi, tải lên gói game HTML dưới dạng tệp nén, công bố trò chơi lên khu vực marketplace và khởi chạy trò chơi trực tiếp trong iframe. Đề tài cũng đặt phạm vi xây dựng Game SDK để trò chơi nhúng có thể giao tiếp với ứng dụng chủ thông qua cơ chế postMessage. SDK này hỗ trợ các thao tác như khởi tạo phiên chơi, chia sẻ điểm số, chia sẻ thành tích, tạo và tham gia phòng chơi, đồng bộ trạng thái phòng và truy vấn bảng xếp hạng. Nhờ đó, các trò chơi HTML không chỉ được hiển thị như nội dung tĩnh mà có thể tham gia vào luồng tương tác xã hội của nền tảng.

Về phía tổ chức hoạt động cộng đồng, hệ thống cần hỗ trợ tạo và quản lý giải đấu trong phạm vi máy chủ. Các chức năng chính bao gồm tạo giải đấu, cập nhật thông tin, quản lý trạng thái đăng ký, check-in, danh sách người tham gia, đội thi đấu, trận đấu, nhánh đấu, bảng xếp hạng và lịch sử kết quả. Phạm vi đề tài tập trung vào việc xây dựng nguyên mẫu có đầy đủ các thành phần lõi để chứng minh tính khả thi của kiến trúc tích hợp. Các vấn đề nâng cao như tối ưu vận hành ở quy mô rất lớn, hệ thống chống gian lận chuyên sâu, thanh toán, thương mại hóa nội dung game và đóng gói bộ cài hoàn chỉnh cho desktop chưa phải là trọng tâm chính của đồ án.

1.3 Định hướng giải pháp

Để giải quyết bài toán đã nêu, đề tài lựa chọn định hướng xây dựng một nền tảng web và desktop theo kiến trúc client-server, kết hợp API truyền thống với các cơ chế thời gian thực. Backend được phát triển bằng ngôn ngữ Go, sử dụng Gin để xây dựng REST API, GORM để thao tác cơ sở dữ liệu, MySQL hoặc SQLite để lưu trữ dữ liệu, Redis cho các tác vụ trạng thái và định tuyến thời gian thực, WebSocket cho thông báo và đồng bộ sự kiện, đồng thời tích hợp LiveKit cho chức năng thoại, ghi hình và phát trực tiếp. Frontend được xây dựng bằng React, TypeScript, Tailwind, Vite và Electron, cho phép chia sẻ phần giao diện giữa phiên bản web và desktop.

Giải pháp của đề án được tổ chức theo các miền chức năng chính. Miền cộng đồng quản lý người dùng, quan hệ bạn bè, máy chủ, kênh, vai trò, quyền hạn và tin nhắn. Miền thời gian thực xử lý thông báo, trạng thái kênh thoại, sự kiện phòng game và các cập nhật cần đẩy trực tiếp đến người dùng. Miền game marketplace quản lý vòng đời của trò chơi do người dùng đăng tải, từ tạo metadata, tải lên bản build, kiểm duyệt, hiển thị, đánh giá, tìm kiếm đến khởi chạy phiên chơi. Miền Game SDK đóng vai trò cầu nối giữa trò chơi chạy trong iframe và nền tảng, giúp trò chơi có thể sử dụng các chức năng xã hội của hệ thống mà không cần gọi trực tiếp backend. Miền giải đấu hỗ trợ các nghiệp vụ liên quan đến tổ chức thi đấu, quản lý người tham gia, trận đấu và kết quả.

Lý do lựa chọn hướng tiếp cận này là vì bài toán của đề tài không chỉ nằm ở một chức năng đơn lẻ, mà nằm ở khả năng kết nối nhiều hoạt động thường xuất hiện trong một cộng đồng game. Kiến trúc client-server giúp hệ thống tách biệt rõ ràng giữa giao diện, nghiệp vụ và lưu trữ dữ liệu. REST API phù hợp với các thao tác quản lý có cấu trúc, trong khi WebSocket phù hợp với các sự kiện cần cập nhật nhanh như tin nhắn, thông báo, trạng thái thoại và trạng thái phòng chơi. Việc sử dụng iframe kết hợp SDK giúp nền tảng có thể chạy các game HTML do người dùng đăng tải mà vẫn kiểm soát được luồng giao tiếp giữa game và hệ thống chủ. Bên cạnh đó, việc phát triển cả phiên bản web và desktop giúp mở rộng khả năng sử dụng của hệ thống trong nhiều môi trường khác nhau.

Đóng góp chính của đề án là xây dựng một nguyên mẫu nền tảng cộng đồng game tích hợp, trong đó các chức năng giao tiếp, quản lý cộng đồng, kho game, SDK game realtime và tổ chức giải đấu được thiết kế để hoạt động trong cùng một hệ sinh thái. Kết quả đạt được là hệ thống có backend API theo nhiều miền nghiệp vụ, frontend dùng chung cho web và desktop, cơ chế WebSocket cho thông báo và đồng bộ phòng game, tích hợp LiveKit cho kênh thoại và media, cùng bộ Game SDK hỗ trợ trò chơi HTML tương tác với nền tảng. Đây là cơ sở để tiếp tục phát

triển hệ thống thành một sản phẩm hoàn chỉnh hơn cho cộng đồng game, đặc biệt là các nhóm phát triển độc lập hoặc cộng đồng cần một môi trường vừa giao tiếp, vừa chơi thử, vừa tổ chức thi đấu.

1.4 Bố cục đề án

Phần còn lại của báo cáo đề án tốt nghiệp được tổ chức như sau. Chương 2 trình bày khảo sát hiện trạng và phân tích yêu cầu của hệ thống, bao gồm các nền tảng tương tự, biểu đồ use case, quy trình nghiệp vụ và yêu cầu phi chức năng.

Chương 3 trình bày nền tảng lý thuyết và các công nghệ sử dụng trong đề án. Nội dung chương tập trung vào các nhóm công nghệ chính gồm frontend, backend, cơ sở dữ liệu, giao tiếp thời gian thực và hạ tầng triển khai.

Chương 4 trình bày phân tích thiết kế, triển khai và đánh giá hệ thống, bao gồm lựa chọn kiến trúc, thiết kế gói, thiết kế giao diện, thiết kế lớp, thiết kế cơ sở dữ liệu, công cụ sử dụng và kết quả đạt được.

Chương 5 trình bày các giải pháp và đóng góp nổi bật của đề án, tập trung vào phân quyền máy chủ, đăng tải game cộng đồng, Game SDK, đồng bộ thời gian thực và tổ chức giải đấu nhiều vai trò. Cuối cùng, Chương 6 tổng kết kết quả đạt được, nêu các hạn chế còn tồn tại và đề xuất hướng phát triển tiếp theo cho hệ thống.

CHƯƠNG 2. KHẢO SÁT VÀ PHÂN TÍCH YÊU CẦU

Trong Chương 1, đồ án đã trình bày tổng quan về bài toán xây dựng một nền tảng cộng đồng game trực tuyến, trong đó người dùng có thể giao tiếp, tham gia các không gian cộng đồng, chơi game được đăng tải trên hệ thống và tổ chức các hoạt động thi đấu. Chương này tiếp tục làm rõ cơ sở hình thành các yêu cầu của hệ thống thông qua việc khảo sát hiện trạng, phân tích một số nền tảng tương tự và xác định những chức năng cần phát triển cho nền tảng của đồ án. Nội dung chương được chia thành bốn phần chính. Phần 2.1 trình bày khảo sát hiện trạng, phân tích cách các cộng đồng game hiện nay đang sử dụng những công cụ như Discord, Challonge, itch.io và Steam để phục vụ nhu cầu giao tiếp, phát hành game và tổ chức giải đấu. Phần 2.2 trình bày tổng quan chức năng của hệ thống thông qua các tác nhân và các use case chính. Phần 2.3 đặc tả chi tiết một số use case quan trọng của đồ án. Cuối cùng, phần 2.4 trình bày các yêu cầu phi chức năng liên quan đến hiệu năng, độ tin cậy, bảo mật, khả năng mở rộng và khả năng bảo trì của hệ thống.

2.1 Khảo sát hiện trạng

Trong thực tế, một cộng đồng game trực tuyến thường có nhiều nhu cầu diễn ra song song. Người dùng cần một nơi để trao đổi thông tin, trò chuyện bằng văn bản hoặc giọng nói, chia sẻ nội dung, chơi thử các trò chơi mới, ghi nhận điểm số hoặc thành tích, đồng thời tổ chức các sự kiện thi đấu giữa các thành viên. Nếu không có một nền tảng tích hợp, các hoạt động này thường được thực hiện bằng nhiều công cụ rời rạc. Ví dụ, nhóm người chơi có thể dùng một ứng dụng nhắn tin để trao đổi, dùng bảng tính để ghi danh sách người tham gia, dùng một trang web khác để tạo bảng đấu, dùng liên kết ngoài để chia sẻ game và dùng thêm các kênh riêng để thông báo kết quả. Cách tổ chức này có thể đáp ứng nhu cầu ở quy mô nhỏ, nhưng khi số lượng thành viên, số lượng game hoặc số lượng sự kiện tăng lên thì việc quản lý trở nên khó khăn, dữ liệu bị phân tán và trải nghiệm của người dùng không còn liền mạch.

Đối với nhu cầu giao tiếp cộng đồng, Discord là một trong những nền tảng phổ biến nhất hiện nay [1]. Discord cho phép người dùng tạo server, tổ chức các kênh văn bản và kênh thoại, phân quyền thành viên, gọi thoại, gọi video và chia sẻ màn hình. Mô hình server và channel của Discord phù hợp với các cộng đồng game vì mỗi nhóm có thể tự tổ chức không gian sinh hoạt riêng, chia nhỏ nội dung thảo luận theo từng chủ đề và duy trì kết nối thời gian thực giữa các thành viên. Tuy nhiên, Discord chủ yếu tập trung vào giao tiếp và quản lý cộng đồng. Nền tảng này không được thiết kế như một hệ thống quản lý vòng đời game do người dùng đăng

tải, không cung cấp sẵn cơ chế upload và chạy game HTML như một marketplace độc lập, cũng như không tập trung vào các luồng nghiệp vụ như quản lý phiên chơi, đồng bộ trạng thái phòng chơi, chia sẻ điểm số từ game nhúng hoặc tổ chức giải đấu theo cấu trúc nghiệp vụ riêng của từng cộng đồng.

Đối với nhu cầu tổ chức thi đấu, Challonge là một nền tảng tiêu biểu hỗ trợ tạo giải đấu, quản lý người tham gia, chia bảng, cập nhật kết quả và hiển thị bracket [2]. Các chức năng này giúp ban tổ chức giảm bớt thao tác thủ công khi vận hành một giải đấu, đặc biệt là trong các thể thức như loại trực tiếp hoặc vòng bảng. Challonge phù hợp với các cộng đồng cần một công cụ chuyên về tournament management. Tuy nhiên, nền tảng này lại không đóng vai trò là không gian sinh hoạt chính của cộng đồng. Người dùng vẫn cần sử dụng một công cụ khác để trao đổi, gọi thoại, chia sẻ thông báo, chơi game hoặc lưu lại các tương tác liên quan đến game. Do đó, nếu chỉ sử dụng Challonge, dữ liệu giải đấu và dữ liệu cộng đồng vẫn bị tách rời khỏi nhau.

Đối với nhu cầu phát hành và chơi thử game độc lập, itch.io là một nền tảng phổ biến cho phép nhà phát triển đăng tải trò chơi, trong đó có các game HTML5 có thể chơi trực tiếp trên trình duyệt [3]. Cách tiếp cận này rất phù hợp với các nhóm phát triển game nhỏ vì người dùng không cần cài đặt phần mềm phức tạp để trải nghiệm sản phẩm. Tuy nhiên, itch.io tập trung chủ yếu vào việc phân phối, giới thiệu và lưu trữ game. Các chức năng giao tiếp theo mô hình server, kênh thoại, phân quyền cộng đồng, tổ chức giải đấu nội bộ hoặc đồng bộ trạng thái phòng chơi thời gian thực không phải là trọng tâm chính của nền tảng này. Vì vậy, khi một cộng đồng muốn vừa phát hành game, vừa trao đổi và tổ chức các hoạt động thi đấu xung quanh game đó, itch.io thường cần được kết hợp với các công cụ khác.

Steam và Steamworks cung cấp một hệ sinh thái hoàn chỉnh hơn cho việc phát hành game, quản lý người chơi, thành tích, bảng xếp hạng và một số dịch vụ liên quan đến cộng đồng [4]. Đây là hướng tiếp cận mạnh đối với các sản phẩm game thương mại hoặc các trò chơi đã đạt mức hoàn thiện nhất định. Tuy nhiên, Steam không phù hợp cho mọi ngữ cảnh, đặc biệt là các nhóm phát triển nhỏ, các dự án thử nghiệm, các game HTML đơn giản hoặc các cộng đồng muốn tự kiểm soát toàn bộ luồng đăng tải, chơi thử và tương tác nội bộ. Việc tích hợp Steamworks cũng đòi hỏi trò chơi phải tuân theo hệ sinh thái và quy trình phát hành của Steam, trong khi nhiều nhóm chỉ cần một môi trường nhẹ hơn để thử nghiệm ý tưởng, chia sẻ bản build và tổ chức hoạt động trong phạm vi cộng đồng.

Từ việc khảo sát các nền tảng trên có thể thấy rằng mỗi hệ thống đều giải quyết tốt một phần của bài toán. Discord mạnh về giao tiếp cộng đồng và thời gian thực;

Challonge mạnh về tổ chức giải đấu; itch.io mạnh về đăng tải và chơi game HTML trên trình duyệt; Steam mạnh về phát hành game, thành tích và bảng xếp hạng trong một hệ sinh thái lớn. Tuy nhiên, điểm hạn chế chung là các nền tảng này thường tách rời nhau về mặt dữ liệu và trải nghiệm. Người dùng phải chuyển đổi qua lại giữa nhiều ứng dụng, còn người quản trị cộng đồng phải tự đồng bộ thông tin thủ công giữa các công cụ. Điều này làm giảm tính liên tục của trải nghiệm, gây khó khăn trong việc theo dõi hoạt động của thành viên và làm tăng chi phí quản lý đối với các cộng đồng game quy mô nhỏ và vừa.

Dựa trên những phân tích trên, một nền tảng phù hợp cho cộng đồng game cần kết hợp được các nhóm chức năng chính trong cùng một hệ thống. Trước hết, hệ thống cần có chức năng quản lý tài khoản, hồ sơ người dùng, quan hệ bạn bè và chặn người dùng để tạo nền tảng cho tương tác xã hội. Tiếp theo, hệ thống cần hỗ trợ mô hình server và channel để cộng đồng có thể tổ chức không gian trao đổi theo từng nhóm, từng chủ đề hoặc từng sự kiện. Bên cạnh đó, các chức năng nhắn tin, phản hồi, gửi tệp đính kèm, thông báo thời gian thực, kênh thoại, ghi hình và phát trực tiếp là cần thiết để phục vụ hoạt động giao tiếp thường xuyên của cộng đồng.

Ngoài nhóm chức năng giao tiếp, hệ thống cũng cần có khả năng quản lý game do người dùng đăng tải. Người phát triển game cần tạo thông tin game, tải lên bản build HTML, công bố game, cho phép người khác tìm kiếm, xem chi tiết và khởi chạy game trực tiếp trên trình duyệt. Để game có thể gắn kết với cộng đồng thay vì chỉ là một nội dung tĩnh, hệ thống cần có SDK cho phép game nhúng trao đổi với nền tảng chủ, chẳng hạn như khởi tạo phiên chơi, chia sẻ điểm số, chia sẻ thành tích, tạo phòng chơi, tham gia phòng chơi và đồng bộ trạng thái phòng theo thời gian thực. Đây là điểm khác biệt quan trọng so với cách chỉ đăng tải game lên một kho lưu trữ thông thường.

Cuối cùng, đối với các cộng đồng game có nhu cầu tổ chức thi đấu, hệ thống cần hỗ trợ quản lý giải đấu ngay trong phạm vi server. Các chức năng cần thiết bao gồm tạo giải đấu, cập nhật thông tin, quản lý người tham gia, đội thi đấu, trạng thái đăng ký, check-in, trận đấu, bracket, bảng xếp hạng và lịch sử kết quả. Khi các chức năng này được tích hợp cùng với hệ thống giao tiếp và kho game, người dùng có thể thực hiện phần lớn hoạt động cộng đồng trong một nền tảng thống nhất. Đây cũng chính là cơ sở để đề án lựa chọn phát triển một nguyên mẫu nền tảng cộng đồng game tích hợp, kết hợp các chức năng giao tiếp, marketplace game, SDK realtime và quản lý giải đấu trong cùng một hệ sinh thái.

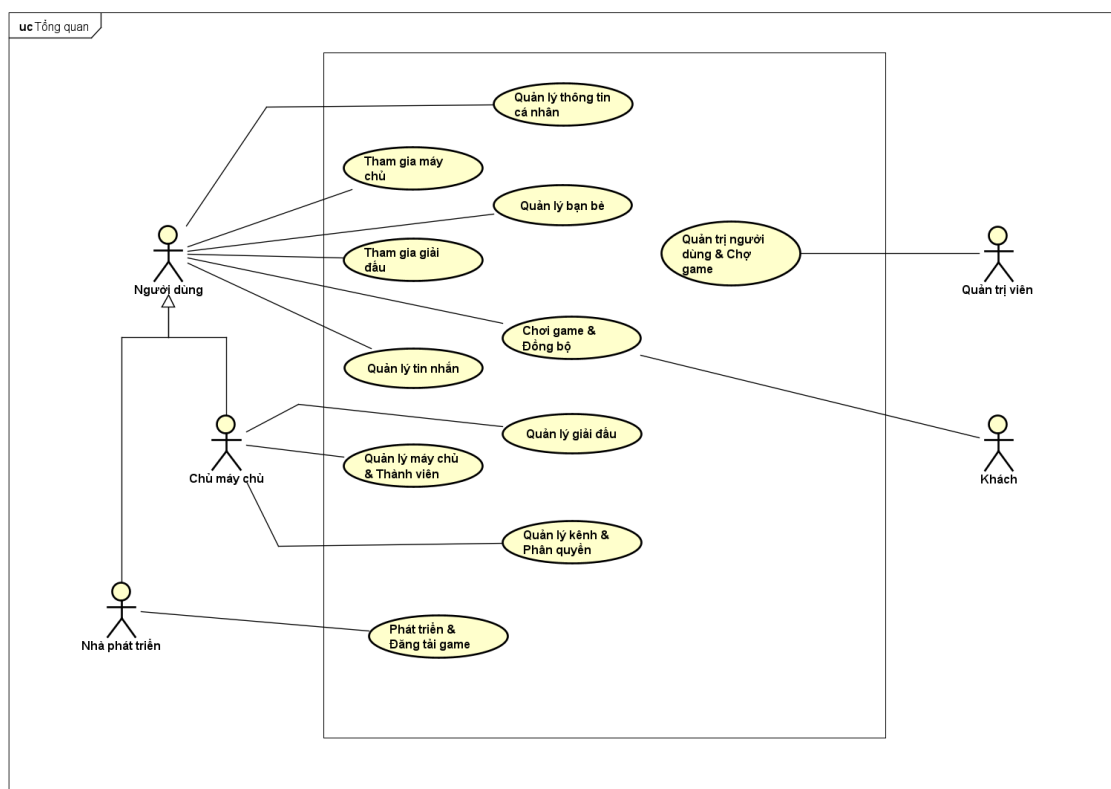
2.2 Tổng quan chức năng

Dựa trên phạm vi đã xác định, hệ thống được xây dựng như một nền tảng cộng đồng game trực tuyến, hỗ trợ người dùng quản lý tài khoản, kết nối bạn bè, tham gia máy chủ, trao đổi tin nhắn, chơi game, đồng bộ trạng thái và tham gia giải đấu. Hệ thống có nhiều nhóm tác nhân khác nhau như khách, người dùng, chủ máy chủ, nhà phát triển và quản trị viên. Mỗi tác nhân được gắn với những nhóm chức năng phù hợp với vai trò của mình trong hệ thống.

Phần này trình bày tổng quan các chức năng thông qua biểu đồ use case tổng quát và hai biểu đồ use case phân rã đầu tiên. Các nội dung ở đây chỉ mô tả chức năng mức cao, chưa đi vào luồng xử lý chi tiết. Phần đặc tả chi tiết cho từng use case quan trọng sẽ được trình bày ở phần 2.3.

2.2.1 Biểu đồ use case tổng quát

Biểu đồ use case tổng quát dưới đây thể hiện các tác nhân chính và các nhóm chức năng lớn của hệ thống.



Hình 2.1: Biểu đồ use case tổng quát

Hình 2.1 xác định năm tác nhân chính gồm Khách, Người dùng, Chủ máy chủ, Nhà phát triển và Quản trị viên. Trong đó, Người dùng là tác nhân trung tâm của hệ thống. Chủ máy chủ và Nhà phát triển là hai vai trò được mở rộng từ Người

dùng, nghĩa là ngoài các chức năng cơ bản của Người dùng, các tác nhân này còn có thêm những chức năng chuyên biệt.

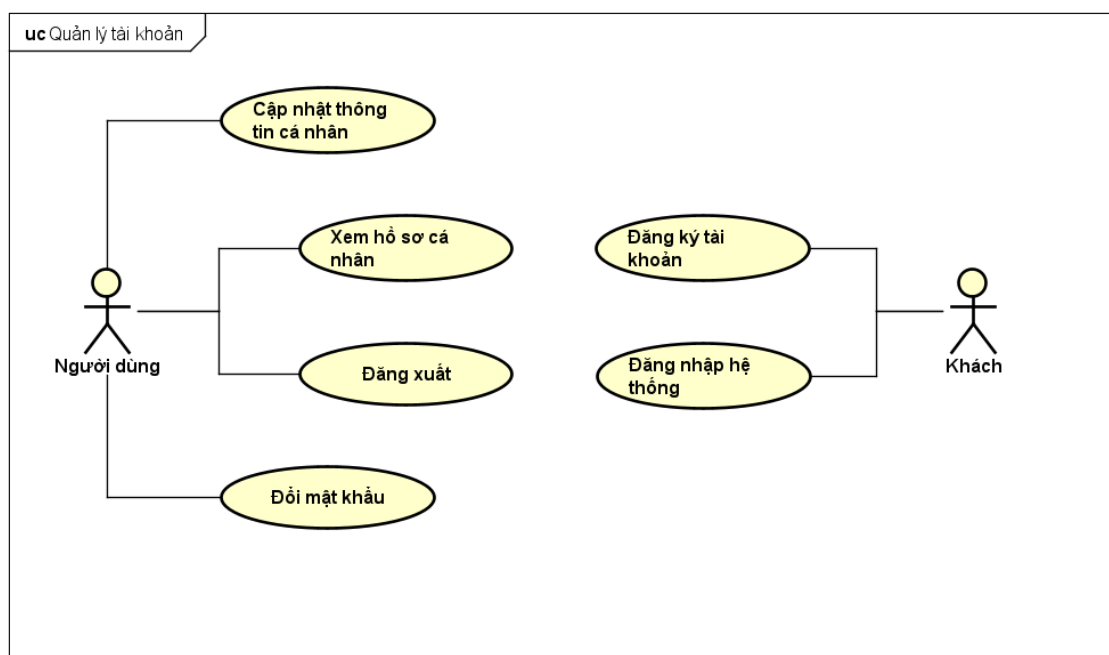
Khách là người chưa đăng nhập hoặc chưa có tài khoản, chỉ có thể sử dụng một số chức năng công khai như Chơi game và Đồng bộ trong phạm vi được hệ thống cho phép. Người dùng sau khi đăng nhập có thể Quản lý thông tin cá nhân, Quản lý bạn bè, Tham gia máy chủ, Quản lý tin nhắn, Tham gia giải đấu và Chơi game, Đồng bộ. Đây là các chức năng cơ bản phục vụ nhu cầu tương tác, giao tiếp và tham gia hoạt động game trong cộng đồng.

Chủ máy chủ có thêm các chức năng Quản lý máy chủ và Thành viên, Quản lý kênh và Phân quyền, đồng thời có thể Quản lý giải đấu trong phạm vi máy chủ. Nhà phát triển thực hiện chức năng Phát triển và Đăng tải game, phục vụ nhu cầu đưa game lên hệ thống để người dùng khác trải nghiệm. Quản trị viên đảm nhiệm nhóm chức năng Quản trị người dùng và Chợ game, nhằm kiểm soát người dùng, nội dung game và các hoạt động quản trị ở cấp toàn hệ thống.

Nhìn chung, biểu đồ tổng quát cho thấy hệ thống có bốn nhóm nghiệp vụ chính: quản lý tài khoản và quan hệ xã hội, quản lý cộng đồng máy chủ, chơi và đăng tải game, quản lý giải đấu và chợ game. Các nhóm chức năng này là cơ sở để tiếp tục phân rã và đặc tả trong các mục sau.

2.2.2 Biểu đồ use case phân rã Quản lý tài khoản

Use case Quản lý tài khoản mô tả các chức năng liên quan đến việc tạo tài khoản, đăng nhập, đăng xuất và quản lý hồ sơ cá nhân.



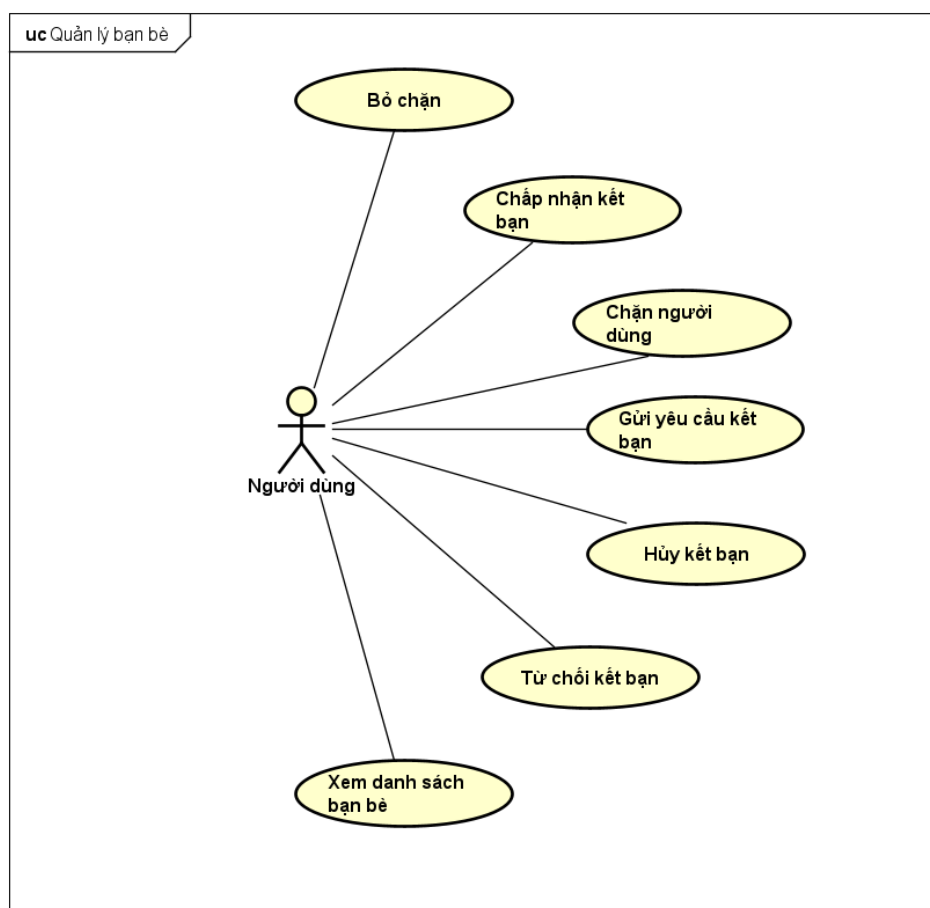
Hình 2.2: Biểu đồ use case phân rã Quản lý tài khoản

Hình 2.2 gồm hai tác nhân là Khách và Người dùng. Khách có thể Đăng ký tài khoản để tạo tài khoản mới hoặc Đăng nhập hệ thống để xác thực và chuyển sang vai trò Người dùng. Đây là hai chức năng đầu vào giúp người dùng bắt đầu sử dụng các chức năng cần xác thực của hệ thống.

Sau khi đăng nhập, Người dùng có thể Xem hồ sơ cá nhân, Cập nhật thông tin cá nhân, Đăng xuất và Đổi mật khẩu. Các chức năng này giúp người dùng kiểm tra thông tin hiển thị, cập nhật hồ sơ, kết thúc phiên làm việc khi cần và đảm bảo an toàn cho tài khoản. Nhóm chức năng Quản lý tài khoản là nền tảng cho các chức năng khác như tham gia máy chủ, gửi tin nhắn, kết bạn, đăng tải game và tham gia giải đấu.

2.2.3 Biểu đồ use case phân rã Quản lý bạn bè

Use case Quản lý bạn bè mô tả các chức năng giúp Người dùng thiết lập và kiểm soát quan hệ xã hội với các tài khoản khác trong hệ thống.



Hình 2.3: Biểu đồ use case phân rã Quản lý bạn bè

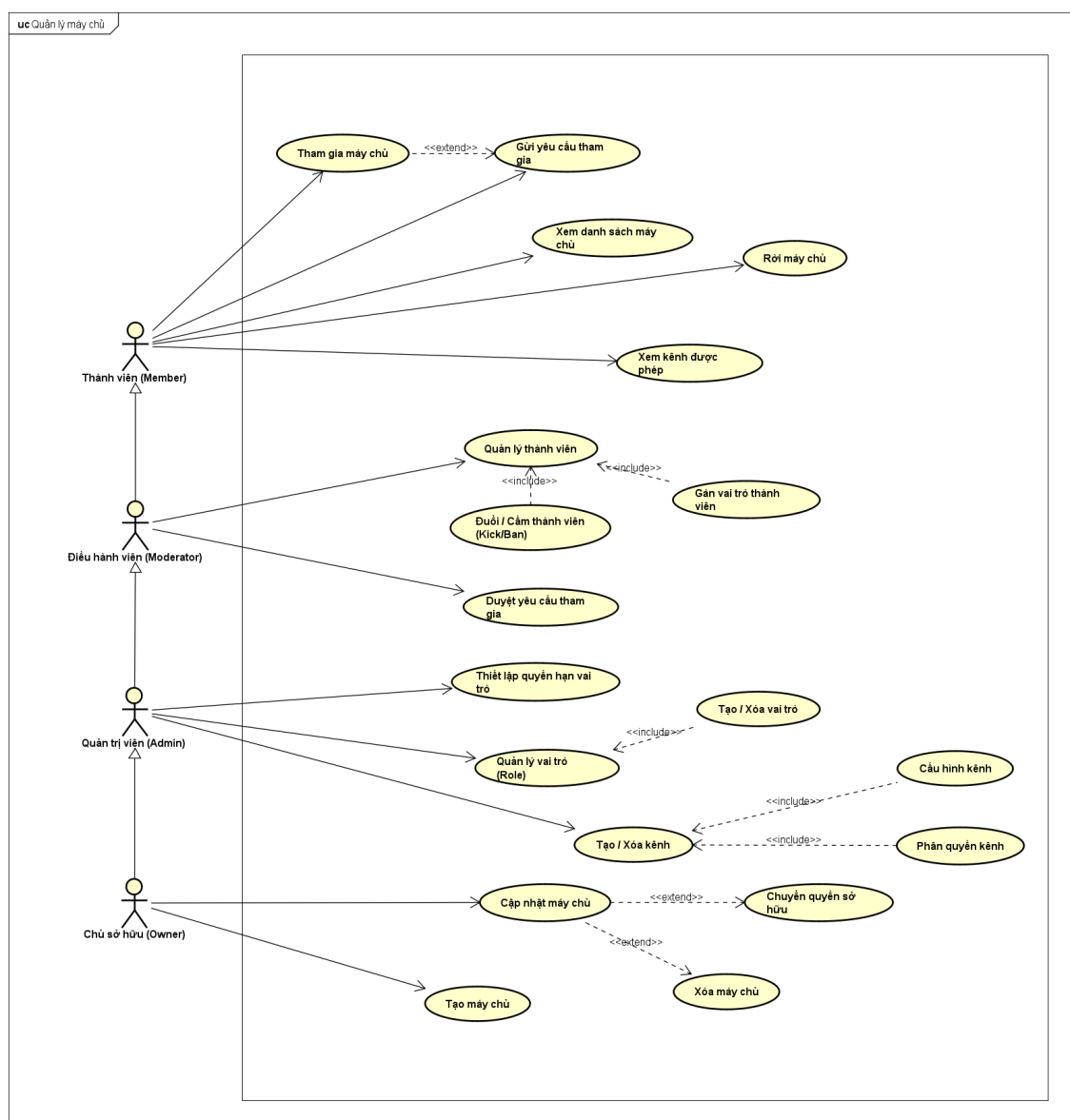
Hình 2.3 cho thấy Người dùng là tác nhân thực hiện toàn bộ các chức năng trong nhóm Quản lý bạn bè. Người dùng có thể Xem danh sách bạn bè để theo dõi các tài khoản đã kết nối, Gửi yêu cầu kết bạn để thiết lập quan hệ mới, Chấp nhận kết

bạn hoặc Từ chối kết bạn khi nhận được yêu cầu từ người khác.

Ngoài ra, Người dùng có thể Hủy kết bạn khi không muốn duy trì quan hệ đã có. Trong trường hợp muốn giới hạn tương tác với một tài khoản, Người dùng có thể Chặn người dùng và sau đó Bỏ chặn nếu muốn khôi phục khả năng tương tác. Nhóm chức năng này giúp người dùng chủ động quản lý mạng lưới quan hệ xã hội, tạo nền tảng cho các hoạt động cộng đồng như nhắn tin, tham gia máy chủ, chơi game cùng nhau và tham gia giải đấu.

2.2.4 Biểu đồ use case phân rã Quản lý máy chủ

Use case Quản lý máy chủ mô tả các chức năng liên quan đến việc người dùng tham gia máy chủ và chủ máy chủ quản lý không gian cộng đồng của mình.



Hình 2.4: Biểu đồ use case phân rã Quản lý máy chủ

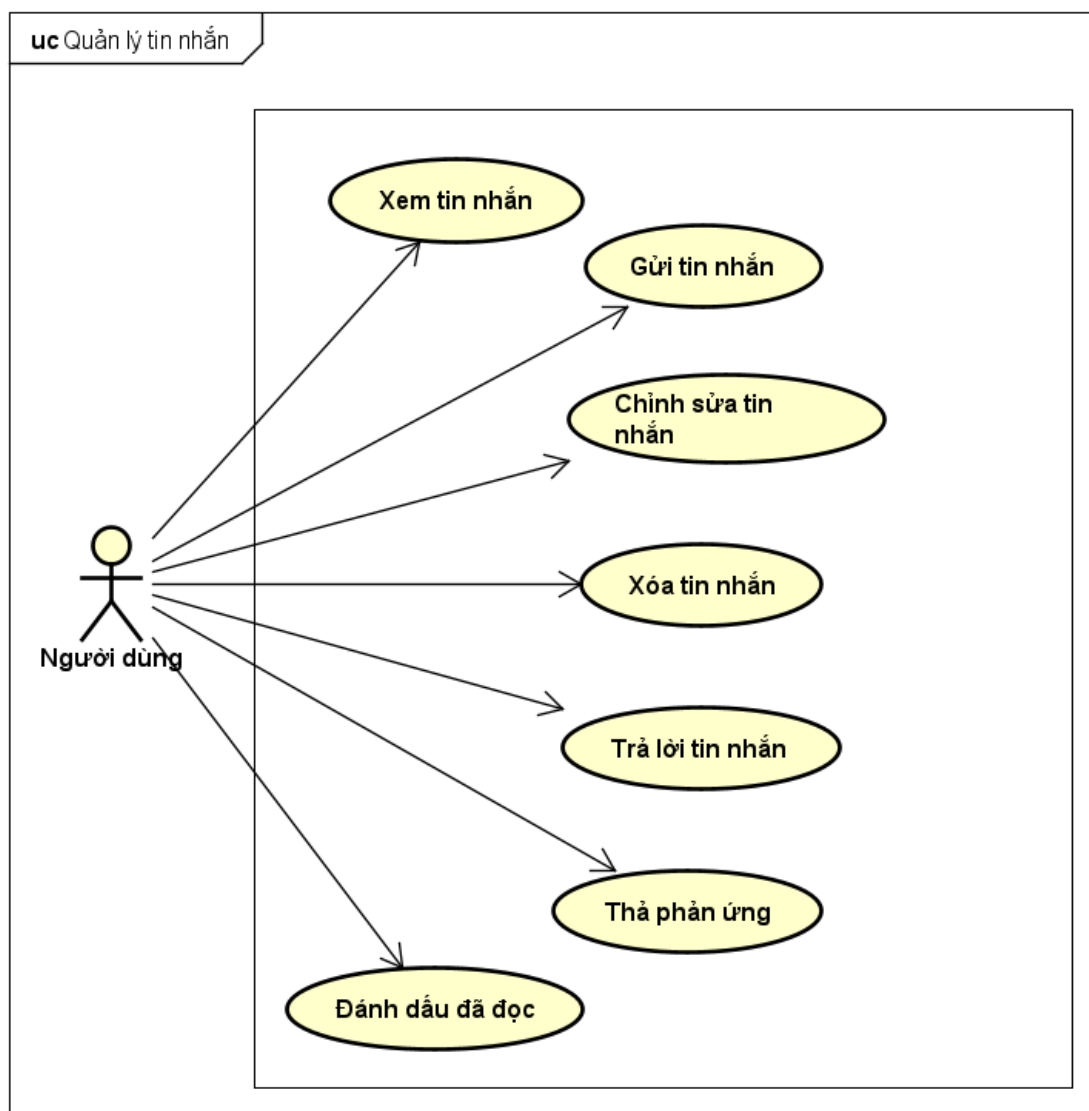
Hình 2.4 cho thấy Người dùng có thể Xem danh sách máy chủ, Tham gia máy

chủ, Gửi yêu cầu tham gia và Rời máy chủ. Các chức năng này giúp người dùng tìm kiếm, gia nhập hoặc rời khỏi các cộng đồng game phù hợp với nhu cầu của mình.

Đối với Chủ máy chủ, hệ thống cung cấp các chức năng Tạo máy chủ, Cập nhật máy chủ, Duyệt yêu cầu tham gia, Quản lý thành viên, Quản lý vai trò, Tạo kênh, Cập nhật kênh và Phân quyền kênh. Nhóm chức năng này giúp chủ máy chủ tổ chức cộng đồng, kiểm soát thành viên và thiết lập phạm vi truy cập cho từng kênh trong máy chủ.

2.2.5 Biểu đồ use case phân rã Quản lý tin nhắn

Use case Quản lý tin nhắn mô tả các chức năng phục vụ hoạt động trao đổi thông tin giữa người dùng trong các kênh của máy chủ.



Hình 2.5: Biểu đồ use case phân rã Quản lý tin nhắn

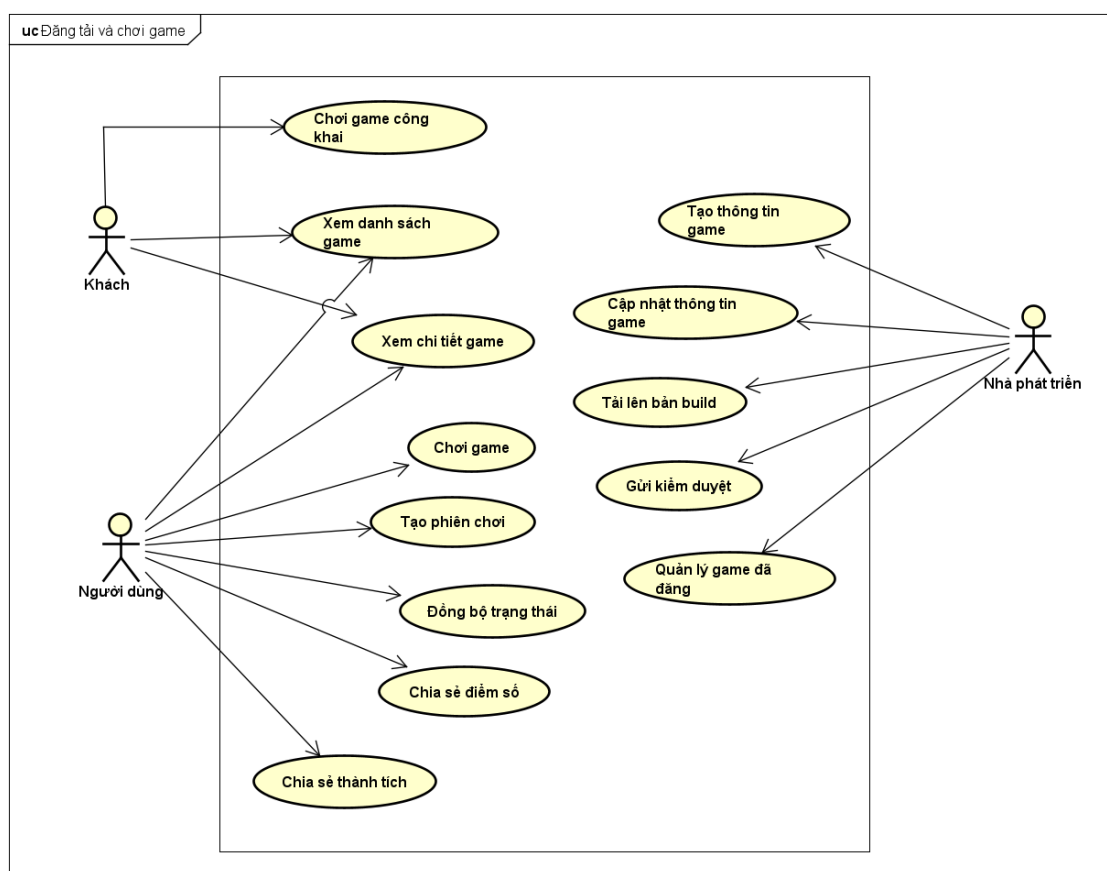
Hình 2.5 thể hiện tác nhân Người dùng với các chức năng Xem tin nhắn và Gửi

tin nhắn. Đây là hai chức năng cơ bản nhất để người dùng theo dõi nội dung trao đổi và tham gia thảo luận trong cộng đồng.

Bên cạnh đó, Người dùng có thể **Chỉnh sửa tin nhắn**, **Xóa tin nhắn**, **Trả lời tin nhắn**, **Thả phản ứng** và **Đánh dấu đã đọc**. Các chức năng bổ sung này giúp quá trình giao tiếp linh hoạt hơn, đồng thời hỗ trợ người dùng quản lý trạng thái đọc và tương tác với nội dung trong kênh.

2.2.6 Biểu đồ use case phân rã Chơi game và Đăng tải game

Use case Chơi game và Đăng tải game mô tả nhóm chức năng liên quan đến việc khám phá, trải nghiệm game và đăng tải game lên hệ thống.



Hình 2.6: Biểu đồ use case phân rã Chơi game và Đăng tải game

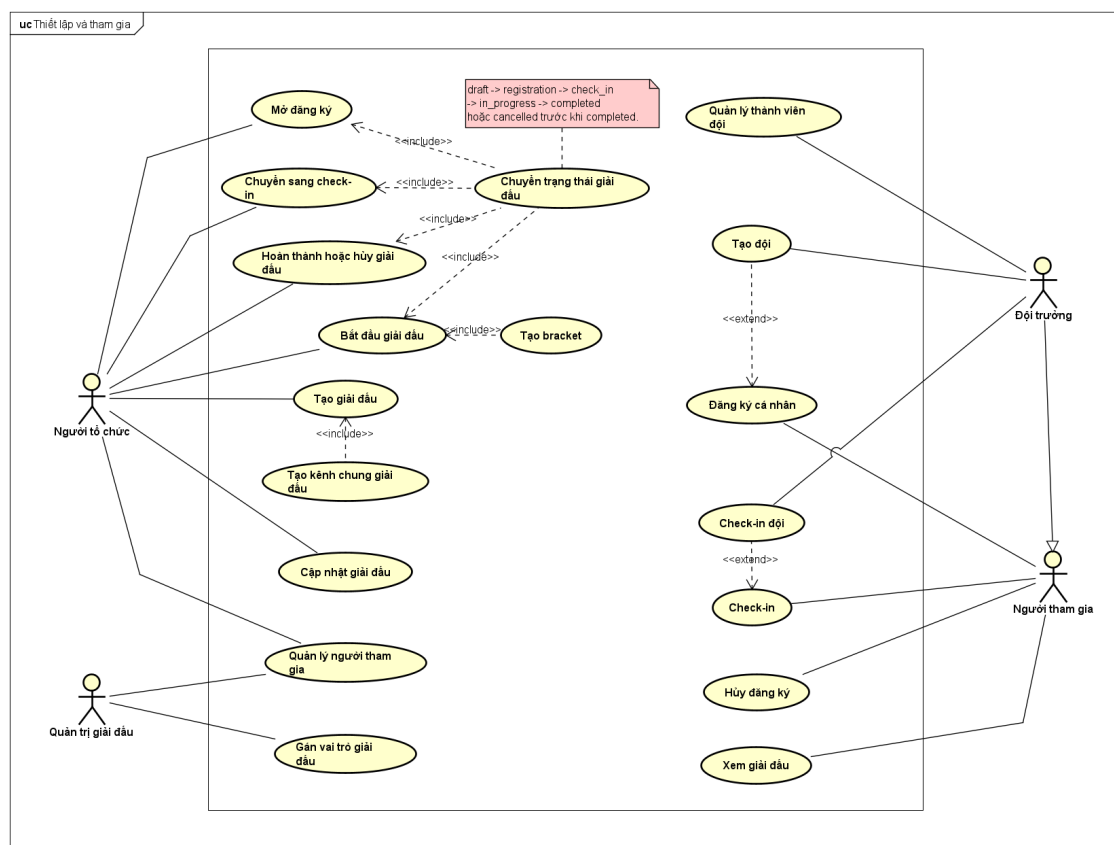
Hình 2.6 cho thấy Khách có thể Xem danh sách game, Xem chi tiết game và Chơi game công khai trong phạm vi được phép. Người dùng sau khi đăng nhập có thể Chơi game, Tạo phiên chơi, Đồng bộ trạng thái, Chia sẻ điểm số và Chia sẻ thành tích. Các chức năng này giúp hoạt động chơi game gắn với tài khoản và tương tác cộng đồng.

Đối với Nhà phát triển, hệ thống hỗ trợ Tạo thông tin game, Cập nhật thông tin game, Tải lên bản build, Gửi kiểm duyệt và Quản lý game đã đăng. Nhóm chức

năng này cho phép nhà phát triển đưa game lên nền tảng, cập nhật nội dung và chuẩn bị game để người dùng khác có thể trải nghiệm.

2.2.7 Biểu đồ use case phân rã Quản lý và Tham gia giải đấu

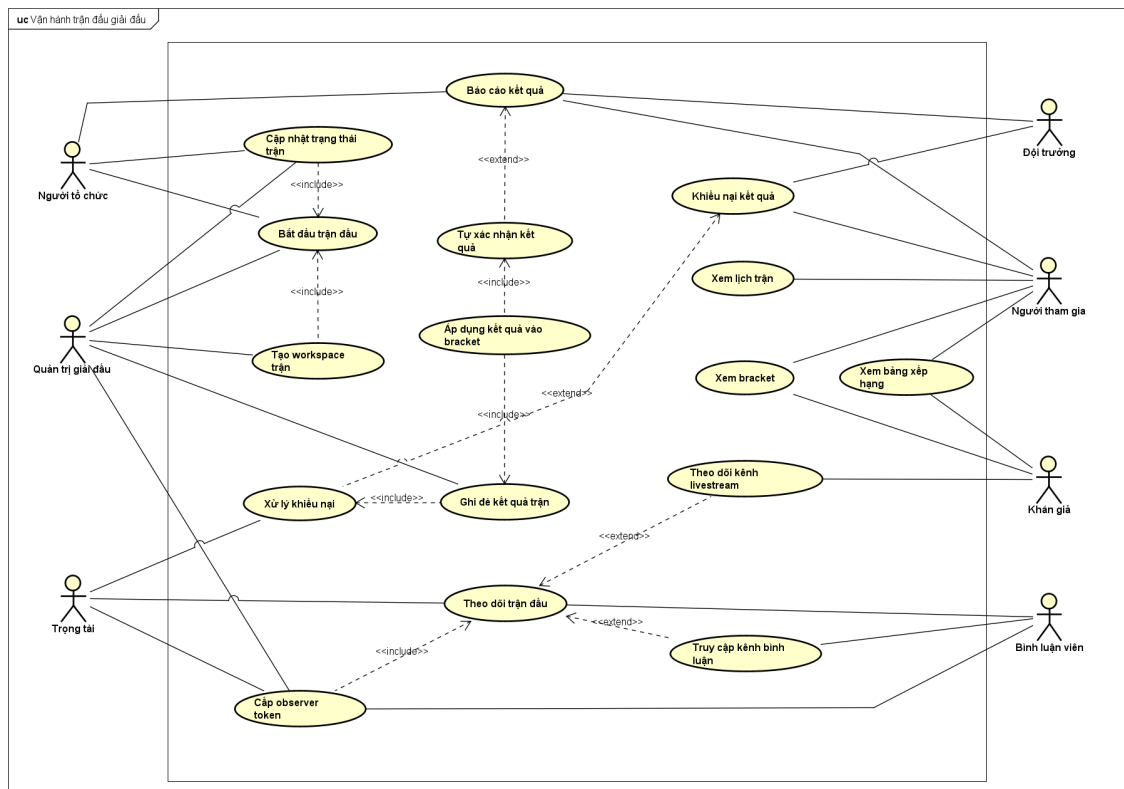
Use case Quản lý và Tham gia giải đấu được tách thành hai biểu đồ để giảm độ phức tạp: nhóm chức năng thiết lập, tham gia giải đấu và nhóm chức năng vận hành trận đấu trong giải đấu.



Hình 2.7: Biểu đồ use case phân rã Thiết lập và Tham gia giải đấu

Hình 2.7 mô tả giai đoạn thiết lập và tham gia giải đấu. Người tổ chức có thể tạo giải đấu, cập nhật thông tin, mở đăng ký, chuyển sang check-in, bắt đầu giải đấu, tạo bracket và kết thúc hoặc hủy giải đấu. Quản trị giải đấu hỗ trợ gán vai trò và quản lý người tham gia trong phạm vi giải đấu.

Người tham gia có thể xem giải đấu, đăng ký cá nhân, hủy đăng ký và check-in khi giải đấu bước vào giai đoạn tương ứng. Với giải đấu theo đội, Đội trưởng có thêm các chức năng tạo đội, quản lý thành viên đội và check-in cho đội.



Hình 2.8: Biểu đồ use case phân rã Vận hành trận đấu trong giải đấu

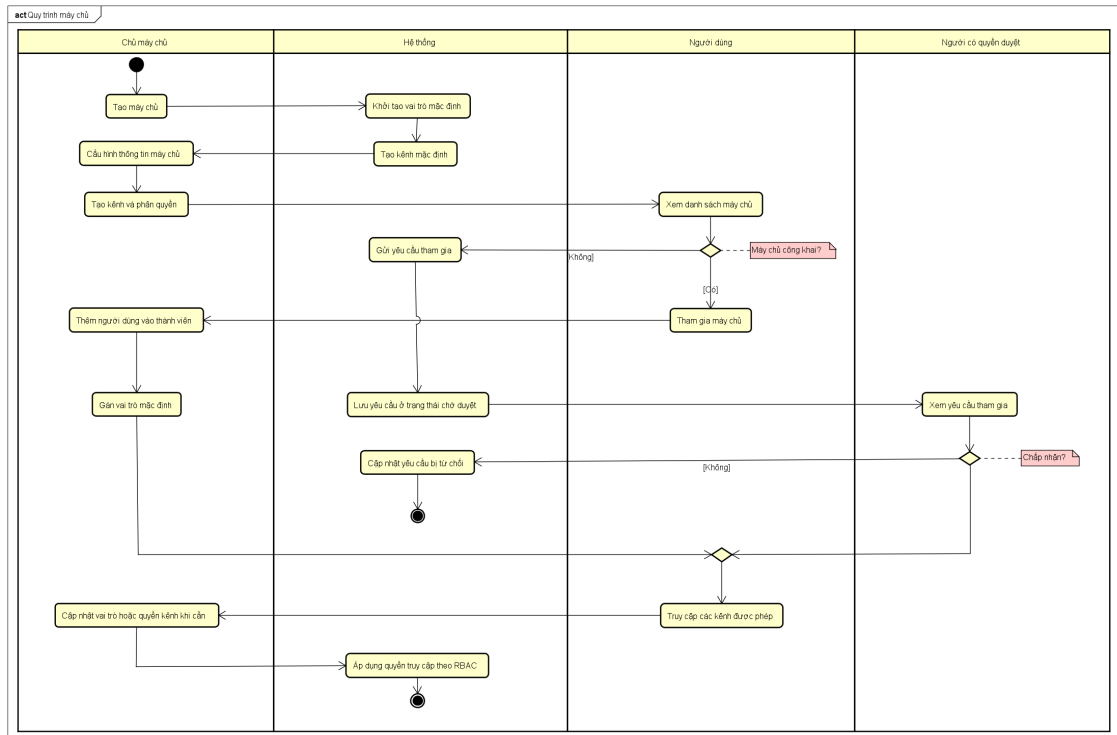
Hình 2.8 mô tả các chức năng sau khi giải đấu đã bắt đầu. Người tổ chức hoặc Quản trị giải đấu có thể bắt đầu trận đấu, cập nhật trạng thái trận, tạo workspace trận và ghi đề kết quả khi cần. Người tham gia và Đội trưởng có thể xem bracket, xem lịch trận, báo cáo kết quả và khiếu nại kết quả.

Các vai trò hỗ trợ gồm Trọng tài, Bình luận viên và Khán giả. Trọng tài theo dõi trận đấu và xử lý khiếu nại; Bình luận viên có thể nhận quyền theo dõi trận để phục vụ bình luận; Khán giả xem bracket, bảng xếp hạng và theo dõi kênh livestream của trận đấu.

2.2.8 Quy trình nghiệp vụ

Bên cạnh các biểu đồ use case, hệ thống còn có một số quy trình nghiệp vụ quan trọng kết hợp nhiều chức năng khác nhau. Trong phạm vi đề án, ba quy trình tiêu biểu được lựa chọn để mô tả bằng biểu đồ hoạt động gồm quy trình quản lý và tham gia máy chủ, quy trình tổ chức và vận hành giải đấu, và quy trình đăng tải game.

a, Quy trình nghiệp vụ Quản lý và Tham gia máy chủ

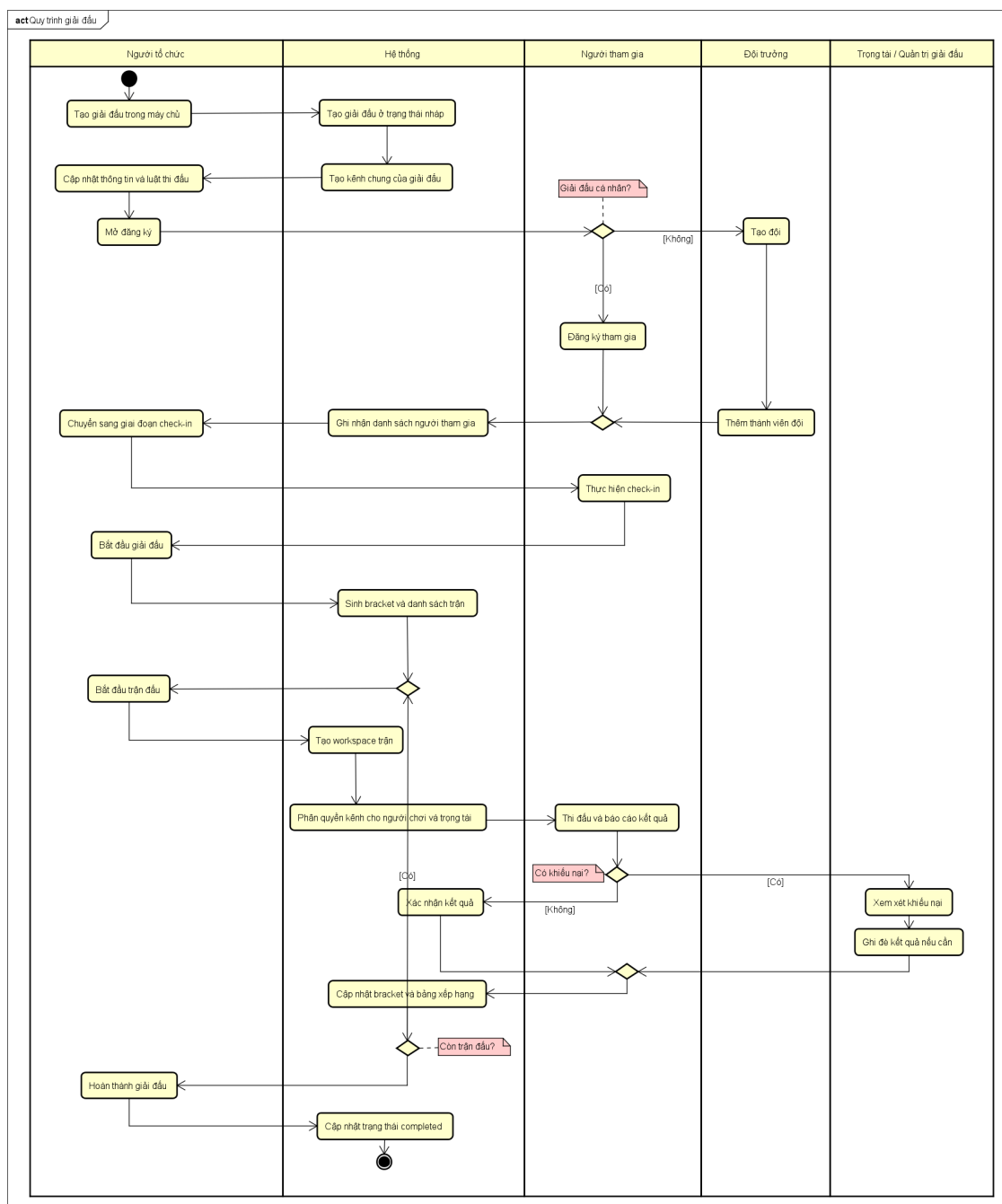


Hình 2.9: Quy trình nghiệp vụ Quản lý và Tham gia máy chủ

Hình 2.9 mô tả quy trình từ khi Chủ máy chủ tạo máy chủ đến khi Người dùng có thể tham gia và truy cập các kênh được phép. Sau khi máy chủ được tạo, hệ thống khởi tạo các vai trò và kênh mặc định. Chủ máy chủ tiếp tục cấu hình thông tin, tạo kênh và thiết lập phân quyền cho máy chủ.

Người dùng có thể xem danh sách máy chủ và tham gia trực tiếp nếu máy chủ công khai. Với máy chủ yêu cầu xét duyệt, Người dùng gửi yêu cầu tham gia, hệ thống lưu yêu cầu ở trạng thái chờ và người có quyền duyệt sẽ chấp nhận hoặc từ chối. Khi được chấp nhận, hệ thống thêm Người dùng vào danh sách thành viên, gán vai trò mặc định và áp dụng quyền truy cập theo cơ chế phân quyền của máy chủ.

b, Quy trình nghiệp vụ Tổ chức và Vận hành giải đấu



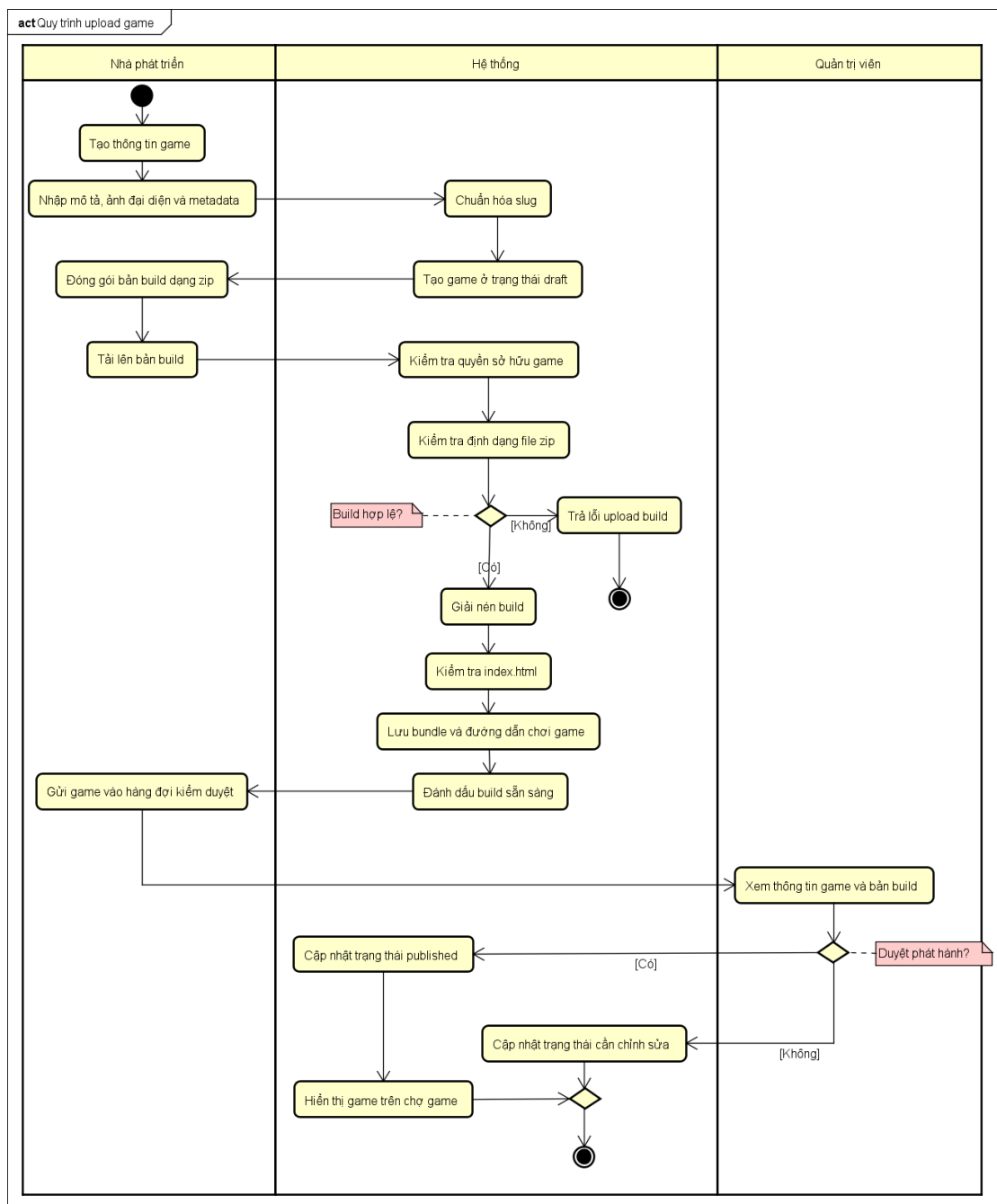
Hình 2.10: Quy trình nghiệp vụ Tổ chức và Vận hành giải đấu

Hình 2.10 thể hiện quy trình tổ chức một giải đấu trong máy chủ. Người tổ chức tạo giải đấu, cập nhật thông tin và mở đăng ký. Người tham gia đăng ký trực tiếp nếu là giải đấu cá nhân; với giải đấu theo đội, Đội trưởng tạo đội và thêm các thành viên cần thiết. Sau giai đoạn đăng ký, Người tổ chức chuyển giải đấu sang check-in để xác nhận các người chơi hoặc đội tham gia.

Khi giải đấu bắt đầu, hệ thống sinh bracket và danh sách trận đấu. Với mỗi trận, Người tổ chức bắt đầu trận đấu, hệ thống tạo workspace trận và phân quyền kênh

cho người chơi, trọng tài hoặc các vai trò liên quan. Sau khi trận kết thúc, người tham gia báo cáo kết quả; nếu có khiếu nại, Trọng tài hoặc Quản trị giải đấu xem xét và ghi đề kết quả khi cần. Hệ thống cập nhật bracket, bảng xếp hạng và lặp lại quy trình cho đến khi toàn bộ trận đấu hoàn thành.

c, Quy trình nghiệp vụ Đăng tải game



Hình 2.11: Quy trình nghiệp vụ Đăng tải game

Hình 2.11 mô tả quy trình Nhà phát triển đưa game lên hệ thống. Trước hết, Nhà phát triển tạo thông tin game, nhập mô tả, hình ảnh và các metadata cần thiết. Hệ thống chuẩn hóa slug và tạo bản ghi game ở trạng thái nháp để Nhà phát triển tiếp

tục tải lên bản build.

Sau khi Nhà phát triển tải lên file build dạng zip, hệ thống kiểm tra quyền sở hữu game, định dạng file, cấu trúc bundle và file `index.html`. Nếu build không hợp lệ, hệ thống trả lỗi để Nhà phát triển chỉnh sửa và tải lại. Nếu build hợp lệ, hệ thống lưu bundle, tạo đường dẫn chơi game, đánh dấu build sẵn sàng và cho phép gửi game vào hàng đợi kiểm duyệt. Quản trị viên kiểm tra nội dung game; nếu được duyệt, hệ thống cập nhật trạng thái phát hành và hiển thị game trên chợ game, ngược lại game được đưa về trạng thái cần chỉnh sửa.

2.3 Đặc tả chức năng

Từ các nhóm use case đã trình bày ở phần 2.2, đề án lựa chọn các use case có luồng xử lý quan trọng để đặc tả chi tiết, bao gồm Tham gia máy chủ, Quản lý kênh và phân quyền, Đăng tải và kiểm duyệt game, Tổ chức giải đấu, và Vận hành trận đấu. Các use case này đại diện cho ba miền nghiệp vụ chính của hệ thống là cộng đồng máy chủ, game marketplace và giải đấu.

Mỗi bản đặc tả use case bao gồm tên use case, tác nhân tham gia, mô tả, tiền điều kiện, luồng sự kiện chính, luồng sự kiện thay thế và hậu điều kiện. Nội dung đặc tả chi tiết được biên soạn thành các bảng riêng để thuận tiện chỉnh sửa và xuất thành hình ảnh trong quá trình hoàn thiện quyền đồ án.

2.3.1 Đặc tả use case Tham gia máy chủ

Use case Tham gia máy chủ mô tả quá trình Người dùng gia nhập một máy chủ trong hệ thống. Tùy theo cấu hình của máy chủ, Người dùng có thể tham gia trực tiếp nếu máy chủ công khai hoặc phải gửi yêu cầu chờ người có quyền xét duyệt.

UC001 - Tham gia máy chủ		
Thuộc tính		Nội dung
Mã Use case	UC001	
Tên Use case	Tham gia máy chủ	
Tác nhân	Người dùng, Người có quyền duyệt yêu cầu tham gia, Hệ thống	
Mô tả	Cho phép Người dùng tham gia một máy chủ công khai hoặc gửi yêu cầu tham gia đối với máy chủ cần xét duyệt. Khi yêu cầu được chấp nhận, hệ thống thêm Người dùng vào danh sách thành viên và gán vai trò mặc định.	
Tiền điều kiện	Người dùng đã đăng nhập; máy chủ tồn tại; Người dùng chưa là thành viên của máy chủ cần tham gia. Với luồng xét duyệt, người xử lý yêu cầu phải có quyền APPROVE_MEMBERS.	
Hậu điều kiện	Nếu thành công, Người dùng trở thành thành viên của máy chủ và có vai trò mặc định. Nếu yêu cầu bị từ chối hoặc dữ liệu không hợp lệ, trạng thái thành viên của Người dùng không thay đổi.	
Luồng sự kiện chính		
STT	Thực hiện bởi	Hành động
1	Người dùng	Xem danh sách máy chủ hoặc mở chi tiết máy chủ muốn tham gia.
2	Hệ thống	Kiểm tra trạng thái đăng nhập và kiểm tra máy chủ có tồn tại hay không.
3	Hệ thống	Kiểm tra Người dùng đã là thành viên của máy chủ hay chưa.
4	Người dùng	Chọn chức năng tham gia máy chủ.
5	Hệ thống	Nếu máy chủ công khai, thêm Người dùng vào danh sách thành viên.
6	Hệ thống	Gán vai trò mặc định của máy chủ cho Người dùng.
7	Hệ thống	Hiển thị máy chủ trong danh sách máy chủ của Người dùng.
Luồng sự kiện thay thế		
STT	Thực hiện bởi	Hành động
4a	Người dùng	Nếu máy chủ không cho tham gia trực tiếp, Người dùng nhập ghi chú và gửi yêu cầu tham gia.
5a	Hệ thống	Tạo yêu cầu tham gia với trạng thái chờ duyệt.
6a	Người có quyền duyệt	Xem danh sách yêu cầu tham gia của máy chủ.
7a	Người có quyền duyệt	Chấp nhận yêu cầu tham gia.
8a	Hệ thống	Cập nhật yêu cầu sang trạng thái được chấp nhận, thêm thành viên và gán vai trò mặc định.
7b	Người có quyền duyệt	Từ chối yêu cầu tham gia.
8b	Hệ thống	Cập nhật yêu cầu sang trạng thái bị từ chối, Người dùng không được thêm vào máy chủ.
5c	Hệ thống	Nếu Người dùng đã là thành viên, hệ thống thông báo lỗi và kết thúc use case.
6c	Hệ thống	Nếu yêu cầu đã được duyệt trước đó, hệ thống không cho xử lý lại yêu cầu.

Hình 2.12: Đặc tả use case Tham gia máy chủ

Bảng đặc tả ở Hình 2.12 làm rõ hai nhánh chính của nghiệp vụ tham gia máy chủ. Với máy chủ công khai, hệ thống thêm Người dùng vào danh sách thành viên và gán vai trò mặc định. Với máy chủ cần xét duyệt, hệ thống tạo yêu cầu tham gia để người có quyền duyệt chấp nhận hoặc từ chối.

2.3.2 Đặc tả use case Quản lý kênh và phân quyền

Use case Quản lý kênh và phân quyền mô tả cách Chủ máy chủ hoặc thành viên có quyền quản lý cấu hình các kênh trong máy chủ. Đây là use case quan trọng vì ảnh hưởng trực tiếp đến khả năng xem kênh, gửi tin nhắn và truy cập các khu vực riêng tư của cộng đồng.

UC002 - Quản lý kênh và phân quyền		
Thuộc tính		Nội dung
Mã Use case	UC002	
Tên Use case	Quản lý kênh và phân quyền	
Tác nhân	Chủ máy chủ, Thành viên có quyền quản lý kênh, Hệ thống	
Mô tả	Cho phép tác nhân có quyền tạo kênh, cập nhật vị trí, đặt kênh riêng tư, thêm hoặc xóa thành viên khỏi kênh riêng tư và thiết lập quyền ghi đè theo vai trò hoặc người dùng.	
Tiền điều kiện	Tác nhân đã đăng nhập, là thành viên của máy chủ và có quyền <code>MANAGE_CHANNELS</code> hoặc <code>ADMINISTRATOR</code> . Máy chủ và kênh cha nếu có phải tồn tại hợp lệ.	
Hậu điều kiện	Kênh và các cấu hình phân quyền được cập nhật trong hệ thống. Thành viên chỉ nhìn thấy hoặc thao tác trên các kênh mà họ có quyền truy cập.	
Luồng sự kiện chính		
STT	Thực hiện bởi	Hành động
1	Tác nhân	Chọn chức năng tạo hoặc cấu hình kênh trong máy chủ.
2	Hệ thống	Kiểm tra quyền quản lý kênh của tác nhân.
3	Tác nhân	Nhập tên kênh, loại kênh, kênh cha và vị trí hiển thị nếu cần.
4	Hệ thống	Kiểm tra loại kênh hợp lệ và kiểm tra kênh cha thuộc cùng máy chủ.
5	Hệ thống	Tạo kênh mới hoặc cập nhật cấu hình kênh.
6	Tác nhân	Thiết lập kênh riêng tư hoặc cấu hình quyền ghi đè cho vai trò/người dùng.
7	Hệ thống	Lưu cấu hình thành viên kênh và quyền ghi đè.
8	Hệ thống	Áp dụng quyền truy cập khi thành viên xem danh sách kênh hoặc truy cập kênh.
Luồng sự kiện thay thế		
STT	Thực hiện bởi	Hành động
2a	Hệ thống	Nếu tác nhân không có quyền quản lý kênh, hệ thống từ chối thao tác.
4a	Hệ thống	Nếu loại kênh không hợp lệ, hệ thống hiển thị lỗi và yêu cầu nhập lại.
4b	Hệ thống	Nếu kênh cha không phải loại danh mục hoặc không thuộc cùng máy chủ, hệ thống từ chối tạo kênh.
6a	Tác nhân	Thêm thành viên cụ thể vào kênh riêng tư.
7a	Hệ thống	Kiểm tra người được thêm phải là thành viên của máy chủ trước khi lưu.
6b	Tác nhân	Xóa quyền ghi đè hoặc xóa thành viên khỏi kênh riêng tư.

Hình 2.13: Đặc tả use case Quản lý kênh và phân quyền

Hình 2.13 đặc tả các thao tác tạo kênh, cấu hình kênh riêng tư, thêm thành viên vào kênh và thiết lập quyền ghi đè theo vai trò hoặc người dùng. Hệ thống luôn kiểm tra quyền quản lý kênh trước khi cho phép thay đổi cấu hình.

2.3.3 Đặc tả use case Đăng tải và kiểm duyệt game

Use case Đăng tải và kiểm duyệt game mô tả quy trình Nhà phát triển đưa một game cộng đồng lên hệ thống. Quy trình này bao gồm tạo thông tin game, tải lên bản build, gửi kiểm duyệt và chờ Quản trị viên duyệt phát hành.

UC003 - Đăng tải và kiểm duyệt game		
Thuộc tính		Nội dung
Mã Use case	UC003	
Tên Use case	Đăng tải và kiểm duyệt game	
Tác nhân	Nhà phát triển, Quản trị viên, Hệ thống	
Mô tả	Cho phép Nhà phát triển tạo thông tin game, tải lên file build dạng zip, gửi game vào hàng đợi kiểm duyệt và cho phép Quản trị viên cập nhật trạng thái phát hành.	
Tiền điều kiện	Nhà phát triển đã đăng nhập. Game cộng đồng phải có tiêu đề, slug, ảnh biểu tượng, ảnh capsule, ảnh hero và ít nhất một ảnh chụp màn hình. File build phải là gói hợp lệ và có index.html.	
Hậu điều kiện	Game được lưu ở trạng thái phù hợp với quy trình kiểm duyệt. Nếu được duyệt, game xuất hiện trên chợ game và có thể được người dùng truy cập để chơi.	
Luồng sự kiện chính		
STT	Thực hiện bởi	Hành động
1	Nhà phát triển	Chọn chức năng tạo game mới.
2	Nhà phát triển	Nhập tiêu đề, slug, mô tả, chế độ hiển thị, danh mục, thẻ và các tài nguyên hình ảnh.
3	Hệ thống	Chuẩn hóa slug, kiểm tra các trường bắt buộc và tạo game ở trạng thái draft.
4	Nhà phát triển	Chọn file build dạng zip và tải lên hệ thống.
5	Hệ thống	Kiểm tra Nhà phát triển có phải chủ sở hữu game hay không.
6	Hệ thống	Lưu file build, kiểm tra checksum, giải nén bundle và kiểm tra file index.html.
7	Hệ thống	Tạo bản build với trạng thái ready và sinh đường dẫn chơi game.
8	Nhà phát triển	Gửi game vào hàng đợi kiểm duyệt.
9	Hệ thống	Kiểm tra game có ít nhất một build sẵn sàng và cập nhật trạng thái pending_review.
10	Quản trị viên	Kiểm tra nội dung game và cập nhật trạng thái phát hành.
11	Hệ thống	Nếu được duyệt, cập nhật trạng thái published và hiển thị game trên chợ game.
Luồng sự kiện thay thế		
STT	Thực hiện bởi	Hành động
3a	Hệ thống	Nếu thiếu tài nguyên bắt buộc của game cộng đồng, hệ thống báo lỗi và không tạo game.
5a	Hệ thống	Nếu Người dùng không phải chủ sở hữu game, hệ thống từ chối upload build.
6a	Hệ thống	Nếu file build không hợp lệ hoặc thiếu index.html, hệ thống lưu build thất bại và trả lỗi.
9a	Hệ thống	Nếu game chưa có build sẵn sàng, hệ thống không cho gửi kiểm duyệt.
10a	Quản trị viên	Nếu game chưa đạt yêu cầu, Quản trị viên cập nhật trạng thái rejected hoặc suspended.

Hình 2.14: Đặc tả use case Đăng tải và kiểm duyệt game

Hình 2.14 thể hiện các bước từ tạo metadata game, tải lên file build dạng zip, kiểm tra bundle có index.html, đến khi gửi game vào hàng đợi kiểm duyệt. Game chỉ được hiển thị trên chợ game sau khi Quản trị viên cập nhật trạng thái phát hành.

2.3.4 Đặc tả use case Tổ chức giải đấu

Use case Tổ chức giải đấu mô tả quy trình Người tổ chức tạo và đưa một giải đấu từ trạng thái nháp sang đăng ký, check-in và bắt đầu thi đấu. Đây là use case trung tâm của phân hệ giải đấu.

UC004 - Tổ chức giải đấu		
Thuộc tính		Nội dung
Mã Use case	UC004	
Tên Use case	Tổ chức giải đấu	
Tác nhân	Người tổ chức, Người tham gia, Đội trưởng, Hệ thống	
Mô tả	Cho phép Người tổ chức tạo giải đấu trong máy chủ, cấu hình thông tin, mở đăng ký, xác nhận check-in và bắt đầu giải đấu để hệ thống sinh bracket.	
Tiền điều kiện	Người tổ chức đã đăng nhập, là thành viên máy chủ và có quyền <code>MANAGE_CHANNELS</code> . Máy chủ tồn tại. Với giải đấu đội, trường số lượng thành viên đội phải hợp lệ.	
Hậu điều kiện	Giải đấu được tạo và đi qua các trạng thái hợp lệ. Khi bắt đầu thành công, hệ thống có bracket và danh sách trận đấu để phục vụ giai đoạn vận hành giải đấu.	
Luồng sự kiện chính		
STT	Thực hiện bởi	Hành động
1	Người tổ chức	Chọn tạo giải đấu trong một máy chủ.
2	Người tổ chức	Nhập tên giải đấu, game, thể thức, số lượng người tham gia, loại tham gia, luật và giải thưởng.
3	Hệ thống	Kiểm tra quyền tổ chức giải đấu và kiểm tra dữ liệu đầu vào.
4	Hệ thống	Tạo giải đấu ở trạng thái draft và tạo kênh chung của giải đấu.
5	Người tổ chức	Chuyển giải đấu sang trạng thái registration.
6	Người tham gia	Đăng ký tham gia nếu giải đấu là cá nhân.
7	Hệ thống	Kiểm tra thời hạn đăng ký, số lượng tối đa và trạng thái giải đấu trước khi lưu người tham gia.
8	Người tổ chức	Chuyển giải đấu sang trạng thái check_in.
9	Người tham gia	Thực hiện check-in trong thời gian cho phép.
10	Người tổ chức	Chuyển giải đấu sang trạng thái in_progress.
11	Hệ thống	Sinh bracket, tạo danh sách trận đấu và ghi nhận thời điểm bắt đầu.
Luồng sự kiện thay thế		
STT	Thực hiện bởi	Hành động
3a	Hệ thống	Nếu Người tổ chức không có quyền phù hợp, hệ thống từ chối tạo hoặc cập nhật giải đấu.
3b	Hệ thống	Nếu thể thức hoặc loại người tham gia không hợp lệ, hệ thống báo lỗi.
6a	Đội trưởng	Nếu là giải đấu đội, Đội trưởng tạo đội và thêm thành viên.
7a	Hệ thống	Nếu đội vượt quá số lượng thành viên cho phép, hệ thống từ chối thêm thành viên.
7b	Hệ thống	Nếu giải đấu đã hết hạn đăng ký, đủ số lượng hoặc người tham gia đã đăng ký trước đó, hệ thống từ chối đăng ký.
10a	Hệ thống	Nếu chuyển trạng thái không đúng thứ tự, hệ thống báo lỗi chuyển trạng thái không hợp lệ.
10b	Người tổ chức	Người tổ chức có thể hủy giải đấu trước khi giải đấu hoàn thành.

Hình 2.15: Đặc tả use case Tổ chức giải đấu

Hình 2.15 đặc tả quá trình tạo giải đấu trong máy chủ, mở đăng ký, xử lý đăng ký cá nhân hoặc đăng ký theo đội, chuyển sang giai đoạn check-in và bắt đầu giải đấu. Khi giải đấu bắt đầu, hệ thống sinh bracket và danh sách trận đấu.

2.3.5 Đặc tả use case Vận hành trận đấu và xử lý kết quả

Use case Vận hành trận đấu và xử lý kết quả mô tả quá trình bắt đầu một trận đấu, tạo workspace phục vụ thi đấu, ghi nhận kết quả và xử lý các trường hợp khiếu nại hoặc ghi đè kết quả.

UC005 - Vận hành trận đấu và xử lý kết quả		
Thuộc tính		Nội dung
Mã Use case	UC005	
Tên Use case	Vận hành trận đấu và xử lý kết quả	
Tác nhân	Người tổ chức, Quản trị giải đấu, Người tham gia, Đội trưởng, Trọng tài, Hệ thống	
Mô tả	Cho phép người có quyền bắt đầu trận đấu, tạo workspace gồm các kênh phục vụ thi đấu, tiếp nhận báo cáo kết quả, xử lý khiếu nại và cập nhật bracket.	
Tiền điều kiện	Giải đấu đang ở trạng thái in_progress; trận đấu tồn tại và có đủ hai người tham gia hoặc hai đội. Người tổ chức hoặc Quản trị giải đấu có quyền cập nhật giải đấu.	
Hậu điều kiện	Trận đấu có trạng thái và kết quả được cập nhật nhất quán. Bracket, bảng xếp hạng và trận kế tiếp được cập nhật theo kết quả đã xác nhận hoặc kết quả được ghi đè.	
Luồng sự kiện chính		
STT	Thực hiện bởi	Hành động
1	Người tổ chức	Chọn bắt đầu một trận đấu trong bracket.
2	Hệ thống	Kiểm tra quyền cập nhật giải đấu và kiểm tra trạng thái trận đấu.
3	Hệ thống	Chuyển trận đấu từ pending hoặc ready sang in_progress.
4	Hệ thống	Tạo workspace trận gồm danh mục, kênh đội A, kênh đội B, kênh trọng tài và kênh livestream.
5	Hệ thống	Gán người chơi vào kênh đội tương ứng và gán vai trò hỗ trợ vào kênh phù hợp.
6	Người tham gia	Thi đấu và gửi báo cáo kết quả gồm người thắng, điểm số và ảnh minh chứng nếu có.
7	Hệ thống	Lưu báo cáo kết quả ở trạng thái chờ xác nhận.
8	Người tổ chức hoặc Quản trị giải đấu	Xác nhận hoặc ghi đè kết quả trận đấu.
9	Hệ thống	Áp dụng kết quả, cập nhật người thắng, điểm số, bracket và trạng thái trận đấu.
10	Hệ thống	Nếu không còn trận đấu tiếp theo, cập nhật thứ hạng hoặc hỗ trợ hoàn thành giải đấu.
Luồng sự kiện thay thế		
STT	Thực hiện bởi	Hành động
2a	Hệ thống	Nếu trận đấu thiếu một trong hai người tham gia, hệ thống không cho bắt đầu trận.
3a	Hệ thống	Nếu trận đấu đã bắt đầu, hệ thống trả về trạng thái hiện tại và không tạo trùng luồng bắt đầu.
4a	Hệ thống	Nếu workspace đã tồn tại, hệ thống tái sử dụng workspace và cập nhật lại thành viên kênh.
6a	Người tham gia	Nếu không đồng ý với kết quả đã báo cáo, Người tham gia gửi khiếu nại kết quả.
7a	Hệ thống	Cập nhật báo cáo gần nhất sang trạng thái disputed.
8a	Trọng tài hoặc Quản trị giải đấu	Xem xét khiếu nại và ghi đè kết quả nếu cần.
8b	Hệ thống	Nếu người báo cáo là Người tổ chức, hệ thống tự xác nhận và áp dụng kết quả ngay.

Hình 2.16: Đặc tả use case Vận hành trận đấu và xử lý kết quả

Hình 2.16 trình bày luồng vận hành trận đấu từ lúc bắt đầu trận, tạo workspace, phân quyền kênh cho người chơi và trọng tài, đến báo cáo kết quả. Nếu có khiếu nại hoặc cần can thiệp, Trọng tài hoặc Quản trị giải đấu xử lý và hệ thống cập nhật lại bracket theo kết quả cuối cùng.

2.4 Yêu cầu phi chức năng

Bên cạnh các yêu cầu chức năng, hệ thống cần đáp ứng một số yêu cầu phi chức năng cơ bản. Về hiệu năng, các thao tác thường xuyên như xem máy chủ, tải tin nhắn, gửi tin nhắn và xem danh sách game cần có thời gian phản hồi ngắn. Về độ tin cậy, dữ liệu người dùng, máy chủ, game và giải đấu phải được lưu trữ bền vững, hạn chế mất mát khi có lỗi xử lý.

Về bảo mật, hệ thống cần xác thực người dùng, kiểm tra quyền truy cập theo vai trò, giới hạn quyền thao tác với máy chủ, kênh, game và giải đấu. Về khả năng mở

rộng và bảo trì, mã nguồn cần được tổ chức theo các tầng rõ ràng, tách biệt giao diện, xử lý nghiệp vụ, truy cập dữ liệu và hạ tầng thời gian thực để thuận tiện phát triển thêm chức năng sau này.

CHƯƠNG 3. NỀN TẢNG LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Trong các chương trước, đề án đã trình bày bài toán xây dựng nền tảng cộng đồng game tích hợp giao tiếp thời gian thực và quản lý giải đấu, đồng thời phân tích các nhóm chức năng chính như quản lý tài khoản, quản lý máy chủ, nhắn tin, chơi và đăng tải game, voice/livestream, cũng như tổ chức giải đấu. Chương này trình bày ngắn gọn các nền tảng lý thuyết và nhóm công nghệ được sử dụng để hiện thực các yêu cầu đó. Nội dung chương được trình bày theo các nhóm chính gồm frontend, backend, cơ sở dữ liệu, giao tiếp thời gian thực và hạ tầng triển khai, tránh liệt kê quá nhiều thư viện phụ không cần thiết.

3.1 Tổng quan công nghệ sử dụng

Hệ thống được xây dựng theo mô hình ứng dụng client-server. Phía client gồm ứng dụng web và ứng dụng desktop, phục vụ các thao tác của người dùng như đăng nhập, tham gia máy chủ, nhắn tin, chơi game, quản lý giải đấu và tham gia voice/livestream. Phía server cung cấp API, xử lý nghiệp vụ, xác thực, phân quyền, lưu trữ dữ liệu và phát các cập nhật thời gian thực. Cách tổ chức này phù hợp với các yêu cầu ở Chương 2 vì hệ thống cần tách biệt rõ giao diện người dùng, xử lý nghiệp vụ và dữ liệu.

Các nhóm công nghệ chính được sử dụng trong đề án gồm: React và TypeScript cho frontend; Go và Gin cho backend; MySQL, GORM và Redis cho lưu trữ dữ liệu; WebSocket và LiveKit cho giao tiếp thời gian thực; Cloudinary cho lưu trữ tài nguyên media; Docker Compose cho môi trường chạy cục bộ. Những công nghệ này không được lựa chọn riêng lẻ, mà được kết hợp để giải quyết các yêu cầu cụ thể của hệ thống.

3.2 Frontend

Frontend của hệ thống được xây dựng bằng React kết hợp TypeScript. React hỗ trợ phát triển giao diện theo mô hình component, cho phép chia các màn hình lớn như danh sách máy chủ, kênh chat, trang game, trang giải đấu và voice channel thành nhiều thành phần nhỏ, có thể tái sử dụng và bảo trì. TypeScript bổ sung kiểu tĩnh cho mã nguồn frontend, giúp giảm lỗi khi xử lý các dữ liệu phức tạp như thông tin máy chủ, kênh, vai trò, tin nhắn, game, giải đấu và trận đấu [5], [6].

Để phục vụ phát triển và đóng gói ứng dụng, dự án sử dụng Vite cho phiên bản web và Electron cho phiên bản desktop. Vite giúp quá trình phát triển frontend nhanh hơn nhờ dev server nhẹ và khả năng build ứng dụng React hiệu quả. Electron cho phép tái sử dụng phần lớn giao diện web để đóng gói thành ứng dụng desktop,

phù hợp với định hướng của nền tảng là cung cấp trải nghiệm cộng đồng game gần với các ứng dụng giao tiếp hiện đại [7], [8].

Một lựa chọn thay thế phổ biến cho React là Vue.js. Vue.js có cú pháp thân thiện và dễ tiếp cận, tuy nhiên React có hệ sinh thái lớn hơn và phù hợp hơn với các thư viện giao diện, quản lý trạng thái và LiveKit mà dự án đang sử dụng. Đối với ứng dụng desktop, Tauri là một lựa chọn nhẹ hơn Electron, nhưng Electron thuận lợi hơn trong việc tái sử dụng giao diện React và tích hợp nhanh trong phạm vi đồ án. Vì vậy, React, TypeScript, Vite và Electron được chọn để cân bằng giữa tốc độ phát triển, khả năng bảo trì và khả năng mở rộng.

3.3 Backend

Backend của hệ thống được phát triển bằng Go và Gin. Go là ngôn ngữ biên dịch, có kiểu tĩnh, hiệu năng tốt và hỗ trợ lập trình đồng thời thuận tiện. Trong đồ án, Go được sử dụng để xử lý các nghiệp vụ chính như xác thực người dùng, quản lý máy chủ, quản lý vai trò, quản lý kênh, tin nhắn, game, giải đấu và các kết nối thời gian thực. Gin là web framework dùng để xây dựng các REST API, tổ chức route, middleware, xử lý JSON request/response và tích hợp với các cơ chế xác thực, phân quyền [9], [10].

Về mặt thiết kế API, hệ thống sử dụng RESTful API cho phần lớn chức năng. Các tài nguyên như người dùng, máy chủ, kênh, tin nhắn, game và giải đấu được thao tác thông qua các phương thức HTTP quen thuộc như GET, POST, PATCH và DELETE. Cách tiếp cận này giúp frontend web, frontend desktop và các công cụ kiểm thử có thể sử dụng cùng một giao diện API.

Đối với xác thực và phân quyền, hệ thống sử dụng JWT và RBAC. JWT giúp truyền thông tin xác thực giữa client và server thông qua token, phù hợp với mô hình frontend-backend tách rời. RBAC cho phép gán quyền theo vai trò trong máy chủ, ví dụ quyền xem kênh, gửi tin nhắn, quản lý kênh, quản lý vai trò hoặc duyệt thành viên. Cơ chế này trực tiếp phục vụ nhóm chức năng quản lý máy chủ, quản lý kênh và phân quyền đã nêu ở Chương 2 [11].

Một lựa chọn thay thế phổ biến cho backend là Node.js với Express hoặc NestJS. Node.js có hệ sinh thái lớn và thuận tiện khi dùng chung ngôn ngữ với frontend, tuy nhiên Go phù hợp hơn với định hướng dịch vụ mạng, xử lý đồng thời và triển khai gọn của đồ án. Vì vậy, Go và Gin được chọn làm nền tảng backend chính.

3.4 Cơ sở dữ liệu và lưu trữ

MySQL được sử dụng làm hệ quản trị cơ sở dữ liệu chính của hệ thống. Các dữ liệu như tài khoản, quan hệ bạn bè, máy chủ, vai trò, kênh, tin nhắn, game, bản

build game, giải đấu, đội thi đấu, trận đấu và kết quả đều có quan hệ rõ ràng và cần lưu trữ bền vững. MySQL phù hợp với yêu cầu này nhờ mô hình cơ sở dữ liệu quan hệ, khả năng truy vấn bằng SQL, hỗ trợ ràng buộc dữ liệu và transaction [12].

Backend sử dụng GORM để thao tác với cơ sở dữ liệu. GORM giúp ánh xạ model trong Go với bảng dữ liệu, hỗ trợ truy vấn, transaction và giảm lượng mã SQL thủ công. Với phạm vi đồ án, GORM giúp tăng tốc độ phát triển mà vẫn đủ linh hoạt cho các nghiệp vụ như quản lý server, game và tournament [13].

Ngoài MySQL, hệ thống sử dụng Redis cho một số dữ liệu tạm thời và tác vụ cần truy cập nhanh, chẳng hạn rate limit hoặc các thành phần phụ trợ liên quan tới realtime. Đối với tài nguyên media như avatar, ảnh máy chủ, ảnh game và một số file upload, hệ thống sử dụng Cloudinary để lưu trữ và trả về URL truy cập công khai [14], [15].

Một lựa chọn thay thế cho MySQL là PostgreSQL. PostgreSQL mạnh và linh hoạt, nhưng dữ liệu của hệ thống chủ yếu là dữ liệu quan hệ phổ biến, không yêu cầu các đặc thù nâng cao nên MySQL đáp ứng tốt nhu cầu lưu trữ và truy vấn. Đối với lưu trữ media, hệ thống có thể dùng Amazon S3 thay cho Cloudinary. Tuy nhiên, Cloudinary dễ tích hợp hơn trong phạm vi đồ án và giảm công sức tự xây dựng hạ tầng lưu trữ file.

3.5 Giao tiếp thời gian thực và truyền thông media

Hệ thống có nhiều chức năng cần cập nhật gần thời gian thực như tin nhắn mới, thông báo, trạng thái phòng game, trạng thái voice channel và một số sự kiện trong giải đấu. Để đáp ứng yêu cầu này, hệ thống sử dụng WebSocket. WebSocket duy trì kết nối hai chiều giữa client và server, cho phép server chủ động gửi dữ liệu đến client khi có sự kiện mới, thay vì client phải liên tục gửi request kiểm tra như polling [16].

Đối với voice channel, livestream và chia sẻ màn hình, hệ thống sử dụng LiveKit. LiveKit là nền tảng realtime audio/video dựa trên WebRTC, hỗ trợ tạo phòng, cấp token truy cập, quản lý người tham gia và hỗ trợ egress để ghi hình hoặc phát luồng ra ngoài. Trong đồ án, LiveKit phục vụ các yêu cầu như tham gia kênh thoại, theo dõi trận đấu, hỗ trợ trọng tài hoặc bình luận viên trong giải đấu [17], [18].

Một lựa chọn thay thế cho WebSocket là polling. Polling dễ triển khai nhưng không hiệu quả với hệ thống chat, thông báo và phòng game vì client phải liên tục gửi request kiểm tra dữ liệu mới. Đối với truyền thông media, Jitsi là một lựa chọn phổ biến thay cho LiveKit, nhưng LiveKit thuận lợi hơn khi tích hợp với backend Go, cấp token theo quyền và tùy biến theo nghiệp vụ voice/livestream của hệ thống.

3.6 Hạ tầng phát triển và triển khai

Docker Compose được sử dụng để cấu hình môi trường phát triển cục bộ cho các dịch vụ phụ trợ như MySQL, Redis, LiveKit và LiveKit Egress. Việc mô tả các dịch vụ bằng file cấu hình giúp môi trường chạy có tính lặp lại, giảm lỗi do cài đặt thủ công khác nhau giữa các máy phát triển [19].

Một lựa chọn thay thế là cài đặt thủ công từng dịch vụ trên máy phát triển. Cách này đơn giản lúc đầu nhưng khó tái lập khi chuyển sang máy khác. Docker Compose được chọn vì đủ đơn giản cho phát triển và demo, đồng thời vẫn mô phỏng được hệ thống gồm nhiều dịch vụ phụ trợ.

3.7 Kết luận chương

Chương này đã trình bày các nhóm công nghệ chính được sử dụng trong đồ án theo hướng tinh gọn: frontend, backend, cơ sở dữ liệu, giao tiếp thời gian thực và hạ tầng triển khai. Các công nghệ được lựa chọn đều gắn với yêu cầu đã phân tích ở Chương 2, từ quản lý máy chủ và phân quyền, nhắn tin realtime, voice/livestream, đăng tải game cho đến tổ chức giải đấu. Trong chương tiếp theo, đồ án sẽ trình bày thiết kế và cài đặt hệ thống dựa trên các nền tảng công nghệ đã lựa chọn.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Qua ba chương trước, đồ án đã trình bày bài toán thực tế, xác định các chức năng chính và phân tích các công nghệ được sử dụng. Chương này trình bày quá trình thiết kế và triển khai hệ thống, bao gồm thiết kế kiến trúc, thiết kế chi tiết, cài đặt, kiểm thử và đánh giá kết quả đạt được.

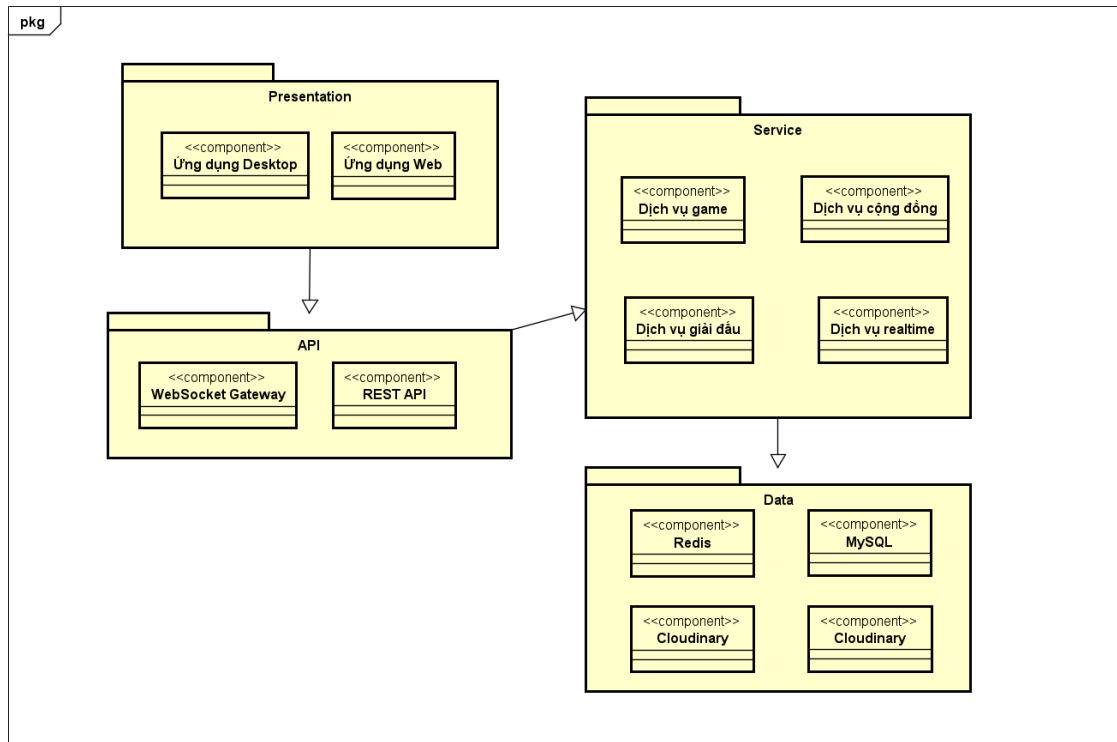
4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống được xây dựng dựa trên kiến trúc đa tầng (N-Tier Architecture) kết hợp mô hình client-server. Kiến trúc đa tầng chia ứng dụng thành nhiều tầng có trách nhiệm riêng biệt, giúp giảm phụ thuộc giữa các thành phần, thuận lợi cho bảo trì và mở rộng. Về cơ bản, kiến trúc này gồm ba tầng chính: tầng trình diễn, tầng logic nghiệp vụ và tầng dữ liệu. Tầng trình diễn chịu trách nhiệm hiển thị giao diện và tiếp nhận thao tác người dùng; tầng logic nghiệp vụ xử lý các quy tắc nghiệp vụ; tầng dữ liệu đảm nhiệm việc lưu trữ, truy vấn và cập nhật dữ liệu.

Trong đồ án, tầng trình diễn được triển khai thành ứng dụng web và ứng dụng desktop. Hai ứng dụng này cung cấp giao diện cho các chức năng như quản lý tài khoản, tham gia máy chủ, nhắn tin, chơi game, đăng tải game, tham gia giải đấu và voice/livestream. Tầng trình diễn giao tiếp với backend thông qua RESTful API, đồng thời sử dụng kết nối thời gian thực cho các chức năng như tin nhắn, thông báo và trạng thái phòng game.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG



Hình 4.1: Kiến trúc đa tầng được áp dụng vào hệ thống

Tầng logic nghiệp vụ được triển khai ở backend, là nơi xử lý các nghiệp vụ đã phân tích ở Chương 2. Tầng này tiếp nhận yêu cầu từ client, kiểm tra dữ liệu đầu vào, xác thực người dùng, kiểm tra phân quyền và thực hiện các xử lý như quản lý máy chủ, quản lý kênh, gửi tin nhắn, upload game, vận hành giải đấu và xử lý kết quả trận đấu. Để tránh trộn lẫn việc tiếp nhận request với xử lý nghiệp vụ, tầng logic được chia thành phần điều phối yêu cầu và phần xử lý nghiệp vụ.

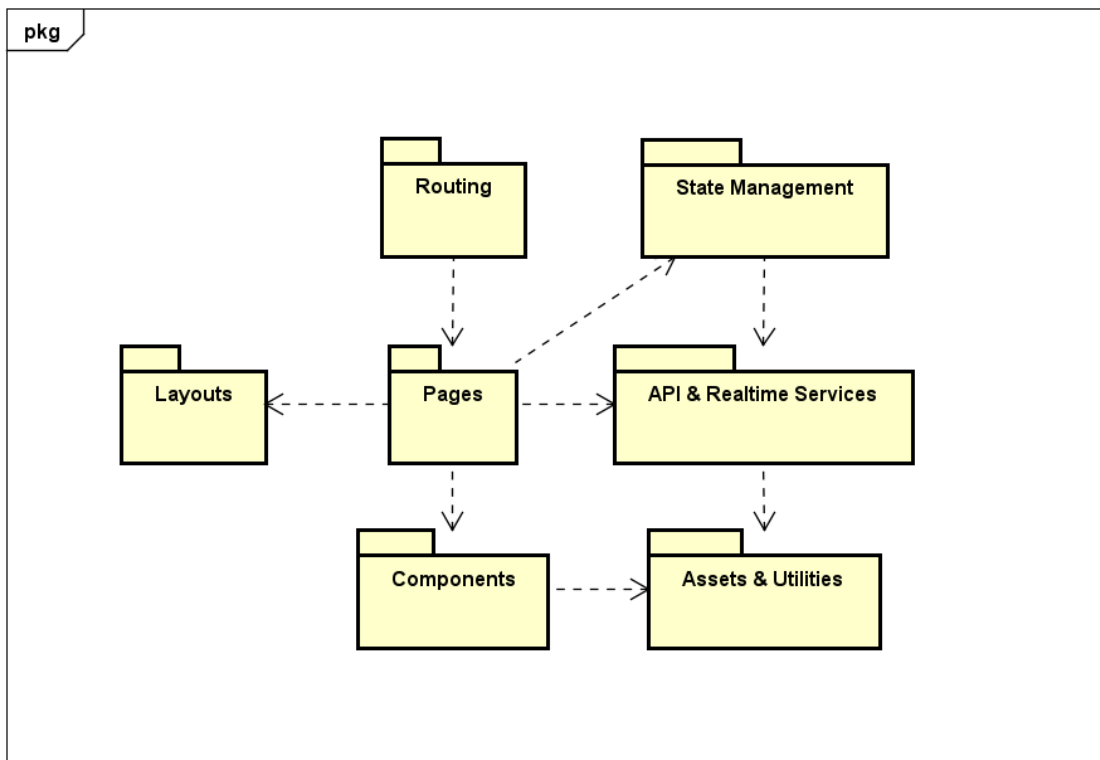
Tầng dữ liệu bao gồm cơ sở dữ liệu quan hệ, bộ nhớ đệm và dịch vụ lưu trữ tài nguyên. Cơ sở dữ liệu lưu các thông tin bền vững như tài khoản, máy chủ, kênh, tin nhắn, game và giải đấu. Bộ nhớ đệm hỗ trợ các dữ liệu tạm thời hoặc thao tác cần truy cập nhanh. Dịch vụ lưu trữ tài nguyên phục vụ các tệp media như ảnh đại diện, ảnh máy chủ, ảnh game và một số tệp tải lên. Ngoài ba tầng chính, hệ thống còn tích hợp các thành phần hạ tầng cho WebSocket, voice/livestream và môi trường triển khai cục bộ.

So với kiến trúc MVC truyền thống, kiến trúc được chọn phù hợp hơn vì giao diện của hệ thống không được render trực tiếp từ backend mà là các client độc lập. So với microservice, kiến trúc đa tầng có độ phức tạp thấp hơn, phù hợp hơn với phạm vi đồ án nhưng vẫn cho phép mở rộng bằng cách tách riêng các thành phần realtime, media hoặc game service khi cần thiết. Vì vậy, kiến trúc đa tầng kết hợp client-server là lựa chọn phù hợp cho hệ thống.

4.1.2 Thiết kế tổng quan

Trong mục này, đề án trình bày biểu đồ phụ thuộc gói cho hai phần chính của hệ thống là frontend và backend. Các gói được phân theo tầng rõ ràng, trong đó tầng trên được phép phụ thuộc vào tầng dưới thông qua các giao diện hoặc dịch vụ được cung cấp, nhưng tầng dưới không phụ thuộc ngược lại tầng trên. Nguyên tắc này giúp hạn chế phụ thuộc chéo và hỗ trợ mở rộng hệ thống trong tương lai.

Trước tiên là biểu đồ phụ thuộc gói cho ứng dụng frontend, tương ứng với tầng trình diễn trong kiến trúc tổng thể.



Hình 4.2: Biểu đồ phụ thuộc gói ứng dụng Frontend

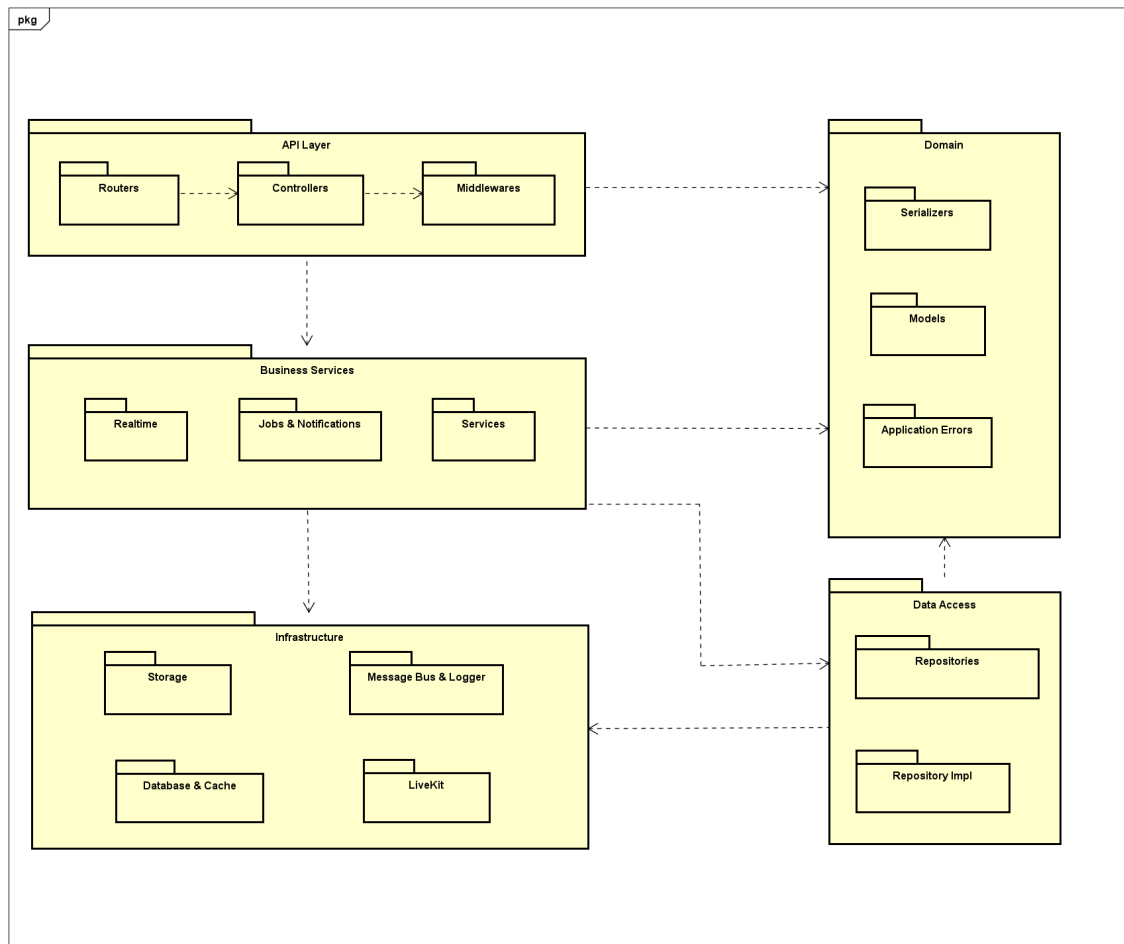
Hình 4.2 thể hiện các gói chính của frontend ở mức tổng quan. Gói *Routing* chịu trách nhiệm ánh xạ đường dẫn URL tới các trang tương ứng. Gói *Layouts* định nghĩa bố cục chung của giao diện như khung ứng dụng, thanh điều hướng và vùng nội dung. Gói *Pages* chứa các màn hình nghiệp vụ chính, là nơi kết hợp dữ liệu và các thành phần giao diện để tạo thành trải nghiệm hoàn chỉnh cho người dùng.

Các gói còn lại đóng vai trò hỗ trợ và tái sử dụng. Gói *Components* chứa các thành phần giao diện dùng chung, gói *State Management* quản lý trạng thái phía client, gói *API & Realtime Services* đảm nhiệm việc giao tiếp với backend qua REST API, WebSocket và các dịch vụ thời gian thực, còn gói *Assets & Utilities* chứa tài nguyên tĩnh và các hàm tiện ích. Quan hệ phụ thuộc đi từ *Routing* tới

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Pages, từ *Pages* tới các gói hỗ trợ, trong khi các gói dùng chung không phụ thuộc ngược lại vào màn hình cụ thể. Cách tổ chức này giúp frontend dễ mở rộng khi bổ sung thêm màn hình hoặc chức năng mới.

Tiếp theo là biểu đồ phụ thuộc gói cho backend, tương ứng với tầng logic nghiệp vụ, tầng dữ liệu và các thành phần hạ tầng.



Hình 4.3: Biểu đồ phụ thuộc gói ứng dụng Backend

Hình 4.3 thể hiện backend ở mức package tổng quát, gồm năm gói chính. Gói *API Layer* là điểm vào của hệ thống, bao gồm *Routers*, *Controllers* và *Middlewares*. Trong đó, *Routers* định tuyến các endpoint, *Controllers* tiếp nhận và điều phối request, còn *Middlewares* xử lý các bước trung gian như xác thực, phân quyền, giới hạn truy cập và xử lý lỗi. Gói *Business Services* chứa các xử lý nghiệp vụ chính, được chia thành *Service Interfaces*, *Service Impl*, *Realtime Hubs* và *Notifications & Jobs*.

Gói *Data Access* đảm nhiệm truy cập dữ liệu, gồm *Repository Interfaces* và *Repository Impl*. Gói *Domain* chứa các thành phần mô tả miền nghiệp vụ như *Models*, *Serializers* và *Application Errors*. Đây là nhóm gói có mức phụ thuộc

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

thấp, được sử dụng bởi các tầng khác để thống nhất cấu trúc dữ liệu và lỗi nghiệp vụ. Cuối cùng, gói *Infrastructure* cung cấp các dịch vụ hạ tầng như *Initialize & Config*, *Database & Cache*, *Storage*, *LiveKit* và *Message Bus & Logger*. Nhóm này giúp backend khởi tạo cấu hình, giao tiếp với cơ sở dữ liệu, Redis, lưu trữ tệp, hệ thống realtime/media và cơ chế ghi log.

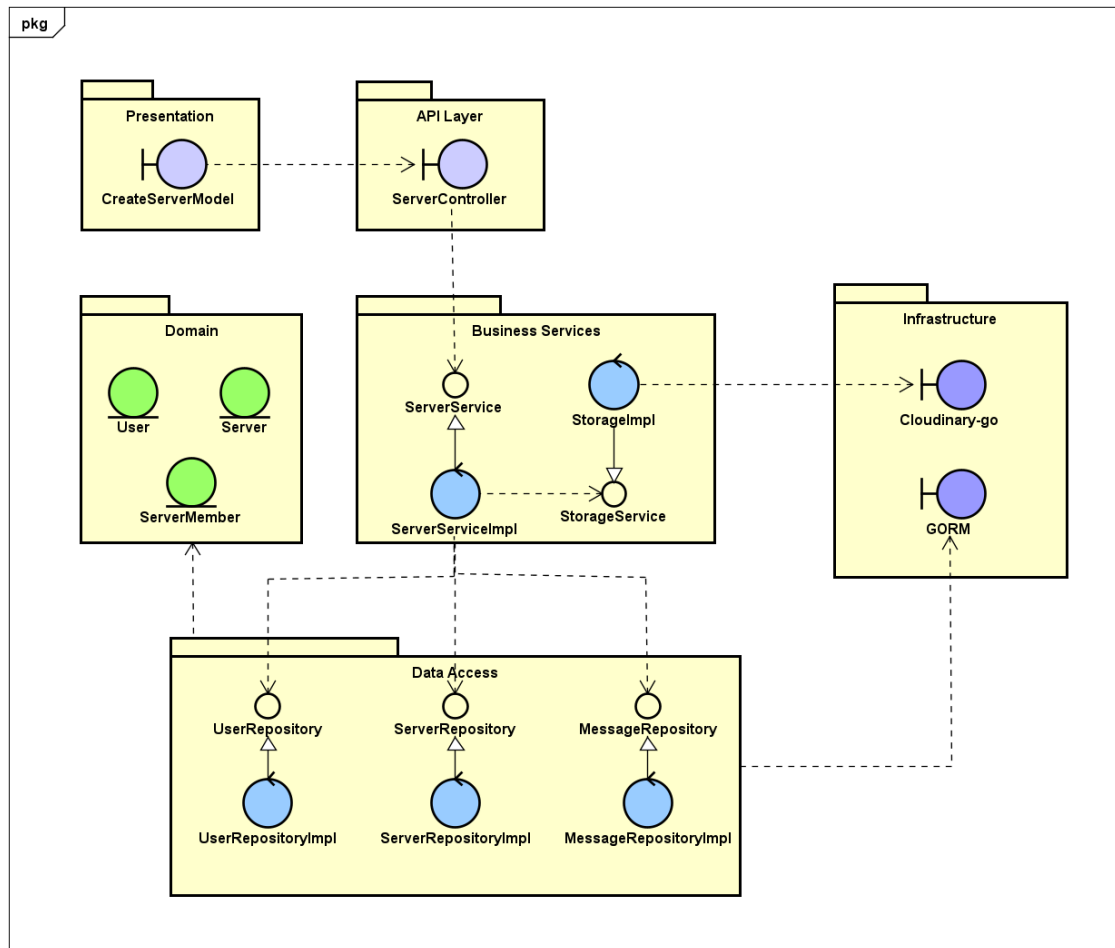
Quan hệ phụ thuộc của backend đi từ *API Layer* xuống *Business Services*, từ *Business Services* xuống *Data Access*, *Domain* và *Infrastructure*. *Data Access* tiếp tục phụ thuộc vào *Domain* để sử dụng mô hình dữ liệu và phụ thuộc vào *Infrastructure* để thao tác với cơ sở dữ liệu hoặc cache. Cách phân tách này giúp phần nghiệp vụ không bị trộn lẫn với chi tiết hạ tầng, đồng thời hỗ trợ mở rộng các chức năng như realtime, voice/livestream hoặc lưu trữ media mà không làm thay đổi cấu trúc tổng thể của backend.

4.1.3 Thiết kế chi tiết gói

Trong mục này, đề án trình bày thiết kế chi tiết gói cho hai nhóm chức năng tiêu biểu của hệ thống là Tạo mới máy chủ và Tổ chức, vận hành giải đấu. Đây là hai chức năng có sự tham gia của nhiều tầng trong kiến trúc, từ giao diện người dùng, API backend, xử lý nghiệp vụ, truy cập dữ liệu cho đến các dịch vụ hạ tầng.

Đầu tiên là thiết kế chi tiết gói cho chức năng Tạo mới máy chủ, được minh họa ở Hình 4.4.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

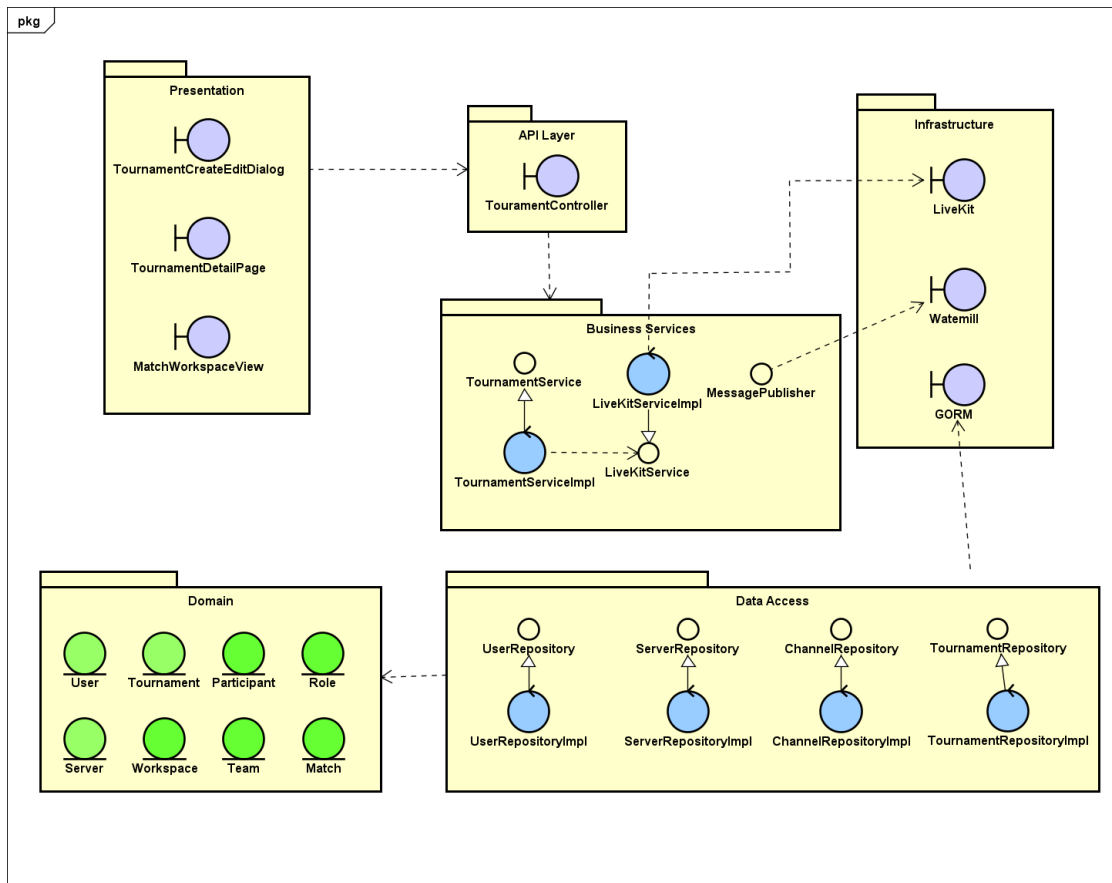


Hình 4.4: Biểu đồ chi tiết gói chức năng Tạo mới máy chủ

Hình 4.4 thể hiện các lớp, interface và gói tham gia vào quy trình người dùng tạo mới một máy chủ cộng đồng. Ở tầng trình diễn, *CreateServerModal* là thành phần giao diện tiếp nhận tên máy chủ, chế độ công khai và một số thông tin khởi tạo từ người dùng. Thành phần này gửi yêu cầu tạo máy chủ tới *ServerController* ở *API Layer*. Controller thực hiện kiểm tra xác thực, đọc dữ liệu đầu vào và chuyển yêu cầu xuống interface *ServerService*; phần cài đặt cụ thể là *serverService*.

Tại tầng *Business Services*, *serverService* kiểm tra thông tin bắt buộc, xác minh người tạo máy chủ qua *UserRepository*, sau đó tạo thực thể *Server* và quan hệ *ServerMember* đại diện cho chủ máy chủ. Các thao tác ghi dữ liệu được chuyển xuống *ServerRepository* và lớp cài đặt *serverRepository*. Repository phụ thuộc vào các model trong gói *Domain* và giao tiếp với *GormDatabase* trong gói *Infrastructure* để lưu dữ liệu. Với các thao tác có liên quan tới ảnh hoặc tài nguyên media của máy chủ, service có thể sử dụng thêm *StorageService/LocalStorage*. Cách thiết kế này giúp giao diện, điều phối API, nghiệp vụ tạo máy chủ và truy cập dữ liệu được tách biệt rõ ràng.

Tiếp theo là thiết kế chi tiết gói cho chức năng Tổ chức và vận hành giải đấu, được minh họa ở Hình 4.5.



Hình 4.5: Biểu đồ chi tiết gói chức năng Tổ chức và vận hành giải đấu

Hình 4.5 trình bày các gói và lớp chính tham gia vào nhóm chức năng Tổ chức và vận hành giải đấu. Ở tầng trình diễn, *TournamentCreateEditDialog* phục vụ tạo hoặc cập nhật thông tin giải đấu, *TournamentDetailPage* hiển thị thông tin tổng quan, danh sách người tham gia, đội, bracket và kết quả, còn *MatchWorkspaceView* phục vụ không gian theo dõi trận đấu. Các thành phần giao diện này gửi yêu cầu tới *TournamentController* ở tầng API.

Tầng *Business Services* được biểu diễn bởi interface *TournamentService* và lớp cài đặt *tournamentService*. Đây là nơi tập trung các quy tắc nghiệp vụ như kiểm tra quyền tổ chức trong máy chủ, tạo giải đấu, đăng ký tham gia, tạo đội, tạo bracket, cập nhật trạng thái trận, xử lý báo cáo kết quả và tạo workspace cho trận đấu. Service không thao tác trực tiếp với cơ sở dữ liệu mà phụ thuộc vào các repository như *TournamentRepository*, *ServerRepository*, *ChannelRepository* và *UserRepository*.

Gói *Domain* chứa các thực thể cốt lõi của nghiệp vụ giải đấu như *Tournament*, *Participant*, *Team*, *Match*, *Workspace* và *Role*. Gói *Infrastructure* cung cấp kết nối

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

lưu trữ qua *GormDatabase*, phát sự kiện thời gian thực qua *Watermill*, đồng thời hỗ trợ LiveKit cho các chức năng quan sát, voice hoặc livestream trong trận đấu. Cách tổ chức này giúp tách biệt rõ phần giao diện, điều phối API, xử lý nghiệp vụ, truy cập dữ liệu và các dịch vụ hạ tầng.

4.2 Thiết kế chi tiết

4.2.1 Thiết kế giao diện

Hệ thống hướng tới nhóm người dùng chính là người chơi, nhà phát triển game và người tổ chức giải đấu sử dụng trên máy tính cá nhân. Vì vậy, giao diện được thiết kế ưu tiên cho màn hình laptop và desktop, đặc biệt là các độ phân giải phổ biến như 1366 x 768, 1440 x 900 và 1920 x 1080 pixels. Với các màn hình có chiều rộng nhỏ hơn, giao diện vẫn cần đảm bảo khả năng co giãn ở các khu vực danh sách, vùng nội dung chính và panel phụ, tuy nhiên trải nghiệm tối ưu nhất vẫn là màn hình từ 13 inch trở lên. Hệ thống sử dụng màu hiển thị theo chuẩn 24-bit RGB thông dụng trên trình duyệt hiện đại, đồng thời hỗ trợ hiển thị tốt các ảnh đại diện, ảnh bìa máy chủ, thumbnail game và nội dung livestream.

Về định hướng thiết kế, giao diện của hệ thống đi theo phong cách ứng dụng cộng đồng thời gian thực, tương tự các nền tảng như Discord nhưng được điều chỉnh cho bài toán chơi game, đăng tải game và tổ chức giải đấu. Bố cục chính gồm thanh máy chủ cố định ở bên trái, thanh kênh hoặc danh sách hội thoại ở bên cạnh, vùng nội dung trung tâm và panel thành viên ở bên phải khi cần. Thanh máy chủ có kích thước cố định khoảng 72 pixels để người dùng chuyển nhanh giữa các cộng đồng. Khu vực còn lại được chia thành các panel ngang có thể thay đổi kích thước, giúp người dùng tập trung vào nội dung chính như trò chuyện, xem kênh voice, theo dõi bracket hoặc quản lý game.

Hệ thống sử dụng giao diện tối làm mặc định nhằm phù hợp với ngữ cảnh chơi game và sử dụng lâu trên màn hình. Nền chính có tông tối, các khối nội dung sử dụng màu card tối hơn hoặc sáng hơn nhẹ để tạo phân cấp thị giác. Màu chữ chính là màu sáng để đảm bảo khả năng đọc, màu chữ phụ dùng sắc xám để thể hiện thông tin ít quan trọng hơn. Màu nhấn được dùng cho trạng thái đang chọn, hiệu ứng hover, focus và các thao tác quan trọng. Các trạng thái đặc biệt được quy ước thống nhất: màu đỏ cho lỗi hoặc thao tác nguy hiểm, màu xanh cho trạng thái thành công, màu vàng/cam cho cảnh báo và màu xám cho trạng thái vô hiệu hóa.

Phông chữ chính của giao diện là Inter kết hợp nhóm phông sans-serif mặc định của hệ thống. Cỡ chữ cơ bản là 14 pixels, phù hợp với mật độ thông tin cao của các màn hình cộng đồng và quản lý giải đấu. Các tiêu đề màn hình, tên máy chủ, tên game hoặc tên giải đấu có cỡ chữ lớn hơn và dùng độ đậm cao hơn để tạo điểm

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

nhấn. Các mô tả phụ, thời gian gửi tin nhắn, trạng thái kênh và chú thích biểu mẫu dùng cỡ chữ nhỏ hơn, thường nằm trong khoảng 11 đến 13 pixels. Cách phân cấp chữ này giúp giao diện hiển thị nhiều dữ liệu nhưng vẫn giữ được khả năng đọc.

Các thành phần giao diện dùng chung được chuẩn hóa để đảm bảo tính nhất quán. Nút bấm có chiều cao mặc định khoảng 36 pixels, nút nhỏ khoảng 32 pixels, nút lớn khoảng 40 pixels và nút biểu tượng có kích thước vuông. Các loại nút chính gồm nút mặc định cho hành động quan trọng, nút phụ cho hành động ít nổi bật, nút viền cho thao tác trung gian, nút ghost cho thao tác phụ trong thanh công cụ và nút nguy hiểm cho hành động xóa hoặc hủy. Tất cả nút đều có trạng thái hover, focus và disabled để phản hồi rõ ràng cho người dùng.

Khung nhập liệu cũng có chiều cao khoảng 36 pixels, bo góc vừa phải, viền cùng màu hệ thống và hiệu ứng focus bằng vòng nhấn. Các biểu mẫu như tạo máy chủ, tạo kênh, cập nhật hồ sơ, đăng tải game hoặc tạo giải đấu đều sử dụng cùng một nguyên tắc: nhãn trường dữ liệu nằm gần trường nhập, mô tả phụ được đặt ngay bên dưới khi cần, lỗi nhập liệu hiển thị gần vị trí phát sinh lỗi để người dùng dễ sửa. Những thao tác cần xác nhận hoặc có nhiều thông tin được đặt trong hộp thoại trung tâm. Hộp thoại sử dụng lớp phủ mờ nền, bo góc lớn hơn, có tiêu đề, mô tả, nội dung chính và nhóm nút hành động ở cuối.

Thông điệp phản hồi của hệ thống được chia thành hai nhóm. Nhóm thứ nhất là thông báo nhanh dạng toast, thường xuất hiện ở góc dưới bên phải màn hình để phản hồi các thao tác ngắn như tạo máy chủ thành công, sao chép lời mời hoặc cập nhật hồ sơ. Nhóm thứ hai là thông báo trong ngữ cảnh, hiển thị trực tiếp gần khu vực thao tác, ví dụ lỗi tải ảnh, lỗi tạo lời mời, lỗi nhập tên không hợp lệ hoặc trạng thái đang lưu. Với các hành động có rủi ro như xóa dữ liệu, hệ thống ưu tiên sử dụng hộp thoại xác nhận thay vì chỉ hiển thị toast.

Hình 4.6 minh họa bản phác thảo giao diện máy chủ và kênh. Đây là màn hình trung tâm của hệ thống, nơi người dùng chuyển đổi giữa các máy chủ, chọn kênh văn bản, kênh voice, kênh livestream và theo dõi thành viên đang hoạt động. Bố cục gồm thanh máy chủ bên trái, danh sách kênh theo máy chủ, vùng nội dung chính dùng cho tin nhắn hoặc nội dung kênh và panel thành viên bên phải. Cách bố trí này giúp người dùng luôn nhận biết mình đang ở máy chủ nào, kênh nào và đang tương tác với những thành viên nào.

Giao-dien-may-chu-va-kenh

Anh tam thoi de build PDF. Hay thay bang hinh ve chinh thuc sau khi ve lai diagram.

Hình 4.6: Phác thảo giao diện màn hình máy chủ và kênh

Hình 4.7 minh họa nhóm giao diện liên quan đến game. Màn hình danh sách game cần hiển thị các thẻ game theo dạng lưới, mỗi thẻ có thumbnail, tên game, nhãn trạng thái và các thông tin phụ như lượt chơi hoặc chế độ chơi. Màn hình quản lý game của nhà phát triển tập trung vào biểu mẫu tạo thông tin game, tải lên ảnh đại diện, tải bản build và gửi kiểm duyệt. Thiết kế này tách rõ trải nghiệm của người chơi khi khám phá game với trải nghiệm của nhà phát triển khi đăng tải và quản lý sản phẩm.

Giao-dien-game-store-va-upload-game

Anh tam thoi de build PDF. Hay thay bang hinh ve chinh thuc sau khi ve lai diagram.

Hình 4.7: Phác thảo giao diện danh sách game và đăng tải game

Hình 4.8 minh họa bản phác thảo giao diện giải đấu. Màn hình này cần thể hiện thông tin tổng quan của giải, trạng thái đăng ký, danh sách người tham gia hoặc đội, bracket, lịch trận và khu vực thao tác theo vai trò. Người tổ chức có thể thấy các hành động quản lý như cập nhật giải đấu, chuyển trạng thái, tạo bracket hoặc xử lý kết quả. Người tham gia tập trung vào đăng ký, check-in, xem bracket và báo cáo kết quả. Các vai trò như trọng tài, bình luận viên hoặc khán giả chỉ nhìn thấy những thao tác phù hợp với quyền của mình. Nhờ đó, cùng một giao diện giải đấu có thể phục vụ nhiều vai trò mà không gây quá tải thông tin.



Hình 4.8: Phác thảo giao diện màn hình giải đấu

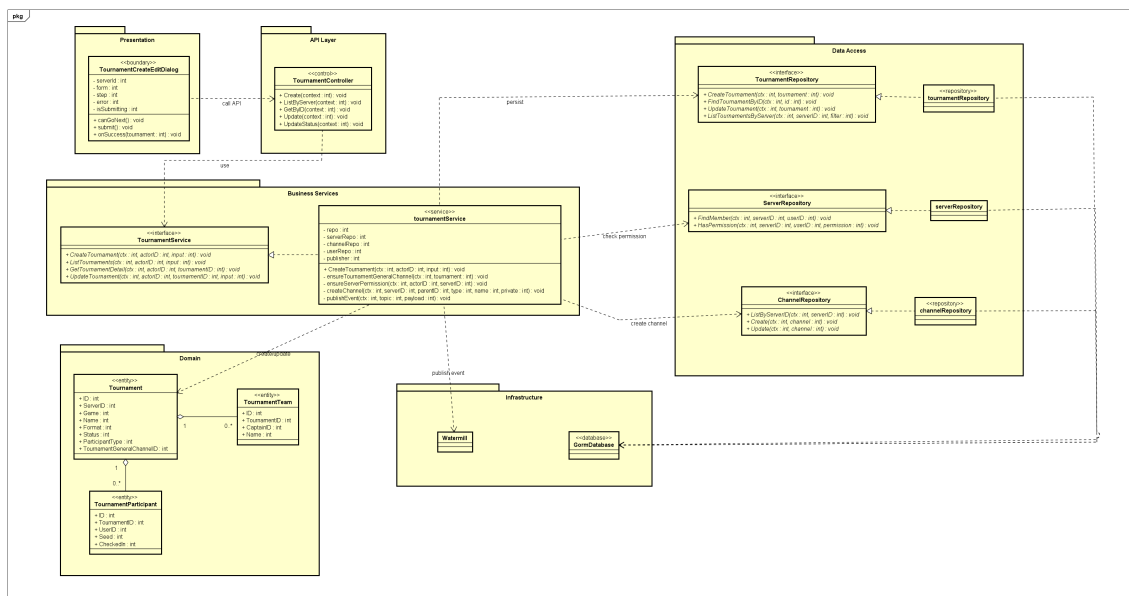
Nhìn chung, thiết kế giao diện của hệ thống tập trung vào ba mục tiêu: dễ điều hướng trong môi trường nhiều máy chủ và kênh, phản hồi nhanh cho các thao tác thời gian thực, và phân tách rõ trải nghiệm của người chơi, nhà phát triển game và người tổ chức giải đấu. Các bản phác thảo trong mục này đóng vai trò định hướng thiết kế, không phải ảnh chụp giao diện sản phẩm cuối cùng.

4.2.2 Thiết kế lớp

Trong mục này, đề án trình bày thiết kế lớp cho chức năng Tạo giải đấu. Đây là chức năng tiêu biểu của hệ thống vì bắt đầu từ một thao tác đơn giản trên giao diện nhưng cần phối hợp nhiều lớp ở cả frontend và backend: hộp thoại tạo giải đấu, lớp điều khiển API, interface nghiệp vụ, lớp cài đặt nghiệp vụ, các repository kiểm tra quyền và truy cập dữ liệu, cùng thành phần phát sự kiện thời gian thực. Luồng chính bắt đầu khi người dùng có quyền quản lý trong máy chủ nhấn nút tạo giải đấu, nhập thông tin cần thiết và gửi biểu mẫu lên hệ thống.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Hình 4.9 minh họa các lớp chính tham gia vào use case Tạo giải đấu theo các tầng đã trình bày ở mục thiết kế gói. Lớp *TournamentCreateEditDialog* đại diện cho giao diện nhập liệu ở tầng *Presentation*. Lớp *TournamentController* thuộc *API Layer*, tiếp nhận request tạo giải đấu và chuyển dữ liệu xuống interface *TournamentService*. Lớp cài đặt *tournamentService* giữ vai trò trung tâm, chịu trách nhiệm chuẩn hóa dữ liệu, kiểm tra quyền qua *ServerRepository*, tạo đối tượng *Tournament*, tạo kênh thảo luận chung qua *ChannelRepository*, lưu dữ liệu qua *TournamentRepository* và phát sự kiện qua *Watermill*. Nhờ cách tách interface và lớp cài đặt, tầng nghiệp vụ không phụ thuộc trực tiếp vào chi tiết truy cập cơ sở dữ liệu.



Hình 4.9: Thiết kế lớp chức năng Tạo giải đấu

Bảng 4.1 trình bày các thuộc tính và phương thức quan trọng của lớp *Tournament*. Đây là thực thể trung tâm của chức năng giải đấu, lưu thông tin định danh, máy chủ sở hữu, game được tổ chức thi đấu, thể thức, trạng thái, số lượng người tham gia và các thiết lập liên quan đến đăng ký/check-in.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Thiết kế chi tiết lớp Tournament	
Thành phần	Mô tả
ID	Mã định danh duy nhất của giải đấu.
ServerID	Mã máy chủ nơi giải đấu được tạo và vận hành.
Name, Description	Tên và mô tả tổng quan của giải đấu.
Game	Tên game hoặc bộ môn được sử dụng trong giải đấu.
Format	Thế thức thi đấu, ví dụ loại trực tiếp, nhánh thắng/thua, vòng tròn hoặc Swiss.
Status	Trạng thái vòng đời của giải đấu như nháp, mở đăng ký, check-in, đang diễn ra, hoàn thành hoặc hủy.
MaxParticipants	Số lượng người/đội tham gia tối đa.
ParticipantType, TeamSize	Kiểu tham gia solo hoặc theo đội; nếu theo đội thì có thêm kích thước đội.
RegistrationDeadline	Thời hạn đăng ký tham gia giải đấu.
CheckInDurationMinutes	Thời lượng check-in trước khi bắt đầu giải.
PrizePool, Rules	Thông tin giải thưởng và luật thi đấu.
CreatedBy	Người dùng tạo giải đấu.
TournamentGeneralChannelID	Kênh thảo luận chung được tạo/gắn với giải đấu.
TableName()	Trả về tên bảng dữ liệu tương ứng với lớp.
BeforeCreate()	Tự sinh ID, thiết lập trạng thái nháp và thời lượng check-in mặc định nếu thiếu.

Bảng 4.1: Thiết kế chi tiết lớp Tournament

Bảng 4.2 mô tả lớp *TournamentCreateEditDialog*. Lớp này không phải thực thể dữ liệu trong backend mà là lớp giao diện quan trọng của use case. Nó quản lý biểu mẫu tạo giải đấu theo nhiều bước, kiểm tra sơ bộ dữ liệu nhập và gọi API tạo giải đấu khi người dùng xác nhận.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Thiết kế chi tiết lớp TournamentCreateEditDialog	
Thành phần	Mô tả
serverId	Mã máy chủ hiện tại, dùng để gửi request tạo giải đấu đúng ngữ cảnh.
form	Trạng thái biểu mẫu, gồm tên giải, game, mô tả, luật, giải thưởng, thể thức, kiểu tham gia, số lượng tối đa và thời hạn đăng ký.
step	Bước hiện tại của biểu mẫu tạo giải đấu.
error	Thông báo lỗi hiển thị trên giao diện nếu lưu thất bại.
isSubmitting	Cờ trạng thái cho biết hệ thống đang gửi request tạo/cập nhật giải đấu.
canGoNext()	Kiểm tra điều kiện để người dùng chuyển sang bước tiếp theo của biểu mẫu.
submit()	Tạo payload từ biểu mẫu, gọi API tạo giải đấu và xử lý kết quả trả về.
onSuccess(tournament)	Callback được gọi khi tạo giải đấu thành công, thường dùng để làm mới danh sách và điều hướng.

Bảng 4.2: Thiết kế chi tiết lớp TournamentCreateEditDialog

Bảng 4.3 trình bày lớp cài đặt *tournamentService* cho interface *TournamentService*. Đây là lớp xử lý nghiệp vụ chính, nơi tập trung các quy tắc như người tạo phải là thành viên máy chủ, phải có quyền quản lý phù hợp, thể thức thi đấu phải hợp lệ, số lượng người tham gia phải lớn hơn một và đội phải có kích thước hợp lệ nếu giải đấu theo đội.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Thiết kế chi tiết lớp TournamentService	
Thành phần	Mô tả
repo	Repository thao tác với dữ liệu giải đấu, người tham gia, đội và trận đấu.
serverRepo	Repository kiểm tra thành viên và quyền của người dùng trong máy chủ.
channelRepo	Repository dùng để tìm, tạo hoặc cập nhật kênh thảo luận chung của giải đấu.
userRepo, liveKitSvc, publisher	Các thành phần phụ trợ được dùng ở các nghiệp vụ giải đấu khác như vai trò, workspace trận, voice/livestream và sự kiện thời gian thực.
CreateTournament(ctx, actorID, input)	Tạo giải đấu mới sau khi chuẩn hóa dữ liệu, kiểm tra quyền và kiểm tra ràng buộc nghiệp vụ.
ensureTournamentGeneralChannel(ctx, tournament)	Bảo đảm mỗi giải đấu có một kênh văn bản chung để thảo luận và theo dõi thông tin.
ensureServerPermission(ctx, actorID, serverID)	Kiểm tra người dùng là thành viên máy chủ và có quyền quản lý cần thiết.
createChannel(ctx, serverID, parentID, type, name, private)	Tạo kênh mới trong máy chủ khi giải đấu chưa có kênh chung.
publishEvent(ctx, topic, payload)	Phát sự kiện sau khi tạo giải đấu thành công để các thành phần thời gian thực có thể đồng bộ.

Bảng 4.3: Thiết kế chi tiết lớp TournamentService

Bảng 4.4 mô tả interface *TournamentRepository*. Interface này tách tầng nghiệp vụ khỏi tầng truy cập dữ liệu, đồng thời gom các thao tác liên quan đến giải đấu vào một nhóm rõ ràng. Trong use case Tạo giải đấu, các phương thức quan trọng nhất là tạo giải đấu mới, tìm giải đấu theo định danh và cập nhật lại giải đấu sau khi hệ thống gắn kênh thảo luận chung.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

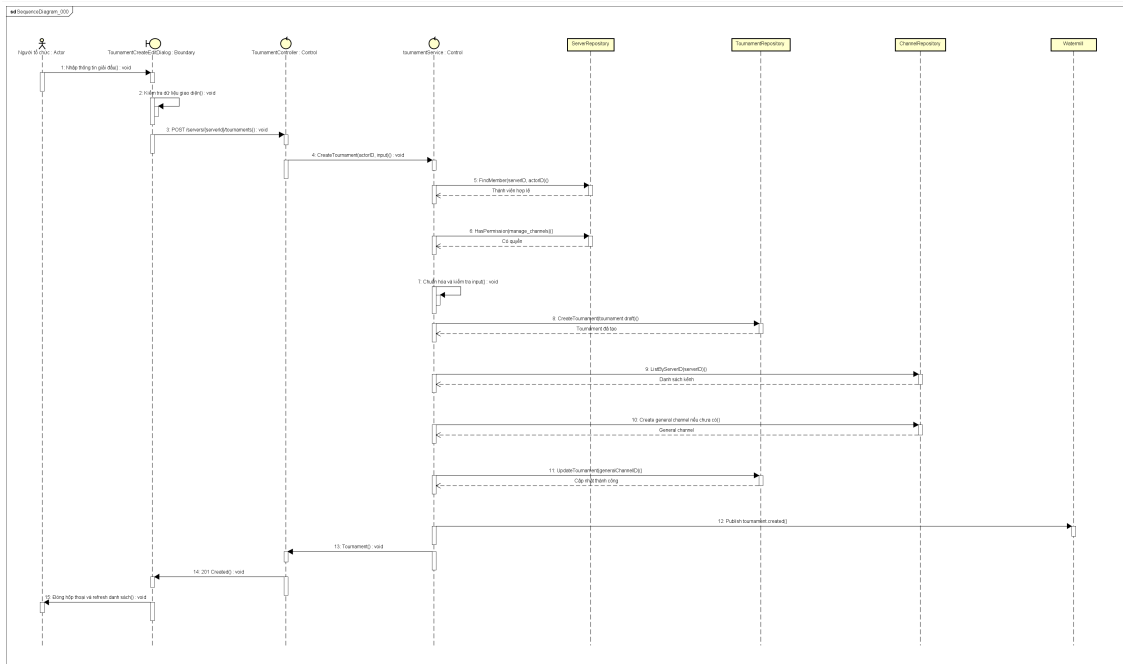
Thiết kế chi tiết interface TournamentRepository	
Thành phần	Mô tả
CreateTournament(ctx, tournament)	Lưu một giải đấu mới vào cơ sở dữ liệu.
ListTournamentsByServer(ctx, serverID, filter)	Lấy danh sách giải đấu trong một máy chủ theo điều kiện lọc.
FindTournamentByID(ctx, id)	Tìm giải đấu theo mã định danh.
UpdateTournament(ctx, tournament)	Cập nhật thông tin giải đấu, ví dụ lưu kênh thảo luận chung sau khi tạo.
CountParticipants(ctx, tournamentID)	Đếm số lượng người/đội tham gia giải đấu.
ListParticipants(ctx, tournamentID)	Lấy danh sách người/đội tham gia để phục vụ màn hình chi tiết giải đấu.

Bảng 4.4: Thiết kế chi tiết interface TournamentRepository

Để minh họa luồng truyền thông điệp giữa các đối tượng, Hình 4.10 trình bày biểu đồ trình tự của use case Tạo giải đấu. Luồng bắt đầu khi người dùng nhấn nút tạo giải đấu trong máy chủ. Hộp thoại tạo giải đấu hiển thị biểu mẫu nhiều bước để người dùng nhập tên giải, game, thể thức, kiểu tham gia, số lượng tối đa, thời hạn đăng ký, luật và giải thưởng. Trước khi gửi request, giao diện kiểm tra các điều kiện cơ bản như tên giải và game không được rỗng, số lượng người tham gia hợp lệ, kích thước đội hợp lệ nếu chọn giải đấu theo đội.

Khi người dùng xác nhận tạo, frontend gửi request tới API tạo giải đấu của máy chủ. Controller lấy thông tin người dùng hiện tại, đọc dữ liệu JSON và gọi interface *TournamentService*; lời gọi này được lớp *tournamentService* xử lý. Tại tầng nghiệp vụ, hệ thống chuẩn hóa các trường đầu vào, kiểm tra người dùng có thuộc máy chủ hay không và có quyền quản lý phù hợp hay không. Sau đó, service kiểm tra thể thức giải đấu, kiểu tham gia, số lượng tối đa và kích thước đội. Nếu một trong các điều kiện không hợp lệ, lỗi nghiệp vụ được trả về để giao diện hiển thị cho người dùng.

Nếu dữ liệu hợp lệ, service tạo đối tượng *Tournament* ở trạng thái *draft* và yêu cầu repository lưu vào cơ sở dữ liệu. Sau khi tạo thành công, service bảo đảm giải đấu có kênh thảo luận chung bằng cách tìm danh sách kênh trong máy chủ, sinh tên kênh theo tên giải đấu và tạo kênh văn bản mới nếu chưa tồn tại. Mã kênh được cập nhật ngược lại vào giải đấu, sau đó service phát sự kiện *TOURNAMENT_CREATED*. Kết quả cuối cùng được trả về frontend, hộp thoại được đóng và danh sách giải đấu trong máy chủ được làm mới.



Hình 4.10: Biểu đồ trình tự use case Tạo giải đấu

4.2.3 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng cơ sở dữ liệu quan hệ để lưu trữ các dữ liệu bền vững như tài khoản, máy chủ, kênh, tin nhắn, game và giải đấu. Các bảng được thiết kế theo hướng chuẩn hóa quan hệ, trong đó mỗi bảng đại diện cho một thực thể hoặc một quan hệ nhiều-nhiều quan trọng. Khóa chính của phần lớn các bảng sử dụng kiểu *CHAR(36)* để lưu UUID, giúp định danh dữ liệu ổn định khi hệ thống mở rộng hoặc đồng bộ giữa nhiều thành phần. Các mốc thời gian được lưu bằng kiểu số nguyên lớn, phù hợp với cách xử lý timestamp thống nhất trong backend. Một số trường có cấu trúc linh hoạt như nội dung tin nhắn, thẻ game hoặc trạng thái phòng chơi sử dụng kiểu JSON.

Để biểu đồ thực thể liên kết dễ theo dõi, đồ án chia cơ sở dữ liệu thành ba cụm chính tương ứng với ba nhóm chức năng quan trọng của hệ thống: cụm cộng đồng và máy chủ, cụm game, và cụm giải đấu. Các bảng dùng chung như *users*, *servers* hoặc *channels* có thể xuất hiện trong nhiều cụm vì đây là các thực thể nền tảng được nhiều nghiệp vụ sử dụng. Cách chia này giúp việc trình bày rõ ràng hơn so với việc đưa toàn bộ bảng vào một sơ đồ duy nhất.

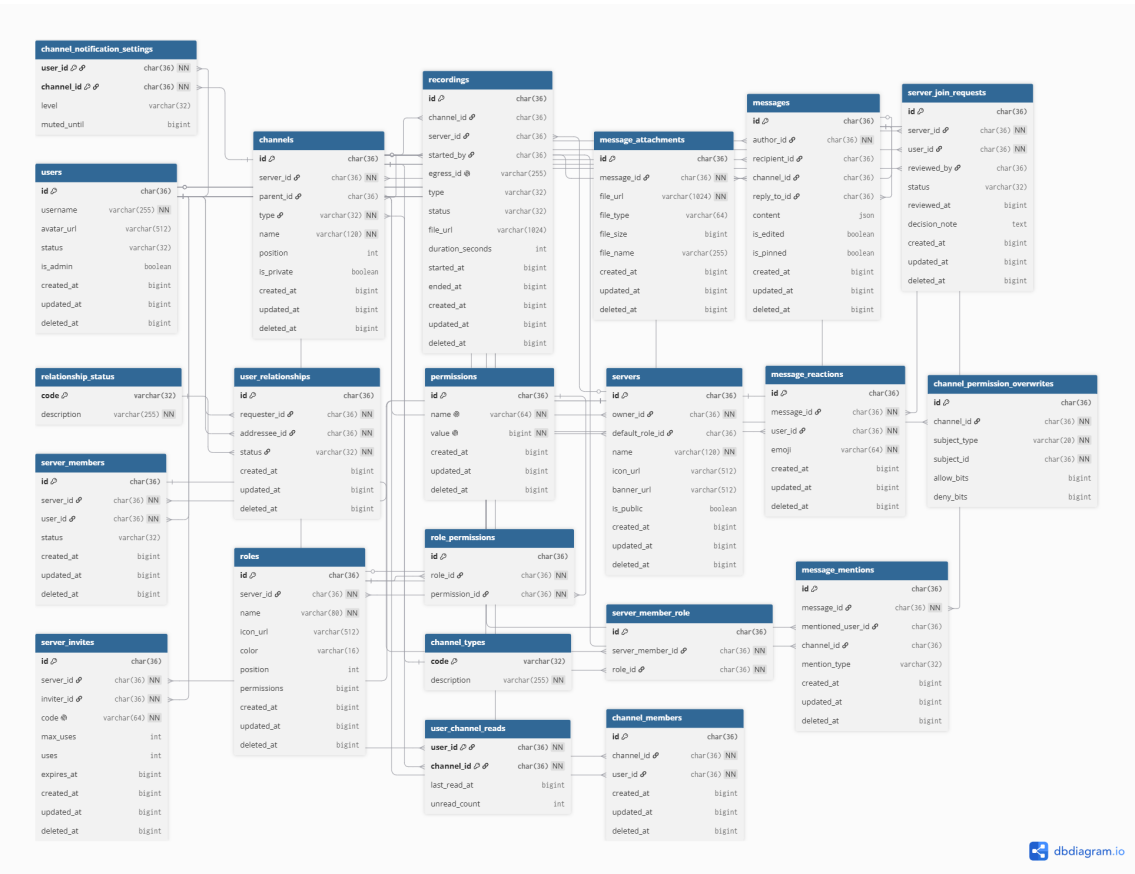
Bảng 4.5 tóm tắt phạm vi của ba cụm dữ liệu được sử dụng trong thiết kế cơ sở dữ liệu.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Tổng hợp các cụm dữ liệu chính của hệ thống		
Cụm dữ liệu	Mục đích	Một số bảng chính
Cộng đồng và máy chủ	Lưu tài khoản, quan hệ bạn bè, máy chủ, thành viên, vai trò, kênh và tin nhắn.	users, servers, server_members, roles, channels, messages
Game	Lưu thông tin game, bản build, đánh giá, phiên chơi, phòng chơi và sự kiện gameplay.	user_games, user_game_builds, game_ratings, game_sessions, game_rooms
Giải đấu	Lưu giải đấu, đội, người tham gia, bracket, trận đấu, kết quả, vai trò giải đấu và workspace trận.	tournaments, tournament_participants, tournament_matches, tournament_roles, tournament_match_workspaces

Bảng 4.5: Tổng hợp các cụm dữ liệu chính của hệ thống

Hình 4.11 trình bày sơ đồ thực thể liên kết của cụm cộng đồng và máy chủ. Đây là cụm dữ liệu nền tảng nhất của hệ thống. Bảng *users* lưu thông tin tài khoản người dùng, đồng thời liên kết tới các bảng quan hệ bạn bè, thành viên máy chủ, tin nhắn và phản ứng tin nhắn. Bảng *servers* đại diện cho một cộng đồng hoặc nhóm chơi game, có chủ sở hữu là một người dùng và chứa nhiều thành viên, vai trò, lời mời, yêu cầu tham gia và kênh.



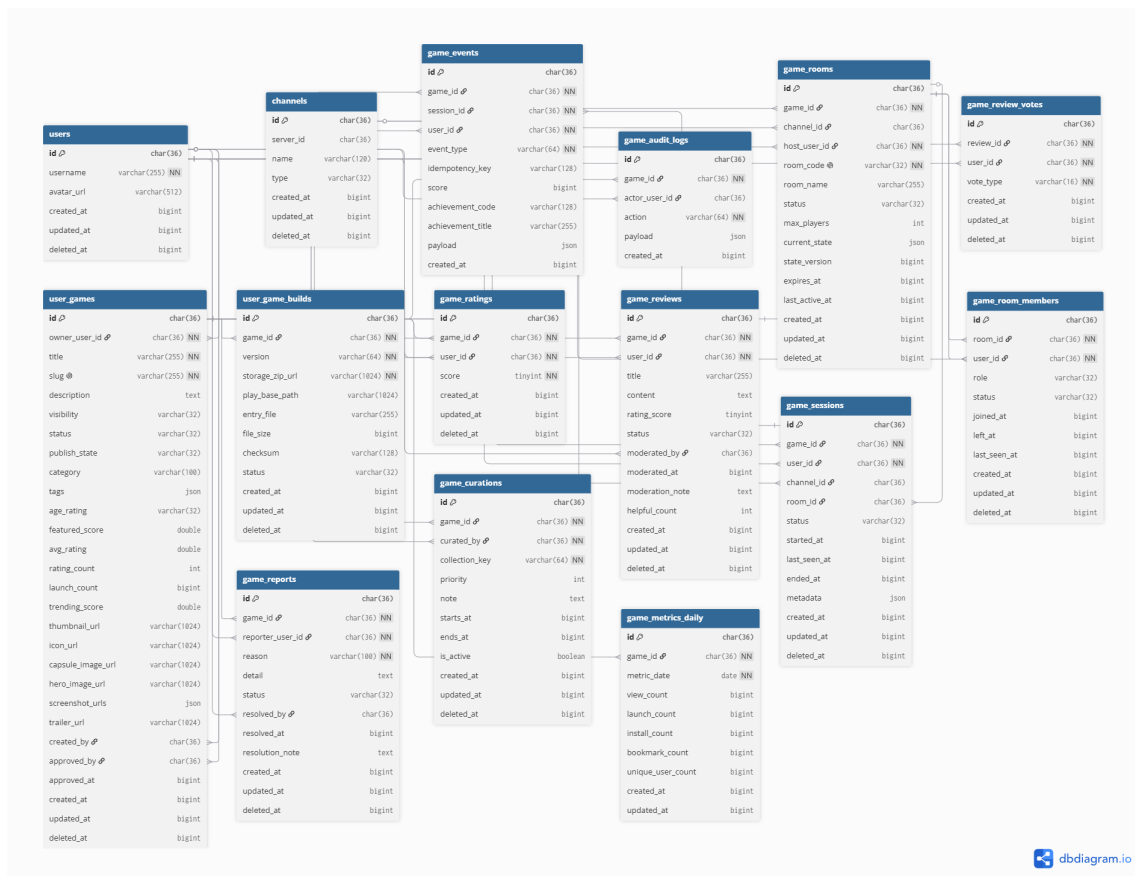
Hình 4.11: Sơ đồ E-R cụm dữ liệu cộng đồng và máy chủ

Trong cụm này, quan hệ giữa *users* và *servers* được tách qua bảng *server_members* để thể hiện quan hệ nhiều-nhiều: một người dùng có thể tham gia nhiều máy chủ, và một máy chủ có nhiều thành viên. Phân quyền trong máy chủ được thiết kế qua

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

các bảng *roles* và *server_member_role*; mỗi vai trò thuộc một máy chủ, còn thành viên có thể được gán nhiều vai trò khác nhau. Đối với kênh riêng tư, hệ thống sử dụng *channel_members* để xác định người được truy cập trực tiếp và *channel_permission_overwrites* để ghi đè quyền theo người dùng hoặc vai trò. Nhóm bảng *messages*, *message_attachments*, *message_reactions*, *message_mentions* và *user_channel_reads* phục vụ nghiệp vụ nhắn tin, đính kèm, thả phản ứng, nhắc tên và đánh dấu đã đọc.

Hình 4.12 thể hiện cụm dữ liệu game. Bảng trung tâm của cụm này là *user_games*, lưu thông tin game do hệ thống hoặc cộng đồng đăng tải. Mỗi game thuộc một người dùng sở hữu, có trạng thái hiển thị, trạng thái công bố, thông tin phân loại, điểm đánh giá và các thông tin media như icon, ảnh capsule, ảnh hero hoặc trailer. Bảng *user_game_builds* lưu các bản build của game; một game có thể có nhiều bản build khác nhau, trong đó backend ưu tiên bản build mới nhất có trạng thái sẵn sàng khi người dùng bắt đầu chơi.



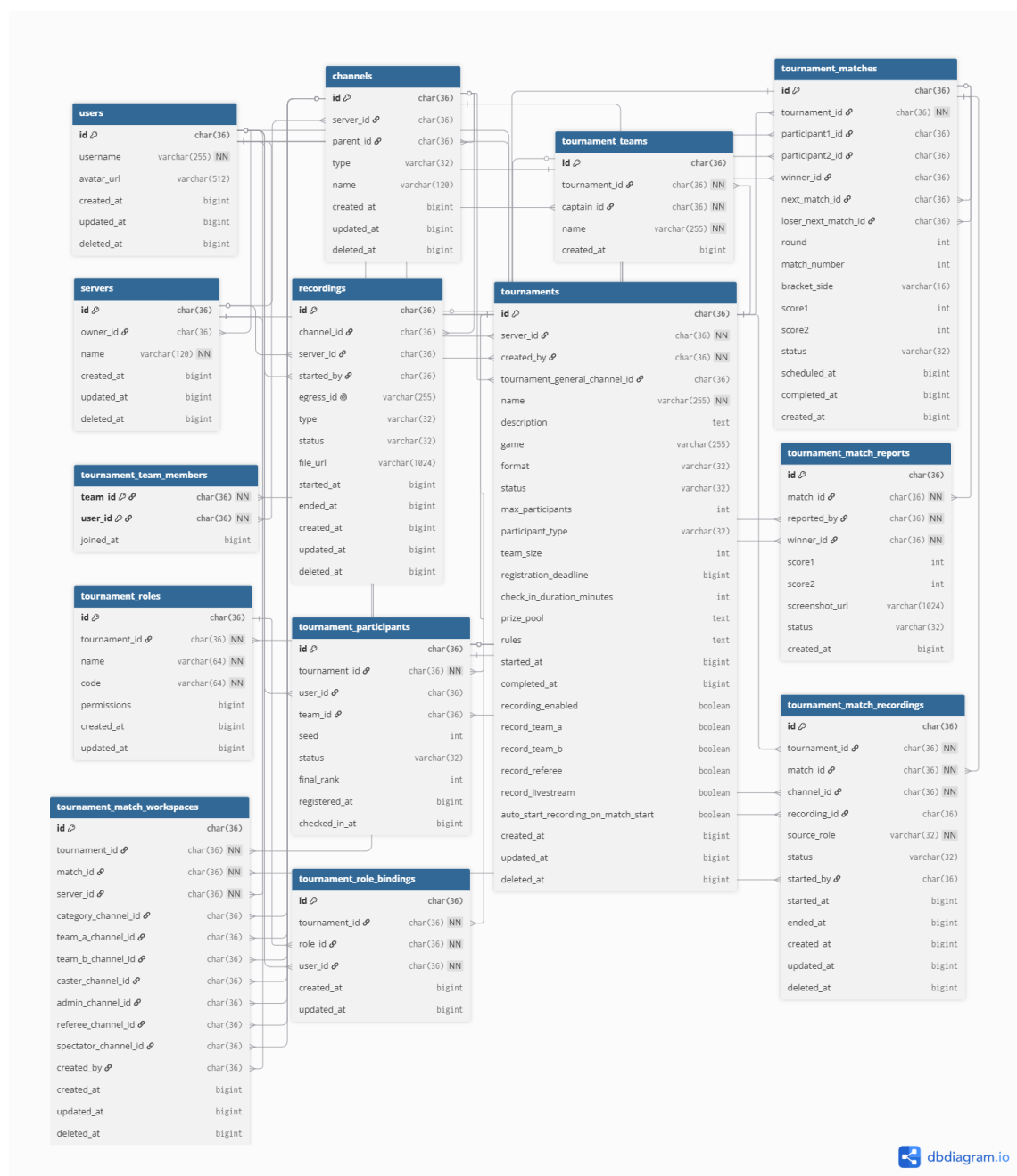
Hình 4.12: Sơ đồ E-R cụm dữ liệu game

Các bảng *game_ratings*, *game_reviews*, *game_review_votes* và *game_reports* phục vụ phần marketplace của game, cho phép người dùng đánh giá, viết nhận xét, biểu quyết nhận xét hữu ích và báo cáo nội dung vi phạm. Đối với quá trình chơi

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

game, bảng *game_sessions* lưu phiên chơi của người dùng, *game_events* ghi nhận sự kiện gameplay như điểm số hoặc thành tích, còn *game_rooms* và *game_room_members* lưu phòng chơi nhiều người. Một số bảng có liên kết tới *channels* để game có thể được chơi hoặc chia sẻ trong ngữ cảnh một kênh cụ thể của máy chủ.

Hình 4.13 trình bày cụm dữ liệu giải đấu. Bảng *tournaments* là thực thể trung tâm, lưu thông tin giải đấu như máy chủ tổ chức, tên giải, game thi đấu, thể thức, trạng thái, số lượng người tham gia tối đa, kiểu tham gia solo/team, thời hạn đăng ký, luật và giải thưởng. Trường *tournament_general_channel_id* liên kết giải đấu với một kênh thảo luận chung trong máy chủ.



Hình 4.13: Sơ đồ E-R cụm dữ liệu giải đấu

Trong cụm giải đấu, nếu giải theo đội, bảng *tournament_teams* lưu thông tin đội và đội trưởng, còn *tournament_team_members* lưu thành viên của từng đội. Bảng *tournament_participants* đóng vai trò chuẩn hóa người tham gia, cho phép hệ thống xử lý thống nhất cả giải solo lẫn giải theo đội. Khi giải đấu bắt đầu, các trận đấu được sinh ra trong bảng *tournament_matches*; bảng này lưu vòng đấu, số trận, hai người/đội tham gia, người thắng, điểm số và liên kết tới trận tiếp theo trong bracket. Kết quả do người tham gia hoặc trọng tài báo cáo được lưu trong *tournament_match_reports*.

Ngoài nhóm bảng bracket cơ bản, hệ thống còn có các bảng phục vụ vận hành giải đấu theo vai trò. *tournament_roles* và *tournament_role_bindings* lưu các vai trò như quản trị giải đấu, trọng tài, bình luận viên, khán giả hoặc người chơi. *tournament_match_workspaces* lưu các kênh riêng được tạo cho từng trận, gồm kênh đội A, đội B, caster, referee, spectator, admin và livestream. Khi bật ghi hình, *tournament_match_recordings* lưu trạng thái ghi hình của từng kênh trong trận. Thiết kế này giúp nghiệp vụ giải đấu tách biệt với nghiệp vụ máy chủ chung nhưng vẫn tận dụng được các bảng nền tảng như *users*, *servers* và *channels*.

Về mặt triển khai vật lý, cơ sở dữ liệu sử dụng hệ quản trị MySQL với các bảng InnoDB và bộ mã ký tự *utf8mb4*. Các quan hệ quan trọng đều được ràng buộc bằng khóa ngoại để bảo toàn toàn vẹn dữ liệu. Với các bảng con phụ thuộc trực tiếp vào bảng cha, ví dụ *server_members* thuộc *servers*, *user_game_builds* thuộc *user_games*, hoặc *tournament_matches* thuộc *tournaments*, hệ thống sử dụng chiến lược xóa dây chuyền khi dữ liệu cha bị xóa. Với các quan hệ tùy chọn như kênh cha của một kênh, tin nhắn được trả lời, hoặc kênh general của giải đấu, hệ thống sử dụng chiến lược đặt về rỗng khi bản ghi được tham chiếu không còn tồn tại.

Các chỉ mục được thiết kế theo các truy vấn thường gặp của hệ thống: truy vấn thành viên theo máy chủ, truy vấn kênh theo máy chủ và thứ tự hiển thị, truy vấn tin nhắn theo kênh, truy vấn game theo trạng thái công bố hoặc điểm xu hướng, truy vấn giải đấu theo máy chủ và trạng thái, truy vấn trận đấu theo giải đấu, vòng đấu và trạng thái. Nhờ đó, cơ sở dữ liệu vừa đảm bảo tính toàn vẹn của quan hệ, vừa hỗ trợ tốt các luồng nghiệp vụ có tần suất cao như mở máy chủ, xem kênh, nhắn tin, duyệt game và theo dõi giải đấu.

4.3 Xây dựng ứng dụng

Sau khi hoàn thành phân tích và thiết kế, hệ thống được xây dựng thành nhiều thành phần có thể chạy độc lập nhưng phối hợp với nhau qua API, WebSocket và cơ sở dữ liệu dùng chung. Phần backend được phát triển bằng Go để xử lý nghiệp vụ chính, xác thực, phân quyền, quản lý máy chủ, game và giải đấu. Phần frontend

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

được xây dựng bằng React và TypeScript, bao gồm ứng dụng web và ứng dụng desktop. Các thành phần hạ tầng như MySQL, Redis và LiveKit được cấu hình bằng Docker Compose để thuận tiện cho quá trình phát triển, kiểm thử và triển khai thử nghiệm.

4.3.1 Thư viện và công cụ sử dụng

Bảng 4.6 liệt kê các ngôn ngữ, thư viện và công cụ chính được sử dụng trong quá trình xây dựng hệ thống. Các phiên bản được ghi nhận theo cấu hình của mã nguồn và môi trường phát triển hiện tại.

Danh sách thư viện và công cụ sử dụng			
Mục đích	Công cụ/Thư viện	Phiên bản	Địa chỉ URL
Ngôn ngữ backend	Go	1.25.3	https://go.dev/
Web framework backend	Gin	1.11.0	https://gin-gonic.com/
Truy cập dữ liệu	GORM, MySQL Driver	1.31.1, 1.6.0	https://gorm.io/
Quản lý migration	Goose	3.27.0	https://github.com/pressly/goose
Xác thực	JWT for Go	5.3.0	https://github.com/golang-jwt/jwt
Cơ sở dữ liệu	MySQL Docker Image	8.4	https://www.mysql.com/
Cache và trạng thái tạm	Redis, Go Redis	7-alpine, 9.18.0	https://redis.io/
Realtime media	LiveKit	latest, Client 2.17.3	https://livekit.io/
Ngôn ngữ frontend	TypeScript	5.5.3	https://www.typescriptlang.org/
Giao diện web	React, Vite, Tailwind CSS	18.3.1, 5.3.4, 3.4.6	https://react.dev/
Ứng dụng desktop	Electron, Electron Builder	28.3.3, 24.13.3	https://www.electronjs.org/
Đóng gói môi trường	Docker, Docker Compose	29.2.0, 5.0.2	https://www.docker.com/
Kiểm thử giao diện	Playwright	1.55.0	https://playwright.dev/
IDE và quản lý mã nguồn	Visual Studio Code, Git	1.123.0, 2.45.1	https://code.visualstudio.com/

Bảng 4.6: Danh sách thư viện và công cụ sử dụng

Ngoài các công cụ chính trong bảng trên, đồ án còn sử dụng một số thư viện hỗ trợ như Axios để gọi API, React Query để quản lý dữ liệu bất đồng bộ, Zustand để quản lý trạng thái phía frontend, Cloudinary SDK để hỗ trợ lưu trữ tài nguyên media và các thư viện WebSocket/message queue phục vụ những tính năng thời gian thực. Các thư viện này không được trình bày tách riêng để tránh làm loãng nội dung, nhưng đều đóng vai trò hỗ trợ cho các nhóm chức năng đã phân tích ở Chương 2.

4.3.2 Kết quả đạt được

Kết quả xây dựng của đồ án là một hệ thống cộng đồng chơi game có thể chạy theo mô hình web, desktop và backend service. Backend cung cấp API và Web-Socket cho các nghiệp vụ tài khoản, bạn bè, máy chủ, kênh, tin nhắn, game và giải

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

đầu. Frontend web cung cấp giao diện chính cho người dùng cuối, trong khi bản desktop đóng gói giao diện thành ứng dụng chạy trên Windows. Bên cạnh đó, đồ án còn có game SDK phục vụ việc tích hợp game vào hệ thống và cấu hình Docker Compose để khởi tạo nhanh các dịch vụ hạ tầng cần thiết.

Bảng 4.7 trình bày các sản phẩm chính thu được sau quá trình xây dựng và đóng gói trong môi trường phát triển.

Các sản phẩm xây dựng và đóng gói			
Sản phẩm	Vai trò	Thư mục/Kết quả	Dung lượng
Backend service	Cung cấp REST API, WebSocket, xử lý nghiệp vụ và kết nối cơ sở dữ liệu.	backend, Dockerfile backend	Theo Docker image
Web frontend	Giao diện web cho người dùng thao tác với máy chủ, game và giải đấu.	frontend/apps/web/dist	3.92 MB
Desktop app	Phiên bản ứng dụng desktop đóng gói cho Windows.	frontend/apps/desktop/release/windows-unpacked	262.73 MB
Game SDK	Bộ thư viện hỗ trợ game giao tiếp với hệ thống.	game-sdk/dist	0.03 MB
Hạ tầng Docker	Khởi tạo MySQL, Redis, LiveKit và các dịch vụ phụ trợ khi phát triển.	docker-compose.yml	Theo image

Bảng 4.7: Các sản phẩm xây dựng và đóng gói

Về mặt mã nguồn, hệ thống được tổ chức thành các thành phần chính gồm backend, frontend, game SDK và các module realtime phụ trợ. Bảng 4.8 thống kê quy mô mã nguồn hiện tại của đồ án. Khi thống kê, các thư mục phụ thuộc và kết quả build như *node_modules*, *dist*, *release*, *build*, *coverage*, *.git* được loại bỏ để phản ánh đúng phần mã nguồn do đồ án quản lý.

Thống kê tổng quan mã nguồn		
Thông tin thống kê	Giá trị	Ghi chú
Tổng số tệp trong cây mã nguồn	1102	Không tính thư mục phụ thuộc và build
Số tệp mã nguồn văn bản	780	Bao gồm code, tài liệu, cấu hình
Số tệp code chính	562	Go, TypeScript, TSX, JavaScript, SQL
Tổng số dòng mã nguồn văn bản	130273	Tính theo tệp văn bản
Tổng số dòng code chính	95314	Không tính tài liệu thuần túy
Dung lượng cây mã nguồn	7.66 MB	Không tính kết quả đóng gói
Dung lượng tệp mã nguồn văn bản	4.84 MB	Không tính tài nguyên nhị phân lớn
Số lượng struct/interface Go	289 / 29	Thống kê xấp xỉ từ mã nguồn Go
Số component React	181	Thống kê xấp xỉ từ mã nguồn frontend
Số module Go / package npm	3 / 15	Theo go.mod và package.json

Bảng 4.8: Thống kê tổng quan mã nguồn

Bảng 4.9 thống kê dung lượng theo từng nhóm thư mục chính. Có thể thấy

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

backend và frontend là hai phần lớn nhất của hệ thống, tương ứng với tầng xử lý nghiệp vụ và tầng giao diện đã trình bày trong kiến trúc tổng quan.

Thống kê theo nhóm thư mục chính			
Thành phần	Số tệp	Dung lượng	Vai trò chính
backend	236	2.77 MB	API, nghiệp vụ, dữ liệu
frontend	588	3.00 MB	Web, desktop, giao diện
game-realtime	12	0.02 MB	Module realtime game
notification	18	0.04 MB	Module thông báo realtime
game-sdk	14	0.08 MB	SDK tích hợp game
docs	74	0.33 MB	Tài liệu và biểu đồ

Bảng 4.9: Thống kê theo nhóm thư mục chính

Nhìn chung, đồ án đã xây dựng được đầy đủ các thành phần cốt lõi để vận hành hệ thống: backend xử lý nghiệp vụ và dữ liệu, frontend phục vụ tương tác người dùng, desktop app phục vụ trải nghiệm cài đặt, game SDK phục vụ tích hợp game, cùng môi trường Docker để khởi tạo các dịch vụ phụ trợ. Các kết quả này là cơ sở để triển khai thử nghiệm, kiểm thử nghiệp vụ và tiếp tục hoàn thiện hệ thống trong các giai đoạn sau.

4.4 Kiểm thử

Trong quá trình phát triển hệ thống, đồ án sử dụng kiểm thử hộp đen để đánh giá các chức năng quan trọng từ góc nhìn đầu vào và đầu ra của hệ thống. Với cách tiếp cận này, người kiểm thử không cần phụ thuộc vào cấu trúc bên trong của mã nguồn mà tập trung vào việc gửi yêu cầu, quan sát phản hồi, trạng thái dữ liệu và sự kiện trả về. Kỹ thuật này phù hợp với hệ thống vì phần lớn nghiệp vụ được cung cấp thông qua REST API và WebSocket, có thể kiểm tra bằng các kịch bản sử dụng tương tự người dùng thực tế.

Các kỹ thuật kiểm thử được sử dụng gồm phân hoạch tương đương, kiểm thử giá trị không hợp lệ và kiểm thử luồng nghiệp vụ. Phân hoạch tương đương được áp dụng cho các trường hợp dữ liệu hợp lệ như đăng ký tài khoản, đăng nhập, tạo máy chủ, tạo kênh hoặc gửi tin nhắn. Kiểm thử dữ liệu không hợp lệ được dùng cho các trường hợp thiếu token, loại kênh không hợp lệ hoặc người dùng không có quyền truy cập tài nguyên. Với chức năng nhắn tin, đồ án bổ sung kiểm thử sự kiện thời gian thực bằng cách mở kết nối WebSocket cho hai người dùng và kiểm tra sự kiện được phát sau khi tin nhắn được tạo.

Môi trường kiểm thử được thiết lập cục bộ bằng Docker Compose, bao gồm cơ sở dữ liệu MySQL, Redis và message broker phục vụ các thành phần real-

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

time. Backend chạy ở địa chỉ *localhost:8080*, còn dịch vụ WebSocket thông báo chạy ở *localhost:8085*. Các kịch bản kiểm thử API được thực hiện theo dạng request/response, trong khi kịch bản realtime sử dụng client WebSocket để ghi nhận sự kiện gửi về cho người dùng liên quan.

Đề án lựa chọn ba nhóm chức năng tiêu biểu để kiểm thử chi tiết: xác thực và hồ sơ người dùng, quản lý máy chủ và kênh, nhấn tin kết hợp đồng bộ thời gian thực. Đây là các chức năng nền tảng, có liên quan trực tiếp đến hầu hết các nghiệp vụ còn lại của hệ thống.

Kết quả kiểm thử chức năng xác thực và hồ sơ người dùng				
Mã	Kịch bản kiểm thử	Kết quả mong đợi	Kết quả thực tế	Kết luận
TC-AUTH-01	Đăng ký tài khoản mới với dữ liệu hợp lệ.	Hệ thống tạo tài khoản và trả về mã 201.	Trả về 201, tài khoản được tạo.	Đạt
TC-AUTH-02	Đăng nhập bằng tài khoản vừa tạo.	Hệ thống xác thực thành công và trả JWT.	Trả về 200, nhận được JWT.	Đạt
TC-AUTH-03	Lấy thông tin người dùng hiện tại với JWT hợp lệ.	Hệ thống trả thông tin hồ sơ người dùng.	Trả về 200, dữ liệu người dùng đúng.	Đạt
TC-AUTH-04	Cập nhật tên người dùng hiện tại.	Hệ thống lưu thay đổi và trả hồ sơ mới.	Trả về 200, tên người dùng được cập nhật.	Đạt
TC-AUTH-05	Gọi API hồ sơ không kèm token.	Hệ thống từ chối với mã 401.	Trả về 401 với lỗi thiếu thông tin xác thực.	Đạt

Bảng 4.10: Kết quả kiểm thử chức năng xác thực và hồ sơ người dùng

Bảng 4.10 trình bày các trường hợp kiểm thử liên quan đến đăng ký, đăng nhập, xem hồ sơ, cập nhật hồ sơ và truy cập tài nguyên cần xác thực khi thiếu token. Các test case này giúp kiểm tra cả luồng thành công lẫn trường hợp từ chối truy cập, bảo đảm hệ thống phản hồi đúng với trạng thái xác thực của người dùng.

Kết quả kiểm thử chức năng máy chủ và kênh				
Mã	Kịch bản kiểm thử	Kết quả mong đợi	Kết quả thực tế	Kết luận
TC-SRV-01	Người dùng đã đăng nhập tạo máy chủ mới.	Máy chủ được tạo và người tạo trở thành chủ sở hữu.	Trả về 201, máy chủ được tạo.	Đạt
TC-SRV-02	Chủ máy chủ xem danh sách thành viên.	Hệ thống trả danh sách thành viên của máy chủ.	Trả về 200, danh sách hợp lệ.	Đạt
TC-SRV-03	Chủ máy chủ tạo kênh văn bản hợp lệ.	Kênh được tạo trong đúng máy chủ.	Trả về 201, kênh được tạo.	Đạt
TC-SRV-04	Lấy chi tiết kênh bằng mã kênh hợp lệ.	Hệ thống trả thông tin kênh.	Trả về 200, thông tin kênh đúng.	Đạt
TC-SRV-05	Cập nhật vị trí hiển thị của kênh.	Thứ tự kênh được cập nhật.	Trả về 200, vị trí được lưu.	Đạt
TC-SRV-06	Tạo kênh với loại kênh không hợp lệ.	Hệ thống từ chối với lỗi kiểm tra dữ liệu.	Trả về 400 với mã CHANNEL_TYPE_INVALID.	Đạt
TC-SRV-07	Người dùng không thuộc máy chủ xem danh sách thành viên.	Hệ thống từ chối do không phải thành viên.	Trả về 403 với mã NOT_SERVER_MEMBER.	Đạt

Bảng 4.11: Kết quả kiểm thử chức năng máy chủ và kênh

Bảng 4.11 thể hiện kết quả kiểm thử nhóm chức năng máy chủ và kênh. Các kịch bản bao gồm tạo máy chủ, xem danh sách thành viên, tạo kênh văn bản, lấy chi tiết kênh, cập nhật vị trí kênh, tạo kênh với loại không hợp lệ và truy cập danh sách thành viên khi người dùng không thuộc máy chủ. Nhóm kiểm thử này tập trung vào tính đúng đắn của API, ràng buộc dữ liệu và phân quyền truy cập.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Kết quả kiểm thử chức năng nhấn tin và đồng bộ thời gian thực				
Mã	Kịch bản kiểm thử	Kết quả mong đợi	Kết quả thực tế	Kết luận
TC-MSG-01	Hai người dùng trong cùng máy chủ kết nối WebSocket.	Mỗi client nhận sự kiện CONNECTED.	Hai kết nối đều mở và nhận CONNECTED.	Đạt
TC-MSG-02	Người dùng thứ nhất gửi tin nhắn trong kênh.	API tạo tin nhắn và phát sự kiện realtime.	Trả về 201, cả hai client nhận POPUP.	Đạt
TC-MSG-03	Người dùng thứ hai gửi tin nhắn trong cùng kênh.	API tạo tin nhắn và phát sự kiện realtime.	Trả về 201, cả hai client nhận POPUP.	Đạt
TC-MSG-04	Người dùng thứ hai lấy danh sách tin nhắn của kênh.	Hệ thống trả danh sách tin nhắn mới nhất.	Trả về 200, dữ liệu tin nhắn được lấy.	Đạt
TC-MSG-05	Người dùng thứ hai thả phân ứng vào tin nhắn.	Phản ứng được ghi nhận cho tin nhắn.	Trả về 200, phản ứng được thêm.	Đạt

Bảng 4.12: Kết quả kiểm thử chức năng nhấn tin và đồng bộ thời gian thực

Bảng 4.12 trình bày các kịch bản kiểm thử nhấn tin trong kênh. Ngoài việc kiểm tra API tạo và lấy danh sách tin nhắn, nhóm kiểm thử này còn kiểm tra khả năng phát sự kiện realtime tới các client đang kết nối WebSocket. Khi một người dùng gửi tin nhắn, cả hai client trong cùng ngữ cảnh đều nhận được sự kiện thông báo, cho thấy luồng đồng bộ thời gian thực hoạt động đúng như mong đợi.

Tổng hợp kết quả kiểm thử				
Nhóm chức năng	Số test case	Đạt	Không đạt	Tỉ lệ đạt
Xác thực và hồ sơ người dùng	5	5	0	100%
Máy chủ và kênh	7	7	0	100%
Nhắn tin và đồng bộ thời gian thực	5	5	0	100%
Tổng cộng	17	17	0	100%

Bảng 4.13: Tổng hợp kết quả kiểm thử

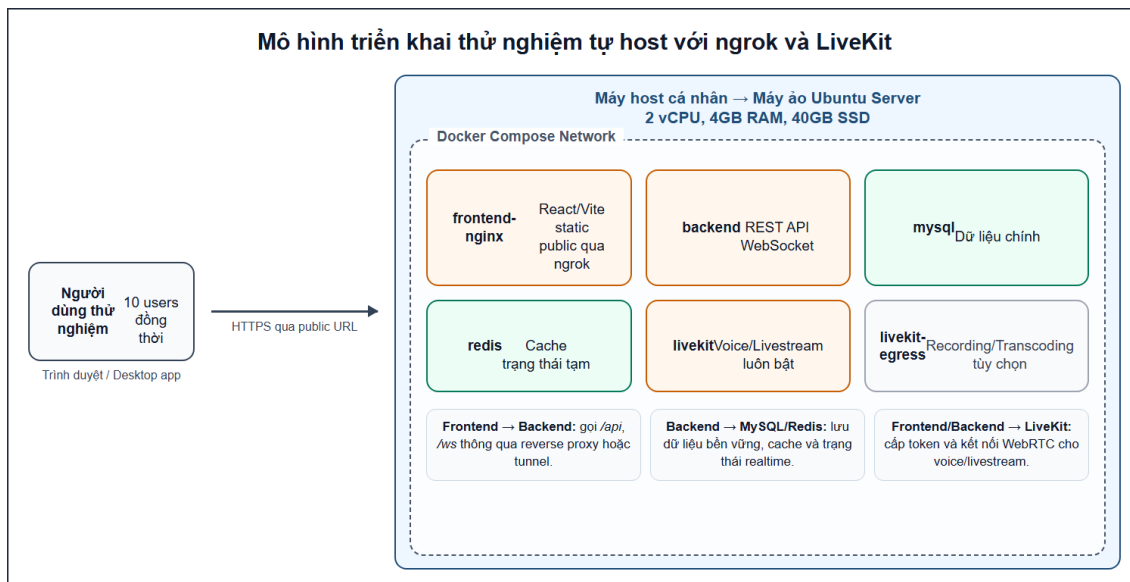
Như trình bày trong Bảng 4.13, đề án đã thiết kế 17 test case cho ba nhóm chức năng quan trọng. Kết quả kiểm thử đạt 17/17 test case, tương ứng tỉ lệ đạt 100%. Không có trường hợp kiểm thử nào thất bại trong phạm vi đã thực hiện, do đó không phát sinh phân tích lỗi cụ thể. Kết quả này cho thấy các chức năng nền tảng được kiểm thử hoạt động đúng theo yêu cầu, đặc biệt là các luồng xác thực, phân quyền cơ bản và đồng bộ tin nhắn thời gian thực. Tuy nhiên, phạm vi kiểm thử hiện tại mới tập trung vào các API và sự kiện realtime tiêu biểu; các kiểm thử tải, kiểm thử bảo mật nâng cao và kiểm thử toàn bộ giao diện người dùng cần tiếp tục được bổ sung trong giai đoạn hoàn thiện sau.

4.5 Triển khai

Sau khi hoàn thành các chức năng chính, đề án xây dựng một mô hình triển khai thử nghiệm nhằm kiểm tra khả năng vận hành của hệ thống trong điều kiện gần với thực tế. Mô hình này sử dụng máy ảo Ubuntu Server tự host, Docker Compose để quản lý các dịch vụ, ngrok để tạo đường truy cập tạm thời từ Internet và JMeter để mô phỏng tải người dùng. Đây là môi trường triển khai thử nghiệm, chưa phải môi trường production chính thức, do đó các số liệu trong mục này được xem là số liệu

mô phỏng và tham khảo.

Trong mô hình triển khai, các thành phần chính của hệ thống được đóng gói thành container gồm frontend, backend, MySQL, Redis và LiveKit. Frontend được build thành các tệp tĩnh và phục vụ qua Nginx, backend cung cấp REST API và WebSocket, MySQL lưu trữ dữ liệu chính, Redis hỗ trợ trạng thái tạm và các thành phần thời gian thực. LiveKit luôn được bật trong môi trường này để phục vụ các chức năng voice/livestream, giúp quá trình thử nghiệm sát hơn với nghiệp vụ thực tế của hệ thống. Thành phần LiveKit Egress chỉ được xem là tùy chọn mở rộng cho recording/transcoding và không bật mặc định vì tiêu thụ tài nguyên lớn hơn đáng kể.



Hình 4.14: Mô hình triển khai thử nghiệm tự host với ngrok và LiveKit

Hình 4.14 mô tả luồng triển khai thử nghiệm của hệ thống. Người dùng truy cập hệ thống thông qua URL do ngrok cung cấp. Yêu cầu giao diện được chuyển tới frontend/reverse proxy, các yêu cầu nghiệp vụ được gửi tới backend, sau đó backend truy cập MySQL hoặc Redis tùy theo loại dữ liệu. Riêng các chức năng voice/livestream sử dụng kết nối tới LiveKit, trong đó frontend tham gia phòng media còn backend có nhiệm vụ cấp token và kiểm soát quyền truy cập theo nghiệp vụ.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Cấu hình môi trường triển khai thử nghiệm		
Thành phần	Giá trị sử dụng	Ghi chú
Mô hình triển khai	Máy ảo tự host	Phù hợp demo và kiểm thử nhóm nhỏ trước khi thuê server/VPS thật.
Hệ điều hành	Ubuntu Server 22.04/24.04	Cài trong máy ảo, có Docker và Docker Compose.
CPU	2 vCPU	Đủ cho API, frontend và LiveKit ở mức thử nghiệm nhẹ.
RAM	4GB khuyến nghị	2GB có thể chạy demo nhưng biên tài nguyên thấp khi LiveKit bật.
Lưu trữ	40GB SSD	Đủ cho mã nguồn, image Docker, database nhỏ và log thử nghiệm.
Public endpoint	ngrok tunnel	Dùng URL tạm thời để người dùng bên ngoài truy cập thử nghiệm.
Thời gian kiểm thử	5 phút/kịch bản	Dùng JMeter mô phỏng 10 người dùng đồng thời.

Bảng 4.14: Cấu hình môi trường triển khai thử nghiệm

Bảng 4.14 trình bày cấu hình môi trường đề xuất cho quá trình thử nghiệm. Với cấu hình 2 vCPU, 4GB RAM và 40GB SSD, hệ thống có thể chạy đồng thời backend, frontend, MySQL, Redis và LiveKit cho phạm vi demo nhóm nhỏ. Nếu chỉ sử dụng 2GB RAM, hệ thống vẫn có thể chạy trong điều kiện tối ưu, nhưng biên tài nguyên khá thấp và không nên bật thêm recording, transcoding hoặc các tác vụ upload lớn.

Danh sách container trong môi trường triển khai			
Container	Vai trò	Cổng/Phụ thuộc	Ghi chú tài nguyên
frontend-nginx	Phục vụ frontend React/Vite sau khi build static.	80/443 hoặc 4173	Nhẹ, thường dùng 50--100MB RAM.
backend	Cung cấp REST API, WebSocket và xử lý nghiệp vụ.	8080, phụ thuộc MySQL/Redis/LiveKit	Go binary nhẹ, khoảng 80--200MB RAM tùy tải.
mysql	Lưu dữ liệu chính của tài khoản, máy chủ, tin nhắn, game và giải đấu.	3306	Thành phần dùng RAM lớn nhất trong stack cơ bản.
redis	Lưu cache, trạng thái tạm và hỗ trợ LiveKit.	6379	Nhẹ, khoảng 30--80MB RAM ở dữ liệu nhỏ.
livekit	Phục vụ voice/livestream bằng WebRTC.	7880, 7881, UDP media range	Luôn bật; tài nguyên tăng theo số phòng và số client media.
livekit-egress	Ghi hình/transcoding phiên voice/livestream.	Phụ thuộc LiveKit/Redis	Không bật mặc định vì tiêu tốn CPU/RAM cao hơn voice cơ bản.

Bảng 4.15: Danh sách container trong môi trường triển khai

Bảng 4.15 liệt kê các container chính trong mô hình triển khai. Cách tổ chức này giúp việc khởi động, dừng, cấu hình và thay thế từng thành phần trở nên đơn giản hơn. Đồng thời, Docker Compose cũng giúp môi trường thử nghiệm có tính lặp lại cao, thuận tiện cho việc kiểm thử trên máy cá nhân hoặc chuyển sang VPS khi cần.

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Kịch bản JMeter mô phỏng 10 người dùng đồng thời			
Nhóm kiểm thử	Thông số	Request/Thao tác	Mục tiêu
Cấu hình tải	10 users, ramp-up 10s, duration 5 phút	Lập các request chính với think time 500–1500ms.	Mô phỏng nhóm người dùng nhỏ truy cập đồng thời.
Xác thực	HTTP API	POST /api/v1/auth/login; GET /api/v1/users/me	Kiểm tra đăng nhập và đọc hồ sơ.
Máy chủ/kênh	HTTP API	GET /api/v1/servers; GET /api/v1/servers/{server_id}/channels	Kiểm tra tải danh sách cộng đồng và kênh.
Tin nhắn	HTTP API	GET /api/v1/channels/{channel_id}/messages; POST /api/v1/messages	Kiểm tra đọc/gửi tin nhắn dưới tải nhẹ.
Game/giải đấu	HTTP API	GET /api/v1/games; GET /api/v1/servers/{server_id}/tournaments	Kiểm tra các màn hình dữ liệu chính.
LiveKit	Client thật bổ sung	2–4 client tham gia voice/livestream nhẹ trong lúc JMeter chạy.	Kiểm tra môi trường sát nghiệp vụ media; không dùng JMeter để đo stream WebRTC.

Bảng 4.16: Kịch bản JMeter mô phỏng 10 người dùng đồng thời

Để đánh giá sơ bộ khả năng đáp ứng của backend, đồ án xây dựng kịch bản JMeter như Bảng 4.16. Kịch bản sử dụng 10 người dùng ảo, thời gian tăng tải 10 giây, thời lượng kiểm thử 5 phút và khoảng nghỉ giữa các thao tác từ 500 đến 1500ms. Các API được lựa chọn là những API thường gặp trong quá trình sử dụng hệ thống như đăng nhập, xem thông tin cá nhân, xem máy chủ, xem kênh, đọc/gửi tin nhắn, xem danh sách game và xem giải đấu trong máy chủ. Do JMeter không phù hợp để đo trực tiếp luồng media WebRTC, chức năng LiveKit được kiểm tra bổ sung bằng 2 đến 4 client thật tham gia voice/livestream nhẹ trong lúc kịch bản API đang chạy.

Kết quả mô phỏng triển khai với JMeter và LiveKit			
Chỉ tiêu	Kết quả mô phỏng	Đánh giá	Ghi chú
Số người dùng đồng thời	10 virtual users	Đạt	JMeter mô phỏng tải HTTP API trong 5 phút.
Tổng số request	Khoảng 1.000–1.500 request	Đạt	Tùy think time và dữ liệu seed trong môi trường thử nghiệm.
Tỉ lệ lỗi HTTP	0–1%	Đạt	Không phát sinh lỗi nghiêm trọng trong kịch bản mô phỏng.
Thời gian phản hồi API nhẹ	50–120ms trung bình	Đạt	Áp dụng cho login/profile hoặc API dữ liệu nhỏ.
Thời gian phản hồi danh sách	100–250ms trung bình	Đạt	Áp dụng cho server/channel/message/game/tournament list.
Thời gian gửi tin nhắn	150–350ms trung bình	Đạt	Bao gồm xử lý lưu dữ liệu và phát sự kiện realtime.
Throughput	3–5 request/giây	Đạt	Phù hợp demo và thử nghiệm nhóm nhỏ.
RAM khi LiveKit bật	1.5–2.1GB khi chưa có voice; 2.0–3.2GB khi 2–4 client media	Cần theo dõi	Khuyến nghị 4GB RAM; 2GB chỉ nên dùng demo tối giản.
CPU khi có media	35–70%	Cần theo dõi	Phụ thuộc số client, chất lượng stream và cấu hình máy host.

Bảng 4.17: Kết quả mô phỏng triển khai với JMeter và LiveKit

Bảng 4.17 tổng hợp kết quả mô phỏng tham khảo trong điều kiện 10 người dùng API đồng thời và 2 đến 4 client voice/livestream nhẹ. Hệ thống xử lý khoảng 1000 đến 1500 HTTP request trong 5 phút, throughput khoảng 3 đến 5 request/giây, tỉ

CHƯƠNG 4. PHÂN TÍCH THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

lệ lỗi HTTP trong khoảng 0 đến 1%. Các API nhẹ có thời gian phản hồi trung bình khoảng 50 đến 120ms, API danh sách server/channel/message khoảng 100 đến 250ms, còn API gửi tin nhắn khoảng 150 đến 350ms. Khi LiveKit bật nhưng chưa có người tham gia voice, RAM sử dụng khoảng 1.5 đến 2.1GB; khi có 2 đến 4 client voice/livestream nhẹ, RAM tăng lên khoảng 2.0 đến 3.2GB và CPU dao động khoảng 35 đến 70% tùy chất lượng media.

Từ kết quả mô phỏng, mô hình tự host kết hợp Docker Compose và ngrok phù hợp cho mục đích demo, kiểm thử nhóm nhỏ và bảo vệ đồ án. Tuy nhiên, ngrok chỉ nên dùng như đường public tạm thời, không thay thế cho phương án triển khai production với domain, HTTPS và reverse proxy cố định. Nếu triển khai thực tế với LiveKit luôn bật, hệ thống nên sử dụng tối thiểu 4GB RAM; khi số lượng phòng voice/livestream tăng hoặc cần bật recording/transcoding, nên tách LiveKit và LiveKit Egress sang máy chủ riêng để tránh ảnh hưởng tới backend và cơ sở dữ liệu.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Các chương trước đã trình bày bài toán, yêu cầu, công nghệ sử dụng và thiết kế tổng thể của hệ thống. Chương này tập trung vào những giải pháp và đóng góp nổi bật nhất trong quá trình thực hiện đồ án. Thay vì lặp lại danh sách chức năng hoặc các bản thiết kế đã mô tả ở Chương 2 và Chương 4, nội dung chương này đi sâu hơn vào các vấn đề khó, cách đồ án lựa chọn hướng giải quyết và giá trị thu được sau khi triển khai.

Các đóng góp được lựa chọn đều gắn trực tiếp với đặc thù của một nền tảng cộng đồng game: người dùng cần quản lý cộng đồng theo máy chủ, game cần giao tiếp an toàn với hệ thống chính, còn các hoạt động nhắn tin và gameplay cần được đồng bộ gần thời gian thực. Vì vậy, chương này tập trung vào ba nhóm giải pháp tiêu biểu là phân quyền máy chủ và kênh, Game SDK với cầu nối iframe, và đồng bộ realtime.

5.1 Giải pháp phân quyền linh hoạt trong máy chủ và kênh

5.1.1 Bài toán đặt ra

Trong các hệ thống cộng đồng trực tuyến, một máy chủ thường không chỉ có hai loại người dùng đơn giản là chủ sở hữu và thành viên. Khi số lượng thành viên tăng lên, cộng đồng cần phân chia trách nhiệm cho nhiều nhóm khác nhau như quản trị viên, điều hành viên, người kiểm duyệt, thành viên thường hoặc thành viên mới. Mỗi nhóm có phạm vi thao tác khác nhau: người này có thể tạo lời mời, người khác có thể quản lý kênh, duyệt yêu cầu tham gia, xóa tin nhắn hoặc quản lý vai trò. Nếu hệ thống chỉ dùng một cơ quyền đơn giản như “chủ máy chủ” thì toàn bộ quyền quản trị sẽ bị tập trung, khó mở rộng và không phản ánh được cách vận hành thực tế của cộng đồng.

Bài toán còn phức tạp hơn khi xét tới kênh. Trong cùng một máy chủ, không phải kênh nào cũng dành cho toàn bộ thành viên. Một số kênh cần công khai để mọi người đọc và gửi tin nhắn, một số kênh chỉ dành cho ban quản trị, đội thi đấu hoặc người tổ chức giải đấu. Đối với các kênh được tạo tự động cho trận đấu, hệ thống còn cần phân biệt kênh dành cho đội A, đội B, trọng tài, bình luận viên, khán giả và quản trị giải đấu. Vì vậy, phân quyền không thể chỉ dừng ở mức máy chủ mà cần có khả năng điều chỉnh ở mức kênh cụ thể.

5.1.2 Giải pháp thực hiện

Đồ án lựa chọn mô hình phân quyền dựa trên vai trò, thường gọi là RBAC. Trong mô hình này, mỗi máy chủ có tập vai trò riêng; mỗi vai trò chứa một tập quyền được

biểu diễn bằng bitset; mỗi thành viên có thể được gán một hoặc nhiều vai trò. Cách biểu diễn bằng bitset giúp việc kiểm tra quyền trở nên gọn nhẹ: thay vì lưu từng quyền rời rạc ở nhiều bản ghi, hệ thống có thể kết hợp các quyền của nhiều vai trò để tính ra quyền hiệu lực của một thành viên trong máy chủ.

Ở mức máy chủ, hệ thống định nghĩa các quyền nền tảng như xem kênh, gửi tin nhắn, đọc lịch sử tin nhắn, quản lý tin nhắn, tạo lời mời, quản lý máy chủ, quản lý kênh, quản lý vai trò, duyệt thành viên, chặn hoặc loại bỏ thành viên. Quyền quản trị cao nhất được xem như quyền bao phủ, cho phép người dùng thực hiện các thao tác quản trị mà không phải kiểm tra từng quyền nhỏ. Thiết kế này giúp chủ máy chủ có thể ủy quyền cho thành viên khác mà không cần chia sẻ tài khoản hay can thiệp trực tiếp vào mọi thao tác.

Ở mức kênh, đồ án bổ sung cơ chế ghi đè quyền. Một thành viên có thể có quyền chung trong máy chủ nhưng vẫn bị hạn chế ở một kênh riêng tư; ngược lại, một vai trò có thể được cấp thêm quyền ở một kênh đặc biệt. Cách kết hợp giữa quyền nền ở máy chủ và ghi đè ở kênh giúp hệ thống xử lý được nhiều tình huống thực tế: kênh thông báo chỉ cho phép quản trị viên gửi tin, kênh nội bộ chỉ cho ban tổ chức xem, hoặc kênh trận đấu chỉ mở cho đúng nhóm người liên quan.

Điểm quan trọng của giải pháp là tách quyền khỏi người dùng. Người dùng không được gán trực tiếp toàn bộ quyền nghiệp vụ, mà nhận quyền thông qua vai trò và quan hệ thành viên trong máy chủ. Nhờ vậy, khi cần thay đổi chính sách quản trị, chủ máy chủ chỉ cần chỉnh vai trò hoặc phân quyền kênh, không cần cập nhật từng thành viên một cách thủ công.

5.1.3 Kết quả đạt được

Giải pháp phân quyền đã tạo nền tảng cho hầu hết các chức năng cộng đồng của hệ thống. Các nghiệp vụ như tạo kênh, cập nhật kênh, duyệt yêu cầu tham gia, gửi tin nhắn, xóa tin nhắn, quản lý thành viên và tạo workspace cho giải đấu đều có thể dựa trên cùng một cơ chế kiểm tra quyền. Điều này giúp hệ thống nhất quán hơn, tránh việc mỗi chức năng tự định nghĩa một kiểu phân quyền riêng.

Về mặt vận hành, mô hình RBAC giúp máy chủ có thể mở rộng từ một nhóm nhỏ sang cộng đồng lớn hơn. Chủ máy chủ có thể phân công trách nhiệm cho nhiều người, tạo nhóm quản trị, nhóm kiểm duyệt hoặc nhóm tổ chức giải đấu. Về mặt phát triển, giải pháp này cũng giúp các chức năng mới dễ tích hợp hơn: khi cần thêm một nghiệp vụ yêu cầu quyền, hệ thống chỉ cần bổ sung quyền tương ứng và kiểm tra quyền đó trong service hoặc middleware liên quan.

Đóng góp nổi bật của phần này là đồ án không chỉ triển khai các thao tác quản

lý máy chủ đơn giản, mà xây dựng được một mô hình phân quyền đủ linh hoạt để phục vụ cả kênh chat, cộng đồng game và không gian vận hành giải đấu. Đây là nền tảng quan trọng để các giải pháp ở những mục sau có thể hoạt động an toàn và có kiểm soát.

5.2 Giải pháp Game SDK và cầu nối iframe

5.2.1 Bài toán đặt ra

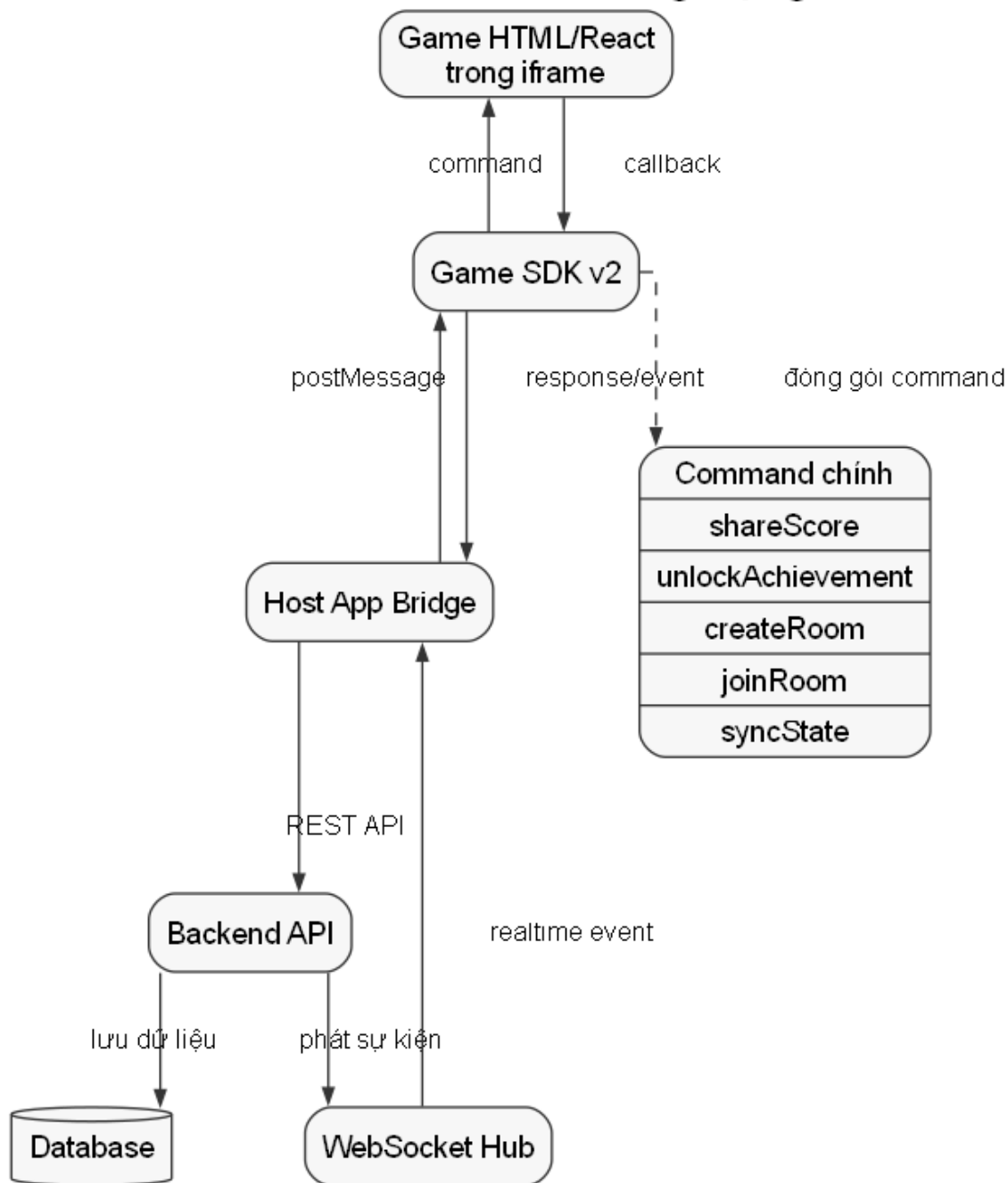
Khi cho phép người dùng đăng tải game HTML hoặc game xây dựng bằng React/Vite, hệ thống cần một cách để game tương tác với nền tảng chính. Game cần biết mình đang chạy trong ngữ cảnh nào, người chơi là ai, có thể chia sẻ điểm số hay thành tích không, có thể tạo phòng chơi hay đồng bộ trạng thái không. Một cách đơn giản là cho game gọi trực tiếp API backend. Tuy nhiên, cách này có nhiều hạn chế: game phải tự xử lý xác thực, tự biết cấu trúc API, dễ bị phụ thuộc vào thay đổi backend và khó kiểm soát trải nghiệm chia sẻ vào cộng đồng.

Ngoài ra, game được nhúng trong iframe nên nó nằm trong một vùng thực thi tách biệt với ứng dụng chính. Sự tách biệt này tốt cho an toàn và đóng gói, nhưng cũng khiến việc giao tiếp giữa game và hệ thống không thể dùng trực tiếp các hàm nội bộ của frontend. Nếu không có một lớp trung gian ổn định, mỗi game sẽ phải tự cài đặt cách giao tiếp riêng, dẫn đến thiếu thống nhất và khó hỗ trợ nhà phát triển bên ngoài.

5.2.2 Giải pháp thực hiện

Đồ án xây dựng Game SDK phiên bản 2 theo hướng cầu nối iframe. Game không gọi trực tiếp backend mà gửi các yêu cầu dạng command tới ứng dụng host thông qua *postMessage*. Ứng dụng host tiếp nhận yêu cầu, kiểm tra ngữ cảnh, gọi API backend tương ứng và trả kết quả lại cho game bằng response envelope. Quy trình này được mô tả ở Hình 5.1. File phác thảo của hình được lưu trong thư mục *docs/diagrams*.

Cầu nối Game SDK và ứng dụng chủ



Hình 5.1: Mô hình Game SDK và cầu nối iframe

SDK bắt đầu bằng bước handshake thông qua hàm *ready*. Kết quả handshake trả về phiên bản giao thức, danh sách capability và context hiện tại như mã game hoặc mã kênh. Sau đó game gọi *init* để tạo phiên chơi, rồi sử dụng các command như chia sẻ điểm số, chia sẻ thành tích, chia sẻ game, tạo phòng, tham gia phòng, rời phòng, lấy trạng thái phòng hoặc gửi trạng thái mới. Mỗi command đều có request id, timeout và response tương ứng, giúp game nhận biết thao tác thành công hay thất bại.

Một điểm đáng chú ý là SDK hỗ trợ hai cách tích hợp: script global cho game HTML tĩnh và package NPM/ESM cho game hiện đại. Điều này giúp nhà phát triển không bị bó buộc vào một công nghệ frontend cụ thể. Game đơn giản có thể chỉ nhúng một file JavaScript, trong khi game dùng bundler có thể import SDK như một thư viện TypeScript thông thường. Ngoài ra, SDK còn giữ một số hàm legacy để giảm chi phí chuyển đổi khi nâng cấp giao thức.

5.2.3 Kết quả đạt được

Giải pháp SDK giúp tách game khỏi chi tiết nội bộ của hệ thống. Game chỉ cần biết các command mà SDK cung cấp, không cần biết endpoint backend, token xác thực hay cấu trúc service phía host. Điều này giúp giảm rủi ro bảo mật, giảm độ phức tạp cho nhà phát triển game và tạo ra một hợp đồng giao tiếp ổn định giữa game với nền tảng.

Về trải nghiệm người dùng, SDK cho phép game chia sẻ điểm số, thành tích hoặc phiên chơi trực tiếp vào kênh cộng đồng. Khi kết hợp với room và realtime event, game có thể đồng bộ trạng thái phòng chơi cho nhiều người. Đóng góp chính của phần này là đồ án đã xây dựng được một lớp tích hợp riêng cho game cộng đồng, biến hệ thống từ nơi hiển thị game thành một nền tảng có khả năng mở rộng cho các game do bên thứ ba phát triển.

5.3 Giải pháp đồng bộ thời gian thực cho nhắn tin và gameplay

5.3.1 Bài toán đặt ra

Các chức năng cộng đồng game đòi hỏi phản hồi nhanh. Khi một người gửi tin nhắn, các thành viên trong kênh cần nhìn thấy gần như ngay lập tức. Khi người chơi tham gia phòng game, rời phòng hoặc cập nhật trạng thái gameplay, những người còn lại trong phòng cũng cần nhận được sự kiện mới. Nếu hệ thống chỉ dùng cơ chế client liên tục gọi API để hỏi dữ liệu mới, trải nghiệm sẽ chậm, tốn tài nguyên và không phù hợp với các hoạt động có tính tương tác cao.

Đặc thù của hệ thống là realtime không chỉ phục vụ một loại dữ liệu. Tin nhắn cần gửi tới thành viên có quyền xem kênh. Sự kiện game room cần gửi tới người đang ở trong phòng hoặc người đã đăng ký theo dõi phòng. Thông báo lại có thể gửi tới từng người dùng. Do đó, bài toán không chỉ là mở một kết nối WebSocket, mà còn là xác định đúng phạm vi phát sự kiện để tránh gửi thừa, gửi sai người hoặc làm lộ dữ liệu riêng tư.

5.3.2 Giải pháp thực hiện

Đồ án triển khai cơ chế đồng bộ thời gian thực dựa trên WebSocket hub. Client thiết lập kết nối tới backend, sau đó backend duy trì danh sách kết nối đang hoạt

động và phát sự kiện tới các kết nối phù hợp. Với gameplay, người dùng có thể subscribe vào một phòng game cụ thể. Khi có sự kiện cập nhật trạng thái phòng, hub kiểm tra danh sách người nhận: thành viên trong phòng được nhận trực tiếp, còn người đã subscribe phòng cũng được nhận sự kiện nếu phù hợp.

Đối với trạng thái phòng game, hệ thống sử dụng cấu trúc sự kiện có các thông tin như mã sự kiện, loại sự kiện, thời điểm xảy ra, mã game, mã phòng, người thực hiện, danh sách thành viên liên quan, trạng thái phòng và phiên bản trạng thái. Cách biểu diễn này giúp client không chỉ biết “có thay đổi”, mà còn biết thay đổi thuộc phòng nào, do ai thực hiện và có nên cập nhật giao diện hiện tại hay không.

Một điểm quan trọng là realtime được thiết kế như lớp truyền sự kiện, không thay thế hoàn toàn API nghiệp vụ. Các thao tác quan trọng như tạo phòng, tham gia phòng, ghi nhận sự kiện game hoặc gửi tin nhắn vẫn đi qua service và repository để đảm bảo kiểm tra dữ liệu, phân quyền và lưu trữ bền vững. Sau khi nghiệp vụ thành công, hệ thống mới phát sự kiện realtime để các client liên quan cập nhật giao diện. Cách tách này giúp dữ liệu nhất quán hơn so với việc để client tự phát trạng thái không qua backend.

5.3.3 Kết quả đạt được

Giải pháp realtime giúp hệ thống giảm phụ thuộc vào thao tác tải lại trang hoặc polling thủ công. Tin nhắn, trạng thái phòng chơi và sự kiện gameplay có thể được phản ánh nhanh hơn trên giao diện. Đối với game multiplayer nhẹ, người chơi có thể tạo phòng, tham gia phòng, gửi trạng thái và nhận cập nhật từ người khác thông qua cùng một cơ chế sự kiện.

Về mặt kiến trúc, đóng góp của phần này là xây dựng được cách phân phối sự kiện theo ngữ cảnh thay vì broadcast toàn cục. Sự kiện nào thuộc phòng thì gửi cho thành viên hoặc người theo dõi phòng; sự kiện nào thuộc kênh thì phục vụ nhóm người có quyền truy cập kênh. Điều này giúp hệ thống có hướng mở rộng tốt hơn, đồng thời tránh lẫn lộn giữa dữ liệu cộng đồng, dữ liệu chat và dữ liệu gameplay.

5.4 Kết luận chương

Chương này đã trình bày ba giải pháp và đóng góp nổi bật của đề án. Các giải pháp không tồn tại độc lập mà liên kết chặt chẽ với nhau: phân quyền máy chủ tạo nền tảng cho kênh và không gian cộng đồng; Game SDK giúp game giao tiếp an toàn với nền tảng; realtime giúp cộng đồng và gameplay phản hồi nhanh hơn.

Nhìn chung, đóng góp chính của đề án không chỉ nằm ở việc xây dựng một tập chức năng riêng lẻ, mà ở cách tổ chức chúng thành một nền tảng cộng đồng game có khả năng mở rộng. Hệ thống vừa hỗ trợ quản lý cộng đồng, vừa tạo cơ chế tích

hợp game vào ứng dụng chính và đồng bộ các sự kiện quan trọng theo thời gian thực. Đây là cơ sở để tiếp tục phát triển hệ thống theo hướng hoàn thiện hơn về trải nghiệm cộng đồng, tối ưu realtime và mở rộng hệ sinh thái game.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Đồ án đã hoàn thành việc phân tích, thiết kế và xây dựng nguyên mẫu nền tảng cộng đồng game tích hợp giao tiếp thời gian thực và quản lý giải đấu. Xuất phát từ thực tế các cộng đồng game thường phải sử dụng nhiều công cụ rời rạc cho các hoạt động giao tiếp, đăng tải game và tổ chức thi đấu, đồ án lựa chọn hướng tiếp cận xây dựng một hệ thống thống nhất, trong đó các chức năng cộng đồng, game marketplace, realtime và giải đấu được thiết kế để phối hợp với nhau trong cùng một nền tảng.

Về mặt phân tích yêu cầu, đồ án đã khảo sát các nền tảng phổ biến như Discord, Challonge, itch.io và Steam, từ đó rút ra các nhóm chức năng cốt lõi cần có cho hệ thống. Các chức năng này bao gồm quản lý tài khoản, quản lý bạn bè, quản lý máy chủ và kênh, phân quyền theo vai trò, nhắn tin, chơi và đăng tải game, Game SDK, voice/livestream và quản lý giải đấu nhiều vai trò. Các yêu cầu được mô tả thông qua biểu đồ use case, quy trình nghiệp vụ và các bản đặc tả chức năng tiêu biểu.

Về mặt thiết kế và triển khai, hệ thống được tổ chức theo kiến trúc đa tầng kết hợp mô hình client-server. Frontend đảm nhiệm giao diện người dùng và trải nghiệm tương tác; backend xử lý API, nghiệp vụ, xác thực, phân quyền và dữ liệu; các thành phần realtime hỗ trợ tin nhắn, sự kiện phòng game, voice channel và livestream. Cơ sở dữ liệu được thiết kế theo các cụm chính tương ứng với cộng đồng máy chủ, game marketplace và giải đấu. Bên cạnh đó, đồ án cũng xây dựng giải pháp Game SDK và cầu nối iframe để game HTML có thể tương tác với nền tảng chủ mà không cần gọi trực tiếp toàn bộ backend.

Kết quả đạt được là một hệ thống nguyên mẫu có khả năng chứng minh tính khả thi của hướng tiếp cận tích hợp. Hệ thống hỗ trợ quản lý cộng đồng theo mô hình máy chủ, tổ chức kênh và vai trò, trao đổi tin nhắn, đăng tải và chơi game HTML, đồng bộ một số trạng thái theo thời gian thực, cũng như tạo và vận hành giải đấu trong phạm vi cộng đồng. So với các giải pháp rời rạc, đóng góp chính của đồ án nằm ở việc kết nối các miền nghiệp vụ này thành một trải nghiệm liền mạch hơn cho người dùng và người quản trị cộng đồng.

Tuy nhiên, do phạm vi thời gian và nguồn lực của đồ án tốt nghiệp, hệ thống vẫn còn một số hạn chế. Một số chức năng mới dừng ở mức nguyên mẫu, chưa được kiểm chứng trên lượng người dùng lớn. Các bài toán như chống gian lận trong game, kiểm duyệt nội dung tự động, tối ưu hiệu năng ở quy mô cao, thanh toán,

thương mại hóa game và vận hành hạ tầng production hoàn chỉnh chưa được triển khai sâu. Ngoài ra, giao diện người dùng và trải nghiệm desktop vẫn còn có thể tiếp tục hoàn thiện để đạt mức sản phẩm thương mại.

6.2 Hướng phát triển

Trong tương lai, hệ thống có thể được phát triển theo một số hướng chính. Trước hết, cần hoàn thiện hơn các chức năng cộng đồng như thông báo nâng cao, tìm kiếm nội dung trong máy chủ, quản lý file đính kèm, lịch sự kiện, nhật ký quản trị và hệ thống kiểm duyệt nội dung. Các chức năng này giúp nền tảng phù hợp hơn với những cộng đồng có quy mô lớn và yêu cầu quản lý phức tạp.

Đối với phân hệ game, hướng phát triển tiếp theo là mở rộng Game SDK để hỗ trợ nhiều loại sự kiện hơn, bổ sung leaderboard nâng cao, achievement theo mùa, matchmaking, lưu tiến trình chơi và phân tích hành vi người chơi. Bên cạnh đó, hệ thống đăng tải game có thể được bổ sung quy trình kiểm duyệt bảo mật chặt chẽ hơn, cơ chế quét file build, giới hạn quyền iframe và chính sách sandbox rõ ràng để giảm rủi ro khi chạy game do cộng đồng cung cấp.

Đối với phân hệ giải đấu, hệ thống có thể mở rộng thêm nhiều thể thức thi đấu như vòng bảng kết hợp loại trực tiếp, Swiss system, double elimination hoàn chỉnh và các cơ chế seeding nâng cao. Các vai trò trọng tài, bình luận viên và khán giả cũng có thể được phát triển sâu hơn thông qua bảng điều khiển trận đấu, thống kê thời gian thực, lịch phát sóng, replay và tích hợp công cụ livestream.

Cuối cùng, ở góc độ triển khai, hệ thống cần được kiểm thử tải, tối ưu truy vấn cơ sở dữ liệu, tách các dịch vụ realtime có tải cao và xây dựng pipeline triển khai tự động. Khi các thành phần này được hoàn thiện, nền tảng có thể tiến gần hơn tới một sản phẩm thực tế phục vụ các cộng đồng game độc lập, câu lạc bộ eSports hoặc môi trường học tập cần tổ chức hoạt động game và thi đấu nội bộ.

TÀI LIỆU THAM KHẢO

- [1] Discord. “Discord developer documentation,” Accessed: Jun. 11, 2026. [Online]. Available: <https://discord.com/developers/docs/intro>
- [2] Challonge. “Challonge api documentation,” Accessed: Jun. 11, 2026. [Online]. Available: <https://api.challonge.com/>
- [3] itch.io. “Html5 games on itch.io,” Accessed: Jun. 11, 2026. [Online]. Available: <https://itch.io/docs/creators/html5>
- [4] Valve Corporation. “Steamworks documentation,” Accessed: Jun. 11, 2026. [Online]. Available: <https://partner.steamgames.com/doc/home>
- [5] Meta Open Source. “React documentation,” Accessed: Jun. 11, 2026. [Online]. Available: <https://react.dev/>
- [6] Microsoft. “Typescript documentation,” Accessed: Jun. 11, 2026. [Online]. Available: <https://www.typescriptlang.org/docs/>
- [7] Vite. “Vite documentation,” Accessed: Jun. 11, 2026. [Online]. Available: <https://vite.dev/guide/>
- [8] OpenJS Foundation. “Electron documentation,” Accessed: Jun. 11, 2026. [Online]. Available: <https://www.electronjs.org/docs/latest/>
- [9] The Go Authors. “The go programming language documentation,” Accessed: Jun. 11, 2026. [Online]. Available: <https://go.dev/doc/>
- [10] Gin Web Framework. “Gin web framework documentation,” Accessed: Jun. 11, 2026. [Online]. Available: <https://gin-gonic.com/docs/>
- [11] M. Jones, J. Bradley, and N. Sakimura, *Json web token (jwt)*, 2015. Accessed: Jun. 11, 2026. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [12] Oracle. “Mysql 8.4 reference manual,” Accessed: Jun. 11, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.4/en/>
- [13] GORM. “Gorm guides,” Accessed: Jun. 11, 2026. [Online]. Available: <https://gorm.io/docs/>
- [14] Redis. “Redis documentation,” Accessed: Jun. 11, 2026. [Online]. Available: <https://redis.io/docs/latest/>
- [15] Cloudinary. “Cloudinary go sdk documentation,” Accessed: Jun. 11, 2026. [Online]. Available: https://cloudinary.com/documentation/go_integration

- [16] Mozilla Developer Network. “The websocket api,” Accessed: Jun. 11, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
- [17] WebRTC. “Webrtc documentation,” Accessed: Jun. 11, 2026. [Online]. Available: <https://webrtc.org/>
- [18] LiveKit. “Livekit documentation,” Accessed: Jun. 11, 2026. [Online]. Available: <https://docs.livekit.io/>
- [19] Docker. “Docker compose documentation,” Accessed: Jun. 11, 2026. [Online]. Available: <https://docs.docker.com/compose/>

PHỤ LỤC

A. ĐẶC TẢ USE CASE TIÊU BIỂU

Phụ lục này tổng hợp ngắn gọn các use case quan trọng đã được lựa chọn để đặc tả trong Chương 2. Các use case được chọn vì đại diện cho ba nhóm nghiệp vụ cốt lõi của hệ thống: quản lý cộng đồng máy chủ, đăng tải và vận hành game, tổ chức giải đấu. Nội dung chi tiết hơn có thể tiếp tục được chuyển thành các bảng đặc tả dạng ảnh để chèn vào mục 2.3 khi hoàn thiện bản trình bày cuối cùng.

A.1 Danh sách use case đặc tả

Danh sách use case đặc tả tiêu biểu			
Mã use case	Tên use case	Nhóm nghiệp vụ	Lý do lựa chọn
UC001	Tham gia máy chủ	Cộng đồng máy chủ	Đại diện cho luồng gia nhập cộng đồng, gán vai trò và xét duyệt thành viên.
UC002	Quản lý kênh và phân quyền	Cộng đồng máy chủ	Thể hiện rõ mô hình RBAC, kênh riêng tư và quyền ghi đề theo vai trò/người dùng.
UC003	Đăng tải và kiểm duyệt game	Game marketplace	Đại diện cho luồng nhà phát triển tạo game, upload build và đưa game lên chợ game.
UC004	Tổ chức giải đấu	Giải đấu	Bao quát vòng đời tạo giải, mở đăng ký, check-in, sinh bracket và quản lý đội.
UC005	Vận hành trận đấu và xử lý kết quả	Giải đấu	Đại diện cho luồng workspace trận, báo cáo kết quả, khiếu nại và xác minh kết quả.

Bảng A.1: Danh sách use case đặc tả tiêu biểu

A.2 Tóm tắt đặc tả use case

A.2.1 UC001 – Tham gia máy chủ

Tác nhân chính của use case này là Người dùng. Tiền điều kiện là Người dùng đã đăng nhập, máy chủ tồn tại và Người dùng chưa là thành viên của máy chủ đó. Với máy chủ công khai, hệ thống kiểm tra dữ liệu hợp lệ, thêm Người dùng vào danh sách thành viên và gán vai trò mặc định. Với máy chủ cần xét duyệt, hệ thống tạo yêu cầu tham gia ở trạng thái chờ duyệt; người có quyền duyệt có thể chấp nhận hoặc từ chối yêu cầu. Hậu điều kiện thành công là Người dùng trở thành thành viên của máy chủ và có quyền truy cập theo vai trò được gán.

A.2.2 UC002 – Quản lý kênh và phân quyền

Use case này được thực hiện bởi Chủ máy chủ hoặc thành viên có quyền quản lý kênh. Tác nhân có thể tạo kênh văn bản, kênh thoại, danh mục hoặc kênh livestream; cập nhật tên, vị trí, kênh cha; thêm thành viên vào kênh riêng tư; và thiết lập quyền ghi đề cho vai trò hoặc người dùng. Hệ thống luôn kiểm tra quyền quản trị trước khi cho phép thao tác. Hậu điều kiện là cấu hình kênh và quyền truy cập được lưu lại, đồng thời danh sách kênh hiển thị cho mỗi thành viên được lọc theo quyền tương ứng.

A.2.3 UC003 – Đăng tải và kiểm duyệt game

Use case này mô tả quy trình Nhà phát triển đưa game lên hệ thống. Nhà phát triển tạo metadata của game, tải lên hình ảnh và file build dạng zip. Hệ thống kiểm tra quyền sở hữu game, định dạng file, cấu trúc bundle và sự tồn tại của file `index.html`. Khi build hợp lệ, hệ thống tạo đường dẫn chơi game và cho phép gửi game vào hàng đợi kiểm duyệt. Quản trị viên xem xét nội dung và chuyển trạng thái game sang công bố, từ chối hoặc tạm ngừng. Hậu điều kiện thành công là game xuất hiện trên chợ game và có thể được người dùng truy cập để chơi.

A.2.4 UC004 – Tổ chức giải đấu

Người tổ chức tạo giải đấu trong phạm vi máy chủ, nhập thông tin game, thể thức thi đấu, loại người tham gia, số lượng người chơi hoặc đội, luật và giải thưởng. Hệ thống kiểm tra quyền tổ chức, tạo giải đấu ở trạng thái nháp và tạo kênh chung cho giải đấu. Sau đó, Người tổ chức lần lượt mở đăng ký, chuyển sang check-in và bắt đầu giải đấu. Với giải đấu đội, Đội trưởng tạo đội và quản lý thành viên đội. Khi giải đấu bắt đầu, hệ thống sinh bracket và danh sách trận đấu, tạo cơ sở cho use case vận hành trận đấu.

A.2.5 UC005 – Vận hành trận đấu và xử lý kết quả

Use case này diễn ra khi giải đấu đã bắt đầu. Người tổ chức hoặc Quản trị giải đấu có thể bắt đầu một trận đấu hợp lệ. Hệ thống chuyển trạng thái trận, tạo workspace gồm các kênh phục vụ đội thi đấu, trọng tài và livestream, sau đó phân quyền truy cập cho các vai trò liên quan. Người tham gia báo cáo kết quả sau trận; nếu có khiếu nại, Trọng tài hoặc Quản trị giải đấu xem xét và ghi đè kết quả khi cần. Hậu điều kiện là kết quả trận đấu, bracket, bảng xếp hạng và trạng thái các trận tiếp theo được cập nhật nhất quán.

A.3 Bảng trường dữ liệu tham khảo

Các trường dữ liệu tham khảo trong đặc tả use case			
Use case	Trường dữ liệu	Mô tả	Điều kiện hợp lệ
UC001	server_id	Định danh máy chủ người dùng muốn tham gia	Máy chủ tồn tại và người dùng chưa là thành viên
UC001	note	Ghi chú trong yêu cầu tham gia	Có thể rỗng, được chuẩn hóa trước khi lưu
UC002	name	Tên kênh	Không rỗng, độ dài nằm trong giới hạn hệ thống
UC002	type	Loại kênh	Thuộc nhóm văn bản, thoại, danh mục hoặc livestream
UC002	allow_bits, deny_bits	Bit quyền cho phép hoặc từ chối	Là giá trị bitset hợp lệ của hệ thống phân quyền
UC003	slug	Định danh thân thiện URL của game	Duy nhất và được chuẩn hóa dạng kebab-case
UC003	file	File build game	Là file zip hợp lệ và chứa index.html
UC004	format	Thể thức giải đấu	Thuộc danh sách thể thức hệ thống hỗ trợ
UC004	participant_type	Loại tham gia	Cá nhân hoặc đội
UC005	winner_id	Người chơi hoặc đội thắng trận	Phải thuộc danh sách participant của trận đấu
UC005	score1, score2	Điểm số hai bên	Là số nguyên không âm

Bảng A.2: Các trường dữ liệu tham khảo trong đặc tả use case